Data-Efficient AI-Guided Energy- and Thermal-Aware Scheduling on Heterogeneous Multicore Systems

by

Mohammad Pivezhandi

DISSERTATION

Submitted to the Graduate School

of Wayne State University,

Detroit, Michigan

in partial fulfillment of the requirements

for the degree of

DOCTOR OF PHILOSOPHY

2025

MAJOR: COMPUTER SCIENCE

Approved By:

_____
Dr. Abusayeed Saifullah, Ph.D. Advisor Date

_____
Dr. Ali Jannesari , Date

_____
Dr. Prashant Modekurthy Date

_____
Dr. Zheng Dong Date

_____

DEDICATION

Dedicated to my cherished family.

ACKNOWLEDGEMENTS

I extend heartfelt gratitude to my advisor, Dr. Abusayeed Saifullah, for his unwavering guidance, encouragement, and support over the past five years. His insightful suggestions and motivation were crucial to the completion of this dissertation. I am also deeply thankful to Professor Ali Jannesari and Dr. Prashant Modekurthy for their valuable advice and collaborative spirit throughout my Ph.D. journey. Additionally, I appreciate Dr. Zheng Dong for graciously agreeing to serve on my Ph.D. committee. I would also like to thank Mahdi Banisharif Dehkordi and Saeed Bakhshan for their help in technical parts of the project, including the projects on zero-shot reinforcement learning development. I am also grateful to my fiancée, Arefeh, for her love and encouragement. Above all, I am profoundly grateful to my family, particularly my father and my mother, for their unwavering emotional support during my doctoral studies.

TABLE OF CONTENTS

Dedication ii

Acknowledgements iii

List of Tables x

List of Figures xi

Chapter 1 Introduction 1

Chapter 2 HiDVFS: A Hierarchical Multi-Agent DVFS Scheduler for OpenMP DAG Workloads 4

2.1 Introduction 1

2.2 Background and Motivation 5

2.2.1 Background 5

2.2.2 Inefficiency of Current Task-to-Core Allocations 7

2.2.3 Irregular Parallel Execution of DAGs and Dependency on Performance Profiling Features 10

2.3 Related Work 12

2.4 Design of HiDVFS 14

2.4.1 Collaborative MARL for High-Dimensional Spaces 15

2.4.2 Enhanced Reward Estimation 15

2.4.3 Reward Function Estimation 16

2.4.4 Hyperparameter Tuning and Target Metrics 18

2.4.5 Complexity Analysis of Agents 20

2.4.6 Hierarchical MARL Algorithm with Reward Model 21

2.4.7 Practical Considerations: Platform and Parallel Execution 22

2.5 Experimental Platform, Benchmark, and Evaluation 24

2.5.1 Platforms and Setup 24

2.5.2 Evaluation of Metrics and Statistical Analysis of Features 26

2.5.3 Implemented Approaches 29

2.5.4 Comparison with Baselines and Results 29

2.6 Conclusion 31

Chapter 3 Feature-Aware Task-to-Core Allocation in Embedded Multi-core Platforms via Statistical Learning 37

3.1 Introduction 1

3.2 Motivation and Challenges 5

3.2.1 Curse of Dimensionality 5

3.2.2 Large Data Management 6

3.2.3 Performance Unpredictability 6

3.3 Design Methodology 7

3.3.1 Data Collection and Environment Setup 8

3.3.2 Embedded Feature Selection Using Random Forest 10

3.3.3 Wrapper-Based Feature Selection 12

3.3.4 Filter-Based Feature Selection 16

3.3.5 Summary of the Multi-Stage Methodology 18

3.4 Experiments 19

3.4.1 Experimental Platform, Benchmarks, and Evaluation 20

3.4.2 Implementation and Training Details 21

3.4.3 Empirical Results 22

3.4.4 Comparative Model Analysis 24

3.5 Related Work 26

3.6 Conclusion 28

Chapter 4 ZeroDVFS: Zero-Shot LLM-Guided Core and Frequency Allocation for Embedded Platforms 35

4.1 Introduction 1

4.2 Related Work 6

4.3 Background and Motivation 7

4.3.1 Background 7

4.3.2 Motivation on Model-Based MARL 10

4.3.3 Limitations of Utilization-Based DVFS 11

4.3.4 Challenges in Task-to-Core Allocation 11

4.3.5 The Need for Environment Modeling and Multi-Agent RL 13

4.4 Design of Model-Based Thermal- Energy-Aware MARL 14

4.4.1 Design of MARL 14

4.4.2 Environment Design and Modeling 20

4.5 Experimental Results 26

4.5.1 Implemented Algorithms 27

4.5.2 Experimental Platform, Benchmark, and Evaluation 29

4.5.3 Temperature and Performance Prediction 30

4.6 Conclusion 32

Chapter 5 FlowRL: Flow-Augmented Few-Shot Reinforcement Learning for Semi-Structured Sensor Data 42

5.1 Introduction 1

5.2 Preliminaries 2

5.2.1 Q-learning and Online RL 3

5.2.2 Regret Bounds and Sample Efficiency 4

5.2.3 Flow Matching for Generative Modeling 5

5.3 Design of Distribution-Aware Flow Matching for Data Generation 5

5.3.1 Redefinition of the Regret Function 6

5.3.2 Flow Matching and Bootstrapping 6

5.3.3 Feature Weighting and Conditional Flow Matching 7

5.3.4 Implications for RL: Regret Bounds under Model-Based Assumptions . 9

5.3.5 Implications for RL: Regret Bounds with Synthetic Data in Model-Free RL 10

5.3.6 From LLN to Finite-Sample Guarantees 11

5.3.7 Algorithmic Overview for Few-Shot Online RL 12

5.4 Experiments 13

5.5 Related Work 20

5.6 Conclusion 22

Chapter 6 GraphPerf-RT: A Graph-Driven Performance Model for Hardware-Aware Scheduling of OpenMP Codes 23

6.1 Introduction 1

6.2 Related Work 2

6.3 Preliminaries 4

6.3.1 Problem Formulation 4

6.3.2 Heterogeneous Graph Abstraction 5

6.3.3 Data and Artifact Pipeline 6

6.3.4 Learning and Outputs 6

6.4 Design Methodology 7

6.4.1 Problem Formulation 7

6.4.2 Heterogeneous Graph Representation 8

6.4.3 Graph Neural Network Architecture 12

6.4.4 Evidential Learning for Uncertainty Quantification 14

6.4.5 Training Procedure 16

6.4.6 Training Procedure 17

6.5 Experiments 17

6.5.1 Experimental Setup 17

6.5.2 Baselines and Metrics 19

6.5.3 Results and Analysis 20

6.5.4 Integration with Scheduling and RL 21

6.5.5 Discussion 22

6.6 Conclusion 22

Chapter 7 Conclusion 23

7.1 Summary of Contributions 23

7.2 Integrated Framework and Validation 25

7.3 Broader Impact 25

7.4 Future Directions 26

Abstract 47

Autobiographical Statement 50

LIST OF TABLES

2.1 Agent States and Actions 18
2.2 Statistical Analysis of Key Features' Influence on Performance Metrics 28
2.3 Comparison of Proposed RL Approaches (HiDVFS_S, HiDVFS) Against SOTA RL Baselines 31

3.1 Key features ranked by Random Forest and backward stepwise OLS (lowest CV Error) for Profiler and Temperature datasets. Temp Core 0-13 denotes per-core temperatures; ΔTemp is the temperature difference. 23
3.2 Temperature predictors (θ_i) vs. energy: Estimates (Est.), t-values (t-val.), p-values (p-val.), sorted by p-value. Significant predictors (e.g., θ_1, θ_0, θ_2) show strong energy impact. 24
3.3 Profiler predictors (excl. temp, freq, speed) vs. energy: Estimates (Est.), t-values (t-val.), p-values (p-val.), sorted by p-value. Key predictors (e.g., Context Switch Rate, Elapsed Time) drive energy variance. 25
3.4 MSE percentage and total number of parameters for different architectures on Intel Core i7 12th Gen. 26

4.1 Comparing profiler and temperature models regarding accuracy, inference latency (mean ± std), and complexity. Inference latency measured on Jetson TX2. 31

5.1 Main Hyperparameters and Tuning Values. 15

6.1 Overall performance across all platforms and benchmarks. Lower is better for RMSE/MAE/MAPE. Higher is better for R^2/Spearman. 18
6.2 Uncertainty calibration summary (lower ECE/MCE is better; higher reliability is better). 21

LIST OF FIGURES

2.1 OpenMP DAG snippet showing tied/untied tasks and dependency-induced variability in execution time. 8

2.2 Monitoring one user DAG (τ_1): five jobs ($J_{1,1,1}$–$J_{3,1,4}$) mapped to cores (c_1–c_3) with target frequencies (f_0–f_2). Core c_0 runs system DAG (τ_0). 9

2.3 `N-Queens` on Xeon (12 cores) and Core i7 (4 cores). More cores increase branch misses for `tied` tasks, raising makespan and energy. Shaded regions show variation over 10 runs; serial reduces misses. 9

2.4 Scenarios on a four-core system. Blue: `performance`; green: `powersave`. PP, PS denote unbounded parallel execution; SP, SS denote bounded serial execution. The green dashed line marks DAG makespan. HiDVFS agents (7–9) align core choice, frequency, and temperature. 10

2.5 Importance of different features on total energy consumption and makespan in parallel and sequential execution of parallel applications. 11

2.6 IRL (A) aligns with expert behavior; our model (B) uses key-state predictions to shape rewards. 19

2.7 Profiling data under parallel and sequential modes for FFT workload variations (untied, tied, serial) show more spontaneity for profiling data in parallel mode. . 33

2.8 Variation of total energy consumption and makespan due to changes in priority combinations, number of cores, and frequency levels in parallel and sequential modes. 34

2.9 Training over 60 epochs: (a) HiDVFS_S vs. zTT/GearDVFS stabilizes by epoch 12; (b) HiDVFS vs. MAMB/MAMF shows faster makespan/frequency stabilization. 35

2.10 Observations over 60 epochs mirror makespan: HiDVFS achieves earlier reward stabilization; HiDVFS_S follows within a few epochs. 36

3.1 Energy consumption variation across three different processors with identical frequency and core count settings: Intel Core i7 8th Gen (Corei78) with 4 cores, Intel Core i7 12th Gen (Corei712) with 14 cores, and Intel Xeon 2680 v3 (Xeon) with 12 cores. The results are shown for three different OpenMP benchmarks. 7

3.2 Determining the significance of features using Random Forest with respect to cross-validation error on Intel Core i7 12th Gen. 13

3.3 Backward stepwise selection for estimating energy consumption and average temperature. Retaining fewer than 8 predictors (features) yields accurate predictions in both cases. Experiments performed on Intel Core i7 12th Gen. . . . 29

3.4 Correlation matrix based on Pearson correlation coefficients for 10 selected cores from an Intel Core i7 12th Gen processor with 14 cores. 30

3.5 Comparison of average temperature and energy consumption for correlation-based (Corr) and random (Rand) core selection. Experiments performed on Intel Core i7 12th Gen processor with 14 cores. 31

3.6 Results for temperature prediction from ground truth on Intel Core i7 12th Gen for regular FCN and FCN via Random Forest feature reduction strategy. 32

3.7 Results for profiler data prediction from ground truth on Intel Core i7 12th Gen for regular FCN and FCN via Random Forest feature reduction strategy. 33

3.8 Comparison of FCN models with and without feature selection and bootstrapping on Intel Core i7 12th Gen. 34

4.1 Performance metrics (CPU utilization, makespan, energy consumption, branch misses, context switches, and average temperature) for NVIDIA Jetson TX2 running the FFT benchmark paralleled through OpenMP API. Results highlight different responses of performance metrics to changes in different frequencies and selected cores. 34

4.2 Hierarchical MARL Action Selection with Two Agents: The Profiler Agent determines the number of active cores and the operating frequency based on performance and energy consumption states. The Temperature Agent assigns priority levels to cores based on temperature states. The combined actions result in prioritized cores and the appropriate number allocated to tasks, reducing computational complexity and decision latency. 35

4.3 Reward function definitions for (a) the Profiler Agent and (b) the Temperature Agent. **(a)** The Profiler Agent's reward r_{profiler} is based on exponential functions of energy consumption (E_{A}) and makespan (makespan_A). Here, a threshold factor ($c_{\text{th}} = 0.3$) and a steepness factor ($c_{\text{st}} = 0.5$) determine the reward focus. **(b)** The Temperature Agent's reward r_{temp} changes linearly based on core temperature (temp_i), aiming to maintain temperatures below the threshold of 50°C. . . 36

4.4 The simplified design of model-based RL for temperature- and energy-aware core allocation for multi-core processors. 37

4.5 Comparison of some of the profiler data prediction using the one dimensional convolution model and the sensor data as the ground truth on Intel Core i7 8th gen with 4 cores. 38

4.6 Comparison of temperature prediction of 4 cores in an Intel Core i7 8th gen. . . 39

4.7 Comparison of performance metrics for three RL approaches (DynaQ, zTT, and PlanGAN) across different time steps, including makespan, power, energy, temperature, Q-value, action frequency, core combination, and average reward. 40

4.8 Performance comparison of three RL approaches (MAMB, MAMF, and MAMB D3QN) across several metrics including makespan, power, energy, temperature, Q-value, action frequency, core combination, and average reward over time. 41

5.1 Workflow of the proposed Flow-Augmented Reinforcement Learning (FARL) approach. 14

5.2 Correlation matrix comparisons. 17

5.3 Performance comparisons across methods. 18

5.4 Real and (Gen)erated data t-SNE. 18

5.5 Comparisons of distributions of FARL and Model-Based (MB) generated data. . 19

6.1 End-to-end pipeline. 8

CHAPTER 1 INTRODUCTION

The relentless demand for energy efficiency in multi-core embedded systems, fueled by applications such as IoT devices, autonomous vehicles, and mobile systems, has exposed critical hardware and software challenges that necessitate innovative solutions. As chip technology scales, static leakage power, which grows exponentially with temperature, often surpasses 63% of total power consumption [115], causing overheating that undermines system reliability and performance [44]. Traditional methods, such as Linux governors [17], falter in addressing per-core thermal dynamics [30], irregular execution patterns in parallel workloads like Directed Acyclic Graphs (DAGs) [15], and the high-dimensional action spaces of many-core systems [57]. These approaches frequently overlook runtime profiling data, such as branch misses or memory accesses [5], and lack scalability across heterogeneous platforms (e.g., CPUs, GPUs, ARM) [68], especially in resource-constrained environments. Moreover, the high cost of collecting real-world profiling data [43] and the complexity of generating diverse unstructured data for few-shot learning [79, 83, 27] exacerbate challenges in efficient core allocation, frequency scaling, and thermal management.

This dissertation presents a comprehensive AI-driven framework that addresses these challenges through five interconnected contributions, each detailed in a dedicated chapter. Chapter 2 introduces HiDVFS, a hierarchical multi-agent reinforcement learning (HMARL) framework for energy- and thermal-aware scheduling of parallel OpenMP DAG workloads. By leveraging per-core temperature monitoring and runtime profiling to optimize task allocation and frequency scaling, HiDVFS achieves a 31.7% reduction in energy consumption and a 34.1%

improvement in task completion time on the NVIDIA Jetson TX2 platform [15, 84, 94]. Chapter 3 presents a hybrid statistical learning approach for task-to-core allocation, combining Random Forest, backward stepwise selection, and Pearson correlation analysis to identify critical features such as core type, temperature, and application parallelism. This approach reduces energy consumption by 10% and core temperature by 5°C while cutting thermal prediction error by 61.6% across embedded platforms [90, 74, 20, 29, 82, 75, 85].

Chapter 4 presents ZeroDVFS, a model-based multi-agent reinforcement learning framework that extends the HMARL approach to handle continuous action spaces for real-time dynamic voltage and frequency scaling (DVFS). By leveraging pre-trained environment models, ZeroDVFS achieves a 7x improvement in thermal prediction accuracy [43] and enables zero-shot transfer across heterogeneous platforms without additional training. Chapter 5 describes a distribution-aware flow matching method to generate synthetic unstructured data for few-shot reinforcement learning, improving frame rates by 30% in DVFS applications [65, 34, 41, 39, 80]. This method addresses the critical challenge of data scarcity in embedded systems where collecting real-world profiling data is costly and time-consuming. Chapter 6 introduces GraphPerf-RT, a graph neural network-driven performance model that analyzes control flow graphs to predict execution characteristics and optimize OpenMP code variation selection (e.g., tied, untied, serial) for minimal makespan. By combining graph-based code representation with hardware-aware performance modeling, GraphPerf-RT enables intelligent scheduling decisions across diverse platforms.

Together, these contributions form an integrated workflow where statistical feature selec-

tion drives task allocation, flow matching augments training data, hierarchical agents optimize DVFS and scheduling, model-based learning enables platform transfer, and graph-driven analysis guides code optimization. Validated on the BOTS benchmark suite across NVIDIA Jetson TX2, Intel Core i7, and Intel Xeon platforms, the unified framework outperforms Linux governors and existing reinforcement learning schedulers that lack per-core thermal awareness [17, 30]. The framework advances energy-efficient embedded systems for IoT, automotive, and mobile applications by reducing cooling costs (approximately $3 per watt [35]), mitigating thermal hotspots, and enhancing system reliability and longevity.

The remainder of this dissertation is organized as follows: Chapter 2 details the HiDVFS hierarchical multi-agent reinforcement learning framework; Chapter 3 presents the statistical feature selection approach for task-to-core allocation; Chapter 4 introduces ZeroDVFS for model-based DVFS optimization; Chapter 5 describes the distribution-aware flow matching method for data augmentation; Chapter 6 presents GraphPerf-RT for graph-driven performance modeling; and Chapter 7 concludes with a summary of contributions and future research directions.

CHAPTER 2 HIDVFS: A HIERARCHICAL MULTI-AGENT DVFS SCHEDULER FOR OPENMP DAG WORKLOADS

Abstract

With advancements in multicore embedded systems, leakage power, exponentially tied to chip temperature, has surpassed dynamic power consumption. Energy-aware solutions use dynamic voltage and frequency scaling (DVFS) to mitigate overheating in performance-intensive scenarios, while software approaches allocate high-utilization tasks across core configurations in parallel systems to reduce power. However, existing heuristics lack per-core frequency monitoring, failing to address overheating from uneven core activity, and task assignments without detailed profiling overlook irregular execution patterns. We target OpenMP DAG workloads. Because makespan, energy, and thermal goals often conflict within a single benchmark, this work prioritizes performance (makespan) while reporting energy and thermal as secondary outcomes. To overcome these issues, we propose HiDVFS (a hierarchical multi-agent, performance-aware DVFS scheduler) for parallel systems that optimizes task allocation based on profiling data, core temperatures, and makespan-first objectives. It employs three agents: one selects cores and frequencies using profiler data, another manages core combinations via temperature sensors, and a third sets task priorities during resource contention. A makespan-focused reward with energy and temperature regularizers estimates future states and enhances sample efficiency. Experiments on the NVIDIA Jetson TX2 using the FFT benchmark from the BOTS suite, an irregular OpenMP DAG workload, compare HiDVFS and our proposed methods against state-of-the-art approaches. HiDVFS achieves a 49.06% makespan

reduction and 40.95% energy reduction over a multi-agent baseline, while our single-agent method reduces makespan by 43.17% and energy by 44.24% over a single-agent baseline, converging in 12 epochs.

2.1 Introduction

With the rapid advancement of computing technologies, energy efficiency has become a critical concern in the design and operation of modern embedded systems. As technology feature sizes continue to shrink, static leakage power, which is directly affected by temperature, increasingly dominates the total power consumption of multicore and many-core embedded systems. Static leakage power can rise from roughly 22% to over 63% of the total power with a halving of the technology scale [115]. This phenomenon, coupled with the growing demand for energy-friendly parallel and distributed processing, has introduced new hardware and software challenges. On the hardware side, scaling voltages and frequencies has been proposed to address overheating and thermal throttling caused by aggressive performance boosting [17]. This involves assigning clusters of cores to designated frequency scales based on temperature limits and core utilization behavior, ensuring enhanced performance while managing power consumption. Meanwhile, software solutions, such as energy-aware scheduling policies, have been developed to efficiently allocate tasks across multiple cores in both parallel and distributed systems [51, 58]. We focus on OpenMP DAG workloads and adopt a makespan-first objective, reporting energy and thermal behavior as secondary outcomes.

Current hardware solutions for processors equipped with Dynamic Voltage and Frequency Scaling (DVFS) do not adequately address the need for per-core frequency tracking and adjustment based on thermal behavior [30]. Similarly, existing software solutions fail to effectively handle runtime features like branch misses and memory accesses in parallel and distributed tasks, resulting in irregularities and unpredictable execution times within subtasks of Directed

Acyclic Graph (DAG) workloads. This leads to inefficient core allocation and suboptimal performance in both parallel and distributed environments. The assignment of tasks to cores in available Linux kernel governors [17] and developed policies [68, 13] is agnostic to each core's temperature, often resulting in high-utilization tasks being allocated to hot cores. These approaches correlate thermal constraints and energy consumption with workload demand rather than considering system profilers' outputs during task execution [5]. Furthermore, existing energy-aware and thermal-aware schedulers face limitations in scalability as the number of cores and frequency levels increase [58, 14], and they lack generalizability across different processor types (e.g., CPU, GPU) and execution environments (e.g., Intel, AMD, ARM) [68]. In distributed systems, these limitations are exacerbated by the additional complexity of managing resources across multiple nodes. Hence, a general solution must effectively balance workload characteristics with device-specific task-to-processor mappings for enhanced energy and performance efficiency in both parallel and distributed computing environments.

This paper introduces **HiDVFS**, a hierarchical multi-agent, performance-aware DVFS scheduler for OpenMP DAGs that prioritizes makespan with energy and temperature as regularizers, a novel approach for parallel DAGs to tackle the NP-hard challenge of optimizing performance and energy consumption in real-time multicore systems [10]. Unlike existing non-learning heuristics, which are limited to specific scenarios such as lightly loaded multicore systems with more cores than tasks [58], and traditional reinforcement learning (RL) methods that face long training times, high computational overhead, and scalability issues due to coarse frequency scaling and high-dimensional action spaces [66, 88, 30, 102, 14, 53, 64], HiDVFS employs

joint action learners (JAL) to make collaborative decision [57]. These agents collaboratively optimize core frequency, core allocation, and temperature-aware core combinations, reducing sample requirements and mitigating overestimation issues [114]. A makespan-focused reward with energy and temperature regularizers and temporal shaping further reduces computational complexity and latency, addressing shortcomings of prior work [64, 124], and enabling low-overhead few-shot learning for DVFS in parallel workloads. Evaluated on the NVIDIA Jetson TX2 using the FFT benchmark from the Barcelona OpenMP Tasks (BOTS) suite [15], HiDVFS demonstrates practical feasibility by addressing deployment challenges such as computational overhead and hyperparameter tuning, outperforming RL-based SOTA in energy and makespan optimization for real-time OpenMP DAG workloads.

We design a parallel DAG scheduler that assigns each application, represented as a DAG, a priority, core count, and frequency, creating tailored combinations based on parallelism levels. This approach suits real-time and distributed scenarios with concurrent applications, unlike sequential DAG executions. By analyzing features and actions impacting energy consumption and makespan, we refine reward function modeling for multi-agent systems. We implement parallel and distributed DAGs on a Linux platform, using online learning to optimize priority, core, and frequency assignments based on diverse runtime observations.

We assess thermal reliability and power management using authentic workloads and Linux in-kernel profiling, surpassing synthetic workloads [84] or control flow graphs [94], which lack runtime monitoring. Our method employs unbiased task distribution, online temperature monitoring, and CPU profiler data to optimize resource use and reduce thermal-induced power

consumption in parallel and distributed systems. Actions involve frequency and core selection tailored to workload behavior. Using OpenMP API [15] tasks to represent irregular workloads, our online learning algorithm ensures low-complexity resource allocation for DAG benchmarks while maintaining thermal feasibility under temperature-aware core grouping constraints.

The contributions of this paper are as follows:

1. **HiDVFS**, a scheduling framework using runtime temperature profiling and a predictive reward function to optimize energy, makespan, and thermal constraints for OpenMP DAGs.

2. Analysis of runtime profiler data to drive real-time scheduling decisions, employing multiple agents to adjust core counts, frequencies, and DAG priorities.

3. Evaluation on the NVIDIA Jetson TX2 using the FFT benchmark from the BOTS suite, achieving a 49.06% makespan reduction and 40.95% energy reduction compared to a multi-agent baseline, while a single-agent variant reduces makespan by 43.17% and energy by 44.24% compared to a single-agent baseline, converging in 12 epochs.

In the remainder of this paper, Section 5.2 provides background on the task model and the motivation for the proposed approach in DVFS and task-to-core allocation. Section 2.3 gives an overview of related work. Section 5.3 details HiDVFS. Section 2.5 provides quantitative results to demonstrate its effectiveness.

2.2 Background and Motivation

We propose a fixed-priority, preemptive scheduler that runs multiple parallel tasks based on their priority and allocates a combination of cores, each with a corresponding frequency. Our objective is makespan-first, with energy and temperature as secondary metrics.

2.2.1 Background

Task Model. We consider n aperiodic parallel tasks, $\tau_1, \tau_2, \ldots, \tau_n$, scheduled on a multi-core platform with m heterogeneous cores. Each task consists of multiple subtasks, represented as a Directed Acyclic Graph (DAG), such that $\tau_i = \{\tau_{i,1}, \tau_{i,2}, \ldots, \tau_{i,j}\}$. Each node in the DAG corresponds to a subtask (a thread of execution), and a directed edge signifies the dependency between two subtasks. Granularity varies from individual threads to larger functions, as specified by OpenMP directives in our BOTS benchmarks (Section 2.5). Each execution instance of the tasks is a *job*, denoted by $J_{k,i,j}$, where k is the core, i the DAG index, and j the subtask index. Each task τ_i has a real-time priority; higher-priority tasks preempt lower-priority ones.

For precision, we define a DAG task τ_i as a tuple (V_i, E_i, P_i), where V_i is the set of subtasks, $E_i \subseteq V_i \times V_i$ the dependencies, and $P_i \in [1, 99]$ the static priority. An example of precedence-constrained jobs appears in Figure 2.2. The execution of subtask $\tau_{i,1}$ by jobs $J_{1,i,1}$ and $J_{2,i,1}$ on cores c_1 and c_2 precedes subtask $\tau_{i,2}$ on c_3. The DAG makespan is the completion time of all jobs respecting dependencies. Our primary performance metric is makespan. Benchmarks are BOTS workloads such as Strassen and FFT, parallelized through OpenMP [31].

DVFS. Power, temperature, and performance depend on voltage/frequency. Linux governors (`ondemand`, `conservative`, `schedutil`) adjust frequencies using utilization heuris-

tics [17, 64]. These governors lack per-core, temperature-aware control and are not DAG/-makespan oriented.

Environment Design. We assume multiple parallel DAGs, each with a priority, running on selected core combinations and frequencies. Real-time DAGs map to cores according to priority $1-99$; equal-priority DAGs are FCFS. Higher-priority DAGs preempt lower-priority ones. We test heterogeneous and homogeneous platforms (Xeon 2680 V3, Intel Core i7 8th/12th gen, Jetson TX2). Jetson TX2 is used for experiments due to fine-grained, software-controlled frequency scaling. HiDVFS assigns (i) core counts and frequencies, (ii) temperature-aware core groupings, and (iii) static DAG priorities before execution.

Profiler and Task-to-Core Allocation. As in Figure 2.2, we group cores into clusters using *cgroup*/`cpuset` to control governors and frequency boundaries. Tasks are bound to clusters for energy/thermal control. We use `perf` for per-task execution profiling, `cpufreq-info` for DVFS state, and `sensors` for per-cluster temperatures. These signals feed HiDVFS agents for makespan-first decisions.

Online Learning Through RL. We employ a hierarchical multi-agent off-policy, value-based RL. Three agents cooperate: a profiler agent selects (core count, frequency), a thermal agent selects core combinations using temperatures, and a priority agent selects task priorities. The DQN backbone uses a replay buffer and target network to stabilize training and reduce overestimation; advantage–value decomposition normalizes advantages. Rewards are shaped for makespan with energy and temperature regularizers.

2.2.2 Inefficiency of Current Task-to-Core Allocations

We explore temperature-aware task-to-core assignment to cut energy and makespan on OpenMP DAGs. Irregular task behavior impacts latency and thermal reliability; we outline the effect and mitigations.

Impact of Unpredictable Task Execution Time on DAG Makespan. Figure 2.1 shows an example where `#pragma omp parallel` creates four threads. The `tied` task $\tau^*_{i,1}$ is confined to its cores; `untied` tasks $\tau_{i,3}, \tau_{i,2}, \tau_{i,0}$ may migrate. Branch mispredictions due to L_1/L_2 loops increase makespan variance. Dependencies propagate timing jitter; $\tau^*_{i,1}$ dominates the critical path.

The behavior of $\tau^*_{i,1}$ is microarchitecture- and core-count-dependent. Parallelism increases shared-resource pressure, causing stalls and mispredictions. More cores for `tied` tasks can worsen makespan and energy (Figure 2.3). This argues for demand-aware core allocation.

Design of Demand-Based Task-to-Core Allocation. Figure 2.3 shows that blindly increasing cores raises energy and makespan due to branch effects. Figure 2.4 sketches scenarios for a four-core system under `performance` (P) and `powersave` (S) governors. Blue blocks run at high frequency; green at low. Numbers denote hypothetical times.

Scenarios 1–6 vary by governor and predictability. Ignoring workload structure yields unstable makespan and possible throttling. Scenarios 7–9 illustrate **HiDVFS**. The thermal agent prioritizes cold cores and selects temperature-aware core combinations. The profiler agent picks core counts and frequency levels to keep the critical path predictable (e.g., one core for $\tau^*_{i,1}$). The priority agent orders DAGs to prevent contention spikes. This yields lower makespan,

```
1  #pragma omp parallel num_threads(4)
2  {
3     #pragma omp master
4     {
5     #pragma omp task // τ_{i,0}
6     { /* part 0 */ }
7     #pragma omp task depend(out: x)
8     final(true) // τ*_{i,1}
9     {
10        #pragma omp parallel for
11        for (int i = 0; i < L1; i++) {
12            if(i % 2 == 0)
13                {/* work */}
14            else{for (int j = 0; j < L2; j++)
15                {/* work */}}
16        }
17    }
18    #pragma omp task depend(in: x) // τ_{i,2}
19    { /* part 2 */ }
20    #pragma omp task // τ_{i,3}
21    { /* part 3 */ }
22    #pragma omp taskwait
23  }
24 }
```

Figure 2.1: OpenMP DAG snippet showing tied/untied tasks and dependency-induced variability in execution time.

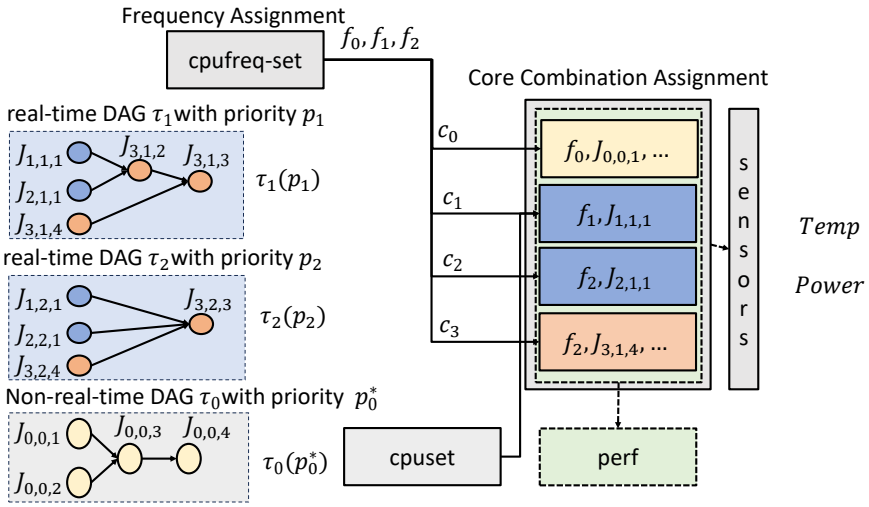


Figure 2.2: Monitoring one user DAG (τ_1): five jobs ($J_{1,1,1}$–$J_{3,1,4}$) mapped to cores (c_1–c_3) with target frequencies (f_0–f_2). Core c_0 runs system DAG (τ_0).

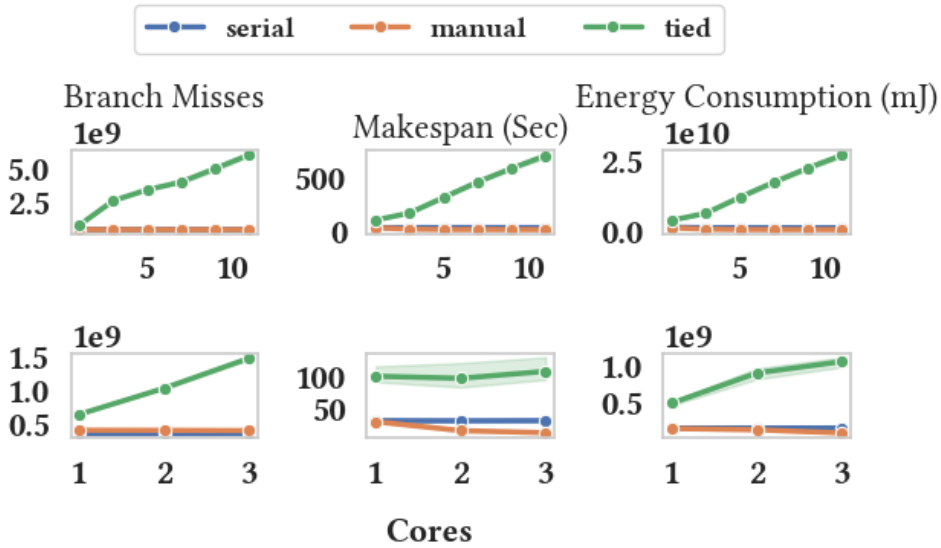


Figure 2.3: `N-Queens` on Xeon (12 cores) and Core i7 (4 cores). More cores increase branch misses for `tied` tasks, raising makespan and energy. Shaded regions show variation over 10 runs; serial reduces misses.

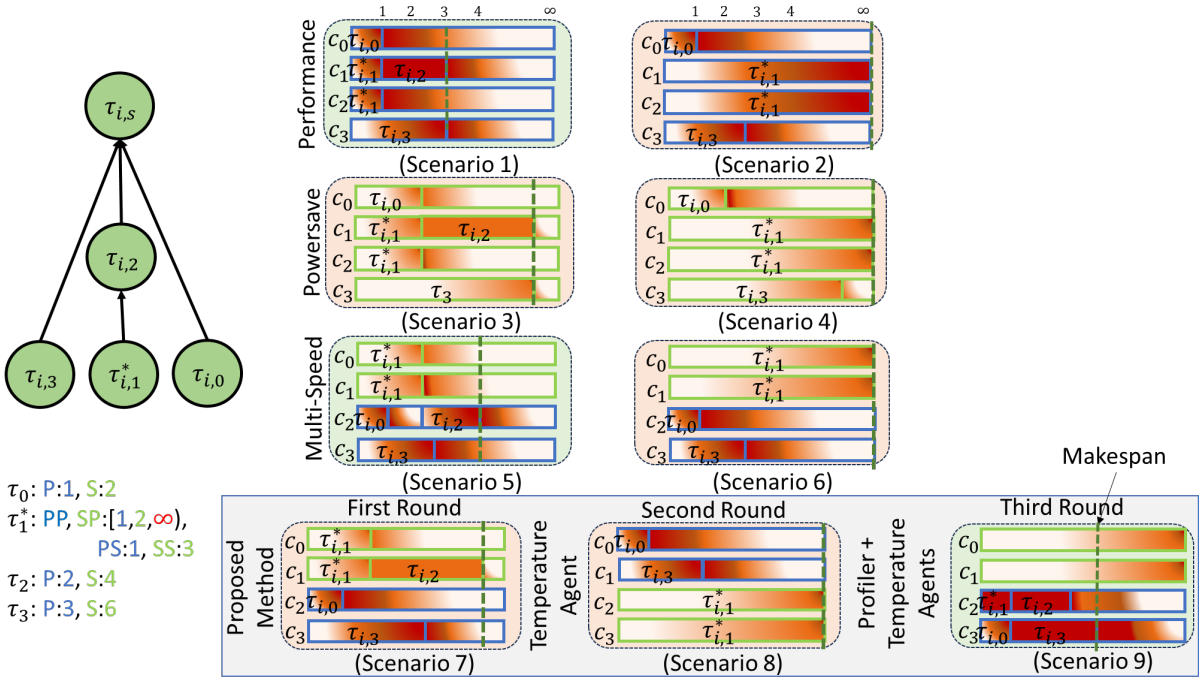


Figure 2.4: Scenarios on a four-core system. Blue: `performance`; green: `powersave`. PP, PS denote unbounded parallel execution; SP, SS denote bounded serial execution. The green dashed line marks DAG makespan. HiDVFS agents (7–9) align core choice, frequency, and temperature.

lower energy, and idle-core creation via targeted allocation.

2.2.3 Irregular Parallel Execution of DAGs and Dependency on Performance Profiling Features

Feature screening (Fig. 2.5) shows parallel DAGs depend on a broader set of runtime features than sequential runs; temperature, frequency, utilization, and miss events are most predictive of makespan/energy. This supports using profiler/sensors to steer per-core DVFS and core grouping.

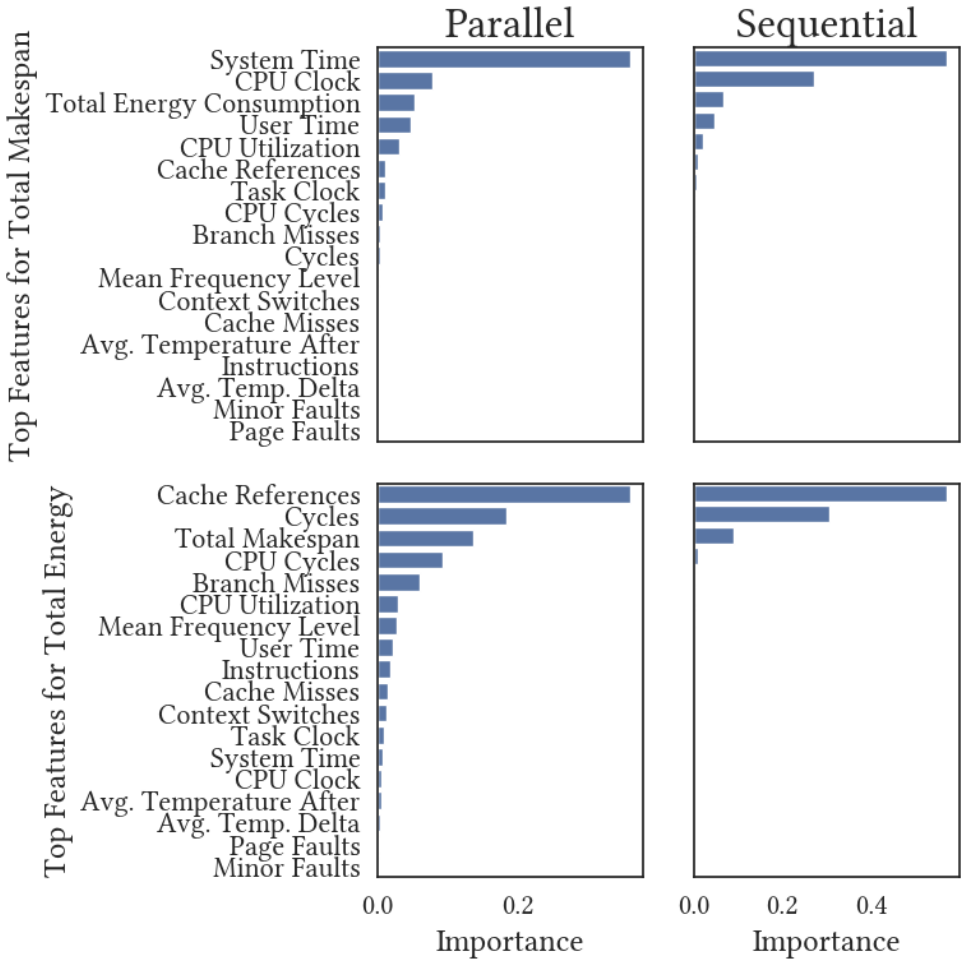


Figure 2.5: Importance of different features on total energy consumption and makespan in parallel and sequential execution of parallel applications.

2.3 Related Work

The current energy- and thermal-aware multi-core parallel scheduling algorithms are based on heuristics, meta-heuristics, integer programming, and machine learning approaches [112]. The existing heuristics, meta-heuristics, and integer programming algorithms are application-specific and cannot be generalized. Since the emerging machine learning approach is data-oriented, it can be generalized to various workloads, platforms, and applications with multiple objectives such as energy efficiency, thermal management, and latency. However, these methods seldom target OpenMP DAGs with a makespan-first objective or expose per-core, temperature-aware DVFS needed for tied/untied tasks.

Data-oriented machine learning design for energy, thermal, and latency management in multi-core processors has been studied recently [30, 74, 66, 103, 14]. A review on energy, thermal, and latency management of multi-core processors using learning-based designs is available in [74]. However, the existing work mainly focuses on extending traditional supervised, unsupervised, and semi-supervised design methods to energy, thermal, and latency management, regardless of runtime design constraints. The high overhead of inference and training makes most previous frameworks useless. Besides, existing work extends the traditional Q-learning approach, increasing the training time and decreasing the action space dimension, hindering the applicability to multi-objective real-time systems [103, 14, 66]. These designs typically optimize average power or throughput rather than end-to-end DAG makespan under embedded runtime limits.

Multi-agent RL and optimizing reward function estimation are two methods to increase the

action space while reducing the number of training iterations [8]. The inverse reinforcement learning (IRL) approach infers the optimal reward function by comparing agent policy with the optimal expert demonstration, leading to high learnability and convergence [8]. However, the current IRL algorithms are computationally demanding, require human expert demonstrations, and are impractical for large state-action space [8]. In our work, we address this limitation by processing only the observation transitions given from the environment. We instead use off-policy value-based agents with a makespan-focused reward and short-horizon model predictions for reward shaping, avoiding expert data.

Efficient task-related and platform-related data help to make accurate decisions, but the existing algorithms on thermal and power management in real-time systems are often built upon limited observations [5, 68, 58, 84, 51, 30]. Many of these approaches consider task-related characteristics to extract the required processor speed and thermal impact, but they might yield imprecise conclusions when dealing with irregular behaviors exhibited by the platform for DAGs [5, 14]. Several works emphasize only historical thermal information or power constraints to make core configuration decisions [68, 30]. We fuse live profiler features with per-cluster temperatures to drive per-core/core-group selection and DVFS for irregular OpenMP DAGs.

When it comes to energy and thermal management, the current implementations often lack the granularity needed for precise decision-making [115, 30, 58, 68, 103]. Many existing strategies on energy- and thermal-aware scheduling employ coarse frequency assignments or resource allocations without the nuanced control required for adjusting individual core fre-

quencies in an embedded context [103, 68]. The works focusing on energy efficiency have an even simpler model that is only sensitive to a few discrete power control actions and a small set of observations. Our hierarchical design separates frequency/core selection, temperature-aware grouping, and static priority to enable per-core DVFS while shrinking the effective action space.

Moreover, the practical applicability of existing energy- and temperature-aware schedulers often comes into question as they primarily target synthetic data and disregard online measurement tools. Contrary to the prior studies based on OpenMP DAG workloads, our paper effectively handles irregular execution behaviors while leveraging runtime profiling. Our work uses Barcelona OpenMP taskset (BOTS), a parallelized workload based on OpenMP API, for training, and we plan to extend it to energy-aware acceleration of ML applications [11]. We therefore evaluate on Jetson TX2 using BOTS `FFT`, prioritizing makespan and reporting energy and temperature as secondary outcomes.

2.4 Design of HiDVFS

This section presents a hierarchical multi-agent RL scheduler for online control of DVFS and task-to-core mapping on multicore processors running OpenMP DAGs. Our objective is to minimize *makespan* while reporting energy and temperature as secondary metrics. We refer to the multi-agent system as **HiDVFS** and to the single-agent variant as **HiDVFS_S**. The framework comprises three cooperative agents: (i) a *profiler* agent that selects frequency and core count, (ii) a *thermal* agent that selects temperature-safe core combinations, and (iii) a *priority* agent that assigns static priorities under contention. This decomposition replaces a

large joint action with three low-dimensional subproblems, improving sample efficiency and stability on embedded hardware.

2.4.1 Collaborative MARL for High-Dimensional Spaces

Optimizing DVFS and task-to-core allocation with a single RL agent can be computationally prohibitive due to the exponentially expanding action space. To overcome this, we employ a hierarchical approach where three agents, each handling a specific sub-problem, collaborate to determine the optimal configuration. The profiler agent selects an appropriate combination of frequency and the number of cores based on performance and energy metrics. The thermal agent adjusts core priorities using temperature clusters to maintain safe operating temperatures. The priority agent selects priority combinations to guide the scheduler toward the desired makespan-first objective.

By distributing actions among multiple agents, we reduce an exponential action space to manageable, linear-scale subspaces. For example, assigning m cores and n frequency levels naively results in an upper bound of m^n combinations. In contrast, using MARL with a thermal agent deciding on cores and a profiler agent choosing frequencies reduces the action space to $m \times n$. Empirical results in Section 5 show this reduces training time by 40% compared to single-agent RL, justifying the use of three agents over one.

2.4.2 Enhanced Reward Estimation

Our off-policy RL approach enhances reward estimation by training a dynamic model of the environment to predict future states, refining reward estimation rather than relying solely on instantaneous interactions. While conventional model-based RL uses the model for planning

future actions, our approach primarily leverages it to refine reward estimation. The learned environment model predicts future states and performance metrics, allowing the reward function to incorporate not only immediate observations but also future outcomes predicted by the model.

This predictive capability enables the reward estimator to consider long-term effects on energy consumption, makespan, and thermal conditions, rather than depending solely on instantaneous signals. By training the dynamic model with both real and synthetic data (generated by the model itself), we achieve few-shot learning and reduce the need for extensive real-world sampling. This corrects prior oversimplification and aligns with D3QN's off-policy nature, improving sample efficiency. Our approach ensures few-shot learning efficiency, as validated in the evaluation on the Jetson TX2 platform.

2.4.3 Reward Function Estimation

Traditional RL schedulers often use static, instantaneous rewards that can be noisy and slow to converge. Suboptimal reward definitions may require numerous iterations and experiments, which is impractical for real-time applications. Imitation learning can reduce the number of iterations by including a policy $\pi(s, a)$ that mimics an expert policy $\pi^*(s, a)$ [1]. Inverse reinforcement learning (IRL) further refines this by inferring a reward function R^* from expert demonstrations [3, 8].

IRL assumes that expert demonstrations are generated by following an optimal policy with an optimal reward function R^*. Given k expert demonstrations $\upsilon_E = \{(s_0^E, a_0^E), (s_1^E, a_1^E), \ldots, (s_k^E, a_k^E)\}$ and l agent policy samples $\upsilon = \{(s_0, a_0), (s_1, a_1), \ldots, (s_l, a_l)\}$, IRL attempts to recover R^*. A

major challenge in IRL is reward ambiguity, where multiple reward functions can produce the observed expert behavior. MaxEnt IRL [128] addresses this by maximizing the entropy of the policy distribution to prevent overfitting to a single solution. However, IRL still heavily relies on expert demonstrations, and poor-quality or suboptimal demonstrations can hinder learning. Moreover, the dependence on expert demonstrations remains a significant limitation.

In this work, we tackle the issue of expert demonstrations by designing a reward function trained using transitions generated by the dynamic environment model alongside real data. Instead of strictly matching expert trajectories, our reward estimation approach utilizes the predictive model to simulate future states and evaluate the long-term impact of actions. Specifically, we define three state and action tuples tailored to each agent, as shown in Table 2.1. The profiler agent's reward, R_1, balances makespan-first with energy as $R_1 = \beta(M_{\text{target}}/(M + \epsilon)) + (1 - \beta)(E_{\text{target}}/(E + \epsilon))$, where M is the actual makespan, E is the average energy consumption, M_{target} and E_{target} are target values, $\beta = 1$ to put makespan first, and $\epsilon = 10^{-3}$ to avoid division issues. The thermal agent's reward, R_2, manages temperature with $R_2 = 1 - 0.05|T - T_{\text{target}}|$ (capped at 1) if the average temperature T is at or below the target T_{target}, or $R_2 = 1 - 0.5(T - T_{\text{target}})$ if above, with a -1 penalty if T crosses T_{target} from below compared to the prior state. The priority agent's reward, $R_3 = (M_{\text{target}} - M)/M_{\text{target}}$, quantifies makespan improvement over the target. Figure 2.6 illustrates this conceptual difference. In IRL, the reward function is shaped by aligning expert and agent policies. In contrast, our approach maximizes information from key states and model-based predictions without solely relying on expert data.

Table 2.1: Agent States and Actions

Agent	**State**	**Action**
Profiler	Utilization, Energy, Makespan	Cores, Frequency
Thermal	Clusters/Cores temperature	Core combination
Priority	Makespan	Priority combination

To achieve this, we incorporate attention models that weight input observations based on their importance in predicting future performance and energy outcomes. By focusing on key features—such as temperatures, frequencies, utilization, and makespan predictions—the reward model better aligns with the long-term objectives of minimizing energy consumption, achieving target makespans, and maintaining thermal feasibility. This approach reduces reliance on expert demonstrations and effectively handles suboptimal or missing expert data by leveraging the environment model's predicted trajectories. Rewards were developed iteratively, validated against baseline scenarios in Section 4.4, with attention models weighting features (e.g., temperature, utilization) based on temporal impact, trained via backpropagation on predicted states to enhance prediction accuracy.

2.4.4 Hyperparameter Tuning and Target Metrics

Careful selection of hyperparameters, including batch size, learning rate, discount factor, planning count, and exploration parameters, is essential for ensuring stable training. Additionally, clearly defining target metrics for makespan and energy consumption facilitates better convergence toward efficient performance. In our case study, we determine hyperparameters through a grid search.

Achieving the target minimum makespan in regular workloads is straightforward when allocating all available cores at the highest frequency. However, this method may not be optimal

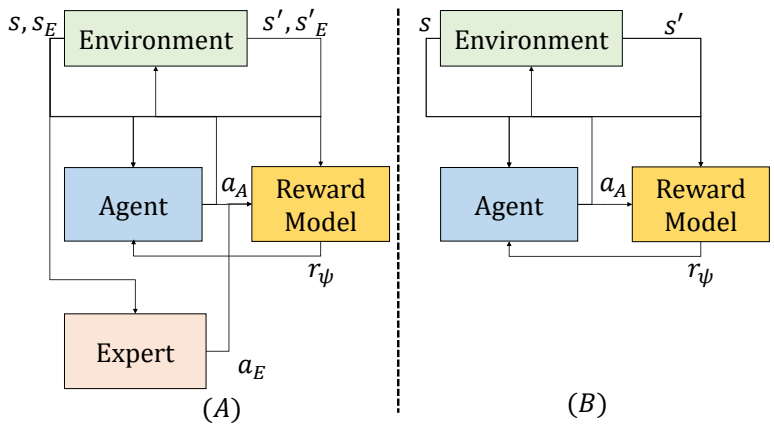


Figure 2.6: IRL (A) aligns with expert behavior; our model (B) uses key-state predictions to shape rewards.

for irregular workloads, as explained in the background section, due to workload characteristics. Energy consumption presents a more complex challenge as it depends on both power consumption and execution time. Specifically, lowering the operating frequency reduces instantaneous power consumption but increases execution time, potentially leading to higher overall energy usage. This relationship can be expressed as:

$$\text{Energy Consumption} = \text{Power Consumption} \times \text{Execution Time}$$

where energy is the total energy consumed by all parallel applications, power is the rate of energy usage at a given frequency and core count assigned to each application, and execution time is the duration of execution of each application.

To approximate the minimum energy consumption and makespan, we evaluate four different scenarios:

1. Running applications sequentially at the lowest frequency with all cores assigned.

2. Running applications sequentially at the lowest frequency with only one core assigned.

3. Running applications sequentially at the highest frequency with all cores assigned.

4. Running applications sequentially at the highest frequency with only one core assigned.

The lowest energy consumption and makespan are determined by selecting the lowest values from these scenarios. This assumes minimum scaling for our workloads, though not universally true; it serves as a practical baseline for target setting.

We impose a thermal limit of 50°C, similar to the approach in [53], to reward the thermal management agent. Priority adjustments are based on the total target makespan, shaping the reward signals to encourage policies that meet or exceed these baselines in efficiency and stability. By balancing the trade-offs between frequency scaling and core allocation, the system aims to achieve optimal energy efficiency without compromising performance, ensuring sustainable and effective resource utilization.

2.4.5 Complexity Analysis of Agents

In the proposed HiDVFS framework, each agent (profiler, thermal, and priority) is implemented using D3QN, which separates value and advantage streams for stable Q-value learning. The complexity of each agent is determined by input dimensionality (number of observed features), hidden layer sizes, and the action space for each agent:

Profiler Agent: Inputs include performance metrics (utilization, energy, instructions) and outputs involve selecting a frequency-core combination. Complexity scales with the number

of cores and frequency levels.

Thermal Agent: Observes temperature metrics and outputs priority adjustments for cores. This agent has lower complexity due to a simpler state representation and fewer action dimensions.

Priority Agent: Manages the selection of one out of a small set of predefined priority combinations. This agent adds minimal complexity since its action space is limited and its state is low-dimensional.

By distributing responsibilities among the agents, each handles a reduced subset of decisions, collectively forming a solution that remains computationally manageable even with a large number of cores and frequency levels. Although the environment model introduces additional computational load, this is managed on a server with parallel computation capabilities, mitigating runtime concerns on embedded platforms.

2.4.6 Hierarchical MARL Algorithm with Reward Model

The training process integrates real-time environment interactions with model-based reward estimation. Instead of using model-based RL solely for planning future actions, we utilize the learned environment model to generate synthetic transitions and refine the reward signal. This approach emphasizes future outcomes in the reward calculation, enhancing sample efficiency and convergence speed.

Algorithm 2 outlines the main steps. The agents interact with the real environment to gather transitions, which are then used to update the environment model. The environment model predicts future states and rewards, which are fed into a refined reward estimation module. By in-

Algorithm 1 HiDVFS with D3QN.

```
Initialize: Replay buffers for profiler, thermal, and priority agents
Initialize environment model and value functions (D3QN) for each agent
for each episode do
    Initialize states for profiler, thermal, and priority agents
    while not terminal do
        Direct RL:
        Agents select actions based on current policies
        Environment executes these actions, returns next states and instant rewards
        Store real transitions in replay buffers
        if Model training condition then
            Train environment model using recent real transitions
        end if
        Model-Based Reward Estimation:
        for each planning step do
            Sample transitions from replay buffers
            Use environment model to predict future states
            Estimate future-based rewards using IRL-inspired logic and attention
            Store these refined transitions in separate buffers for the agents
        end for
        if Agent training condition then
            Sample combined real and model-based transitions
            Train each agent's D3QN with refined, future-oriented rewards
        end if
        Update states
    end while
end for
```

corporating these predicted trajectories, the reward estimation module provides more stable and long-term-focused reward signals to the agents' learning processes. The three agents—profiler, thermal, and priority—are updated using a combination of real and model-based transitions, ensuring rapid convergence toward policies that minimize energy consumption, achieve target makespans, and maintain thermal limits. Agents resolve conflicts via joint optimization of their distinct reward functions, with the priority agent balancing resource contention, as validated in Section 5.

2.4.7 Practical Considerations: Platform and Parallel Execution

In our server-client architecture, complex computations such as environment modeling and reward estimation are offloaded to a high-performance server. The server communicates frequency-core assignments and priority configurations to the client, which operates on an

embedded platform. The client executes the assigned tasks and returns performance and temperature measurements. This division ensures that the client remains responsive and capable of real-time operations, while the server handles computationally intensive tasks like training, hyperparameter tuning, and environment model refinement.

A key enhancement in our implementation is the incorporation of **level_of_parallelism**, which dynamically allocates CPU cores based on the parallelism requirements of each application. In this case, the makespan calculated using only one core will be divided by the makespan calculated using all cores. During each experiment cycle, the server determines the appropriate level of parallelism for each parallel application by referencing profiling data. This parameter dictates the number of cores allocated to each application, ensuring optimal utilization of computational resources without overcommitting available cores. Applications are prioritized, and higher-priority tasks receive core allocations first, maintaining system stability and performance.

The allocation process involves sorting applications based on their priority levels and assigning cores accordingly. If sufficient cores are available, each application receives the number specified by its `level_of_parallelism`. In cases where core availability is limited, the system assigns as many cores as possible while logging any shortages to inform future allocations. Additionally, each allocated core is assigned a frequency step to balance performance with thermal constraints, ensuring that the system operates efficiently and remains within safe temperature limits.

Parallel execution on the server enables rapid generation of synthetic samples, training

of the reward model, and exploration of various configurations. This parallelism accelerates adaptation to workload changes and facilitates convergence toward policies that are efficient, thermally safe, and energy-minimized. By leveraging few-shot learning, as demonstrated in the evaluation, our approach minimizes the need for extensive real-world data collection, enhancing scalability. By minimizing the need for extensive data collection from the real system, our approach enhances scalability and reduces dependency on expert demonstrations.

Overall, our HMARL framework, augmented with IRL-inspired reward estimation and attention-based weighting of input observations, significantly accelerates the learning process. The dynamic allocation of cores based on `level_of_parallelism` ensures practical scalability and stability in real-time multicore scheduling scenarios, maintaining system responsiveness while optimizing performance and resource utilization.

2.5 Experimental Platform, Benchmark, and Evaluation

In this section, we describe the experimental setup and benchmark, present our evaluation methodology, and compare different RL approaches in terms of key performance metrics. Our evaluation prioritizes makespan; energy and temperature are secondary outcomes.

2.5.1 Platforms and Setup

Experimental Platforms: We conduct experiments on the NVIDIA Jetson TX2 to leverage its fine-grained frequency scaling and per-core Dynamic Voltage and Frequency Scaling (DVFS). This platform offers in-kernel status monitoring, per-core sleep states, energy monitoring, and per-cluster temperature profiling. The experiments run on Ubuntu 18.04, the latest version supported by the Jetson TX2 board. We use a FIFO real-time scheduler on a pre-

emptible Linux kernel to prioritize tasks. Intel's p-state and c-state power management features are disabled, and hyper-threading is turned off. Frequency adjustments are performed by controlling the `scaling_max_freq` parameter using `cpufrequtils`, ensuring precise control over per-core frequencies.

The NVIDIA Jetson TX2 development board features six heterogeneous cores with frequency levels ranging from 345,600 kHz to 2,035,200 kHz. This range is divided into 12 steps, where levels 0 and 11 correspond to the minimum and maximum frequencies, respectively. The processor operates on an ARM64 (`aarch64`) Linux platform (kernel version `4.9.337`), optimized for multicore processing with real-time/preemptive capabilities (`SMP PREEMPT`). The six cores of the Jetson TX2 comprise a dual-core high-performance NVIDIA Denver 2 64-bit CPU and a power-efficient quad-core Arm Cortex-A57 MPCore processor. In this setup, cores 1 and 2 refer to the Denver 2 cluster, while cores 0, 3, 4, and 5 form the Arm Cortex cluster. Core 0 is reserved for root tasks, including CPU affinity management, interrupt handling, and task assignments. The remaining five cores run the parallel DAGs depending on CPU affinity through the `CPUSET` tool.

Jetson TX2 is chosen for its fine-grained frequency scaling (12 steps, 345,600–2,035,200 kHz), unlike Intel RAPL's coarser adjustments, enabling precise DVFS control critical for our per-core thermal focus. Compared to modern systems like the Intel Core i7, the ARM-based heterogeneity of NVIDIA Jetson platforms more closely aligns with the requirements of embedded scenarios, as evidenced by consistent profiler accuracy in performance monitoring. Additionally, Intel's DVFS implementation in its processors is often constrained at the hard-

ware level, providing less fine-grained control over frequency scaling compared to ARM-based systems, which are better suited for dynamic embedded workloads.

Benchmark: We use the BOTs benchmark suite [31], focusing on the `FFT` (Fast Fourier Transform) workload with varying levels of parallelism, including untied, tied, and serial implementations. This benchmark is suitable for evaluating AI-driven scheduling policies in parallel DAGs. Our framework is not limited to this benchmark and can handle any parallelized workload for frequency, priority, and core assignment. All results reported below use `FFT` on Jetson TX2.

2.5.2 Evaluation of Metrics and Statistical Analysis of Features

This subsection outlines the evaluation methodology for assessing single-agent and multi-agent Reinforcement Learning (RL) approaches, focusing on key performance metrics—makespan, energy consumption, average temperature, branch misses, and cache misses—while integrating statistical analyses to quantify the impact of critical variables: task priority, number of cores, and average frequency. For single-agent RL, the primary objective is to minimize makespan or energy consumption by optimizing core allocation and frequency selection, as detailed in Section 2.4.3. The reward function balances these objectives using the parameter β, which weights makespan (β) against energy consumption $(1-\beta)(E_{\text{target}}/(E+\epsilon)))$. To evaluate convergence speed, we set $\beta = 1$, isolating makespan and excluding energy terms, though minimizing makespan indirectly reduces energy due to shorter computation times, as confirmed by our statistical results. Target values for makespan and energy consumption, derived from testing all four conditions in Section 2.4.4, serve as benchmarks. In multi-agent RL, rewards are assigned

to a thermal agent for core combination actions and a priority agent for task priority assignments, enhancing temperature reliability and makespan through real-time processing of tasks and learning efficiency of models. Throughout, makespan is the optimization priority.

Statistical analysis, summarized in Table 2.2, employs the Mann-Whitney U test to evaluate the influence of task priority, number of cores, and average frequency on key performance metrics. Low p-values (< 0.05) indicate significant effects, while higher values suggest weaker relationships. Task priority significantly impacts makespan (p = 1.89e-02), increasing from 4.60s (low) to 5.04s (high), reflecting longer completion times for higher-priority tasks due to the SCHED_FIFO scheduler's strict preemption, which prioritizes high-priority tasks, delaying lower-priority ones and increasing context-switching overhead in parallel workloads. Its effect on energy consumption is non-significant (p = 1.42e-01), with a slight decrease from 23549.88 to 23035.29 mJ, but it strongly influences average temperature (p = 3.68e-04), branch misses (p = 1.19e-15), and cache misses (p = 1.22e-24), with higher priority correlating with increased miss rates, indicating a trade-off in system efficiency. The number of cores significantly reduces makespan (p = 6.69e-06) from 5.28s to 3.75s and energy consumption (p = 3.26e-25) from 27017.90 to 16013.48 mJ as core count increases, highlighting multicore efficiency gains, though it has negligible impact on temperature (p = 4.00e-01) and increases branch (p = 4.74e-12) and cache misses (p = 6.28e-18). Frequency exerts the strongest influence, slashing makespan (p = 1.45e-283) from 6.92s to 2.62s and energy consumption (p = 6.03e-144) from 29615.61 to 17083.91 mJ as it rises, while also affecting temperature (p = 3.40e-03) and cache misses (p = 8.37e-10), but not branch misses (p = 2.54e-01). These

Table 2.2: Statistical Analysis of Key Features' Influence on Performance Metrics

Action	Variable	Expected (L)	Expected (H)	p-value
Priority	Makespan	4.60	5.04	1.89e-02
	Energy	23549.88	23035.29	1.42e-01
	Temperature	36.99	37.12	3.68e-04
	Branch Misses	1.72e7	2.25e7	1.19e-15
	Cache Misses	3.42e7	4.28e7	**1.22e-24**
Cores	Makespan	5.28	3.75	6.69e-06
	Energy	27017.90	16013.48	**3.26e-25**
	Temperature	37.03	37.06	4.00e-01
	Branch Misses	1.86e7	2.08e7	4.74e-12
	Cache Misses	3.35e7	4.56e7	6.28e-18
Frequency	**Makespan**	6.92	2.62	**1.45e-283**
	Energy	29615.61	17083.91	6.03e-144
	Temperature	36.99	37.10	3.40e-03
	Branch Misses	1.92e7	1.95e7	2.54e-01
	Cache Misses	3.82e7	3.68e7	8.37e-10

findings underscore the significance of all three factors, with low p-values (< 0.05) indicating their impact on system variables. Frequency dominates in optimizing performance, followed by core count, while priority drives system overhead. Higher priority increases makespan due to the SCHED_FIFO scheduler's strict preemption, which prioritizes high-priority tasks, delaying lower-priority ones and increasing context-switching overhead; the priority agent mitigates this by selecting optimal static priority allocations for parallel tasks before execution. These analyses motivate our makespan-first reward.

Profiling data in Figure 2.7 further informs these insights, illustrating performance under parallel and sequential modes for three FFT workload variations (untied, tied, and serial). Temperature regulation ensures stability, but tied execution incurs higher makespan, branch misses, and cache misses due to restricted task migration, while serial mode, using one core, shows predictable, minimal miss patterns. Parallel execution introduces greater unpredictability in cache misses compared to serial mode. These results align with the statistical findings in 2.5 for more importance of features to guide HiDVFS in dynamically adjusting priority, cores,

and frequency to minimize makespan and energy consumption while mitigating performance penalties.

Figure 2.8 illustrates how variations in priority combinations, number of cores, and frequency levels affect total energy consumption and makespan in parallel and sequential modes. The high variation in parallel mode underscores the importance of adaptive parameter adjustments for energy efficiency and makespan.

2.5.3 Implemented Approaches

We compare our methods—**HiDVFS_S** (single-agent) and **HiDVFS** (multi-agent)—against representative DVFS/RL baselines.

Single-agent baselines. zTT [53] and GearDVFS [64] are model-free DVFS schedulers; DynaQ [7] and PlanGAN [21] are model-based. **HiDVFS_S** uses short-horizon model predictions only for reward shaping, improving stability and convergence.

Multi-agent baselines. MAMB and MAMF are model-based/model-free decompositions; HiDVFSD3 adds D3QN stabilization. **HiDVFS** adds thermal and priority agents on top of HiDVFS_S with short-horizon reward shaping and coordinated actions (core masks, frequencies, priorities).

2.5.4 Comparison with Baselines and Results

Comparative Performance: Figures 2.9 and 2.10 compare our proposed HiDVFS_S and HiDVFS against SOTA RL baselines over 60 epochs. High rewards for lower makespans drive agents to maximize frequency and core counts, with faster stabilization indicating better convergence. HiDVFS_S outperforms zTT and GearDVFS, stabilizing frequency and core de-

cisions by epoch 12 (Figure 2.9). HiDVFS stabilizes makespan and frequency compared to MAMB and MAMF, achieving 49.06% makespan improvement over HiDVFSD3 (1308.21 s to 666.36 s) and 40.95% energy reduction (7665.12 J to 4528.79 J). Multi-agent methods stabilize earlier due to distributed decision making.

Energy consumption mirrors makespan trends (Figure 2.10), with shorter makespans reducing energy use. HiDVFS stabilizes rewards early, HiDVFS_S after a few epochs. HiDVFS's reward spikes reflect accumulated future rewards. Models were fine-tuned with 150 transitions before 60-epoch runs. In high-utilization scenarios, frequency increases reduce makespan but elevate cache misses, as seen in Figure 10, guiding the profiler agent to adjust core counts dynamically.

Evaluation Details: We evaluated RL methods over 150 epochs, defining convergence as stable rewards within a 10-epoch sliding window (makespan variation < 15% of initial 5-epoch average). Accumulated rewards over a planning horizon may spike (Figure 2.10); we used this criterion for consistency. Metrics (makespan, energy, branch misses) were summed over 60 epochs for comparison in Table 2.3. Among single-agent methods, HiDVFS_S achieved the lowest energy (4438.88 J) and makespan (626.01 s), converging at 12 epochs, a 43.17% makespan reduction over DynaQ (1101.40 s) and 44.24% energy reduction (7960.52 J to 4438.88 J). In the multi-agent category, HiDVFS achieved the best makespan (666.36 s) and competitive energy usage (4528.79 J), converging at 84 epochs, improving makespan by 49.06% over HiDVFSD3 (1308.21 s) and energy by 40.95% (7665.12 J to 4528.79 J).

Overhead and Deployment: HiDVFS's server-client architecture incurs a 2 ms commu-

nication latency per cycle. Hyperparameter tuning (grid search) is performed offline, requiring 10–12 hours on a 16 GB RAM, 4 core Intel Core i7 CPU server, which does not add running time.

Table 2.3: Comparison of Proposed RL Approaches (HiDVFS_S, HiDVFS) Against SOTA RL Baselines

Approach	Energy (J)	Makespan (s)	Branch Miss	Conv. Time
Single-Agent Approaches				
zTT [53]	6068.53	705.57	4417467986	15
DynaQ [7]	7960.52	1101.40	4576945600	150
PlanGAN [21]	5404.63	800.72	4559350200	150
GearDVFS [64]	6572.64	1190.95	**4397081278**	150
HiDVFS_S	**4438.88**	**626.01**	4418436701	**12**
Multi-Agent Approaches				
MAMB	5784.75	861.83	2937169254	150
MAMF	6972.42	962.17	3842255529	**44**
HiDVFS	**4528.79**	**666.36**	2789133021	84
HiDVFSD3	7665.12	1308.21	**2295745187**	65

Our proposed HiDVFS_S and HiDVFS, leveraging per-cluster temperature profiling and model-based reward estimation, outperform SOTA RL baselines in makespan reduction, energy savings, and thermal management for FFT workloads. HiDVFS_S converges in 12 epochs for simpler settings, while HiDVFS excels on complex DAGs with manageable deployment overhead on Jetson TX2. HiDVFSD3 improves branch-miss behavior, which may benefit highly irregular workloads.

2.6 Conclusion

We introduced HiDVFS, a hierarchical multi-agent DVFS scheduler for OpenMP DAGs with a makespan-first objective, and its single-agent variant HiDVFS_S. On Jetson TX2 (BOTS `FFT`), HiDVFS cut makespan by 49.06% and energy by 40.95% over a multi-agent baseline, while HiDVFS_S reduced makespan by 43.17% and energy by 44.24% over a single-agent baseline, converging in 12 epochs. The decomposition of frequency/core selection,

temperature-aware grouping, and static priority enabled fine-grained per-core control with low overhead.

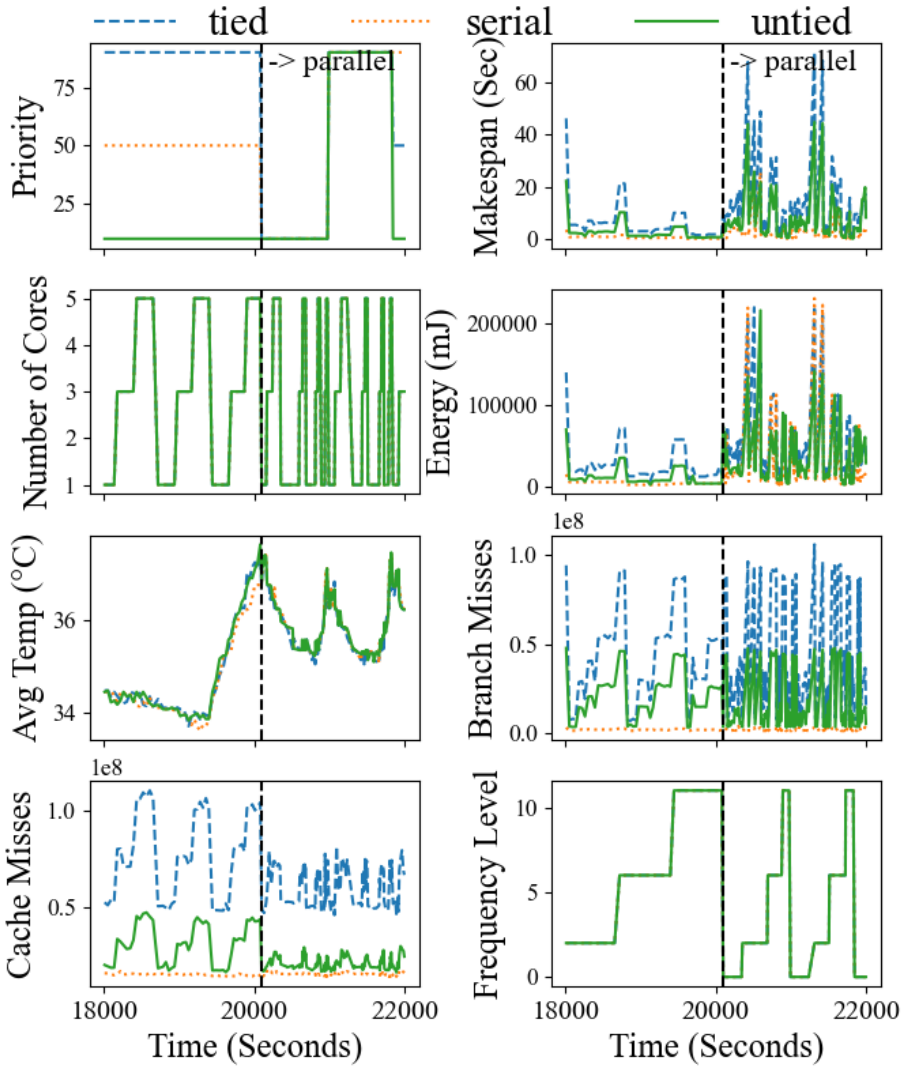


Figure 2.7: Profiling data under parallel and sequential modes for FFT workload variations (untied, tied, serial) show more spontaneity for profiling data in parallel mode.

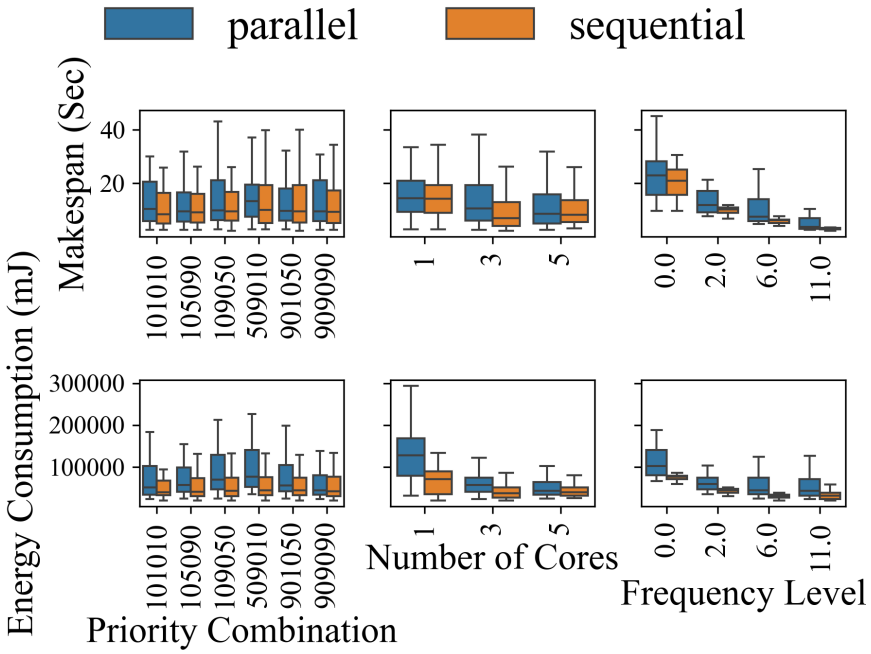


Figure 2.8: Variation of total energy consumption and makespan due to changes in priority combinations, number of cores, and frequency levels in parallel and sequential modes.

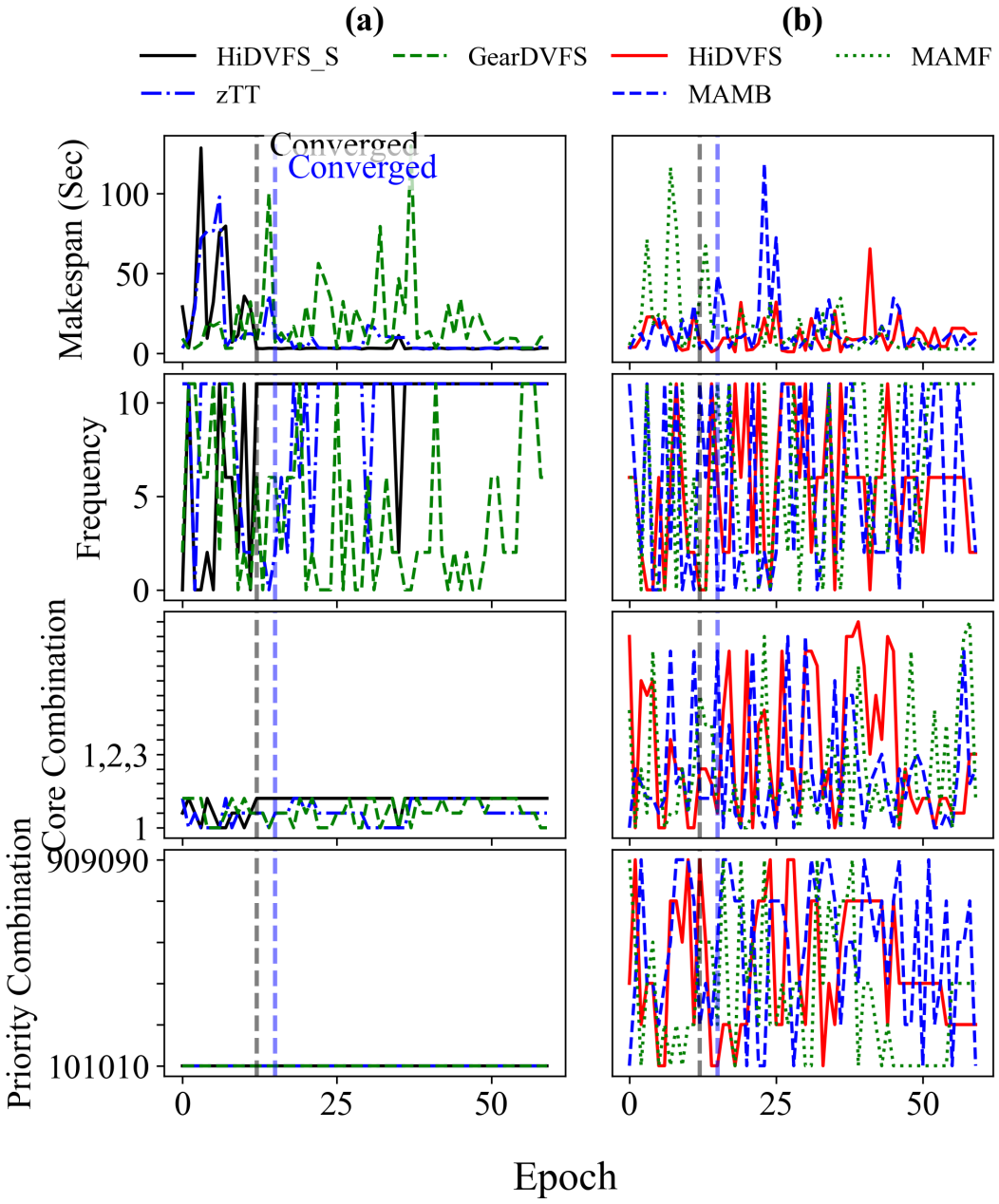


Figure 2.9: Training over 60 epochs: (a) HiDVFS_S vs. zTT/GearDVFS stabilizes by epoch 12; (b) HiDVFS vs. MAMB/MAMF shows faster makespan/frequency stabilization.

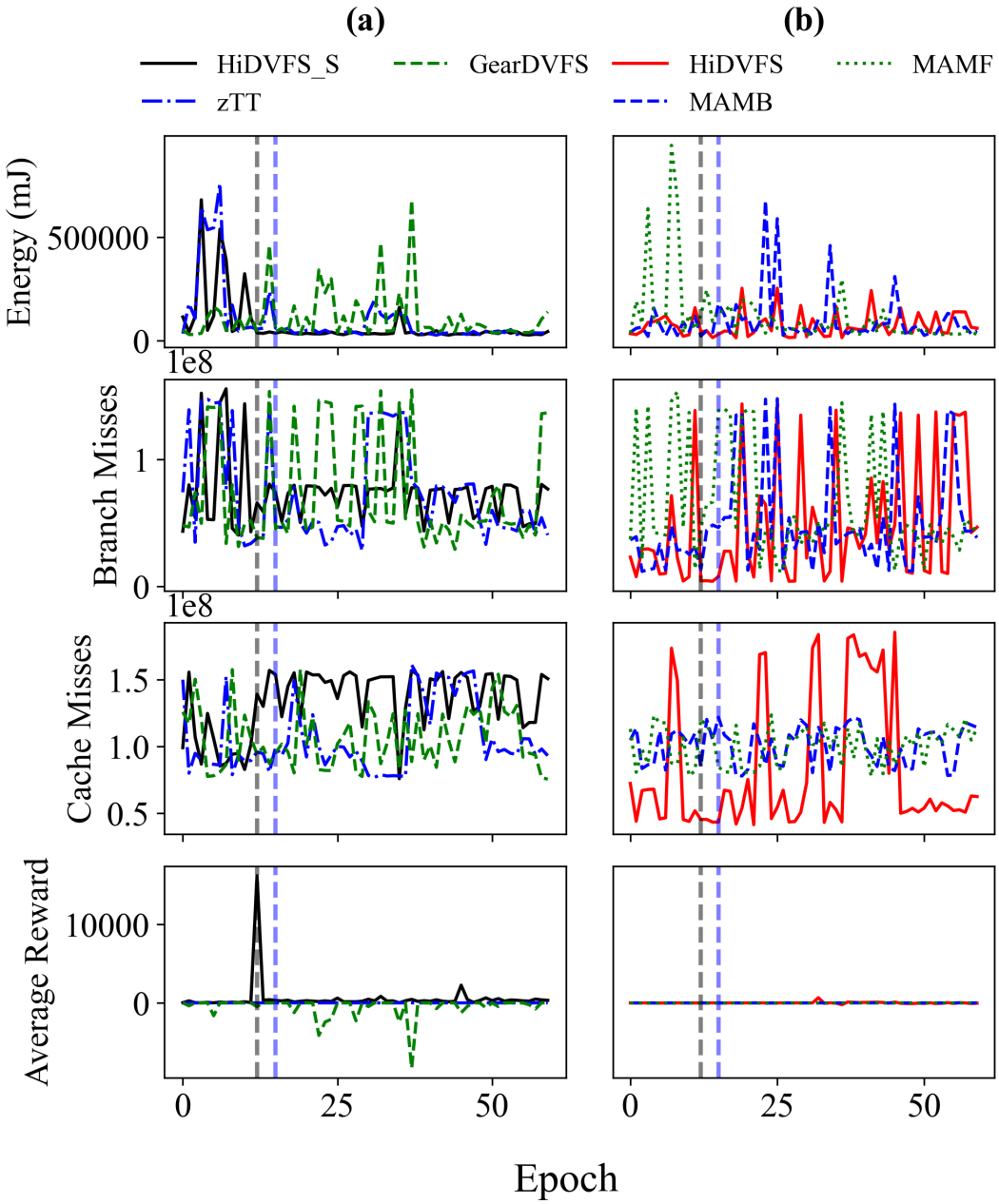


Figure 2.10: Observations over 60 epochs mirror makespan: HiDVFS achieves earlier reward stabilization; HiDVFS_S follows within a few epochs.

CHAPTER 3 FEATURE-AWARE TASK-TO-CORE ALLOCATION IN EMBEDDED MULTI-CORE PLATFORMS VIA STATISTICAL LEARNING

Abstract

Optimizing task-to-core allocation can substantially reduce power consumption in multi-core platforms without degrading user experience. However, existing approaches overlook critical factors such as parallelism, compute intensity, and heterogeneous core types. In this paper, we introduce a statistical learning approach for feature selection that identifies the most influential features—such as core type, speed, temperature, and application-level parallelism or memory intensity—for accurate environment modeling and efficient energy minimization, a critical consideration for embedded systems. Our experiments, conducted with state-of-the-art Linux governors and thermal modeling techniques, show that correlation-aware task-to-core allocation lowers energy consumption by up to 10% and reduces core temperature by up to 5°C compared to random core selection. Furthermore, our compressed, bootstrapped regression model improves thermal prediction accuracy by 6% while cutting model parameters by 16%, yielding an overall mean square error reduction of 61.6% relative to existing approaches. We provided results based on superscalar Intel Core i7 12th Gen processors with 14 cores, and validated our method across a diverse set of hardware platforms and effectively balanced performance, power, and thermal demands through statistical feature evaluation.

3.1 Introduction

Task-to-core allocation is a pivotal technique for enhancing the performance and reliability of embedded systems, where workloads exhibit distinct thermal and power consumption behaviors depending on core assignments. In multi-core processors, allocations can involve high-performance cores, low-energy cores, and graphic processing units (GPUs), each with unique performance characteristics across various configurations. Improper allocation can lead to thermal overheating, triggering throttling and reducing chip reliability and lifespan [44]. Cooling costs for overheated chips are significant, approximately $3 per watt of heat dissipation [35]. While dynamic voltage and frequency scaling (DVFS) complements allocation by adjusting power dynamically—potentially reducing consumption by 75% without altering user experience [80]—our focus is on optimizing task-to-core allocation to manage thermal and energy constraints in embedded systems. Existing approaches often rely on historical workload and sensor data to predict future behavior [68], but lack a systematic, statistically robust framework, which our hybrid methodology addresses with novelty and precision.

Embedded systems, foundational to automotive, telecommunications, and consumer electronics, must operate under strict power and thermal constraints while delivering high performance. Task-to-core allocation is crucial for managing these constraints in heterogeneous architectures with diverse core types (e.g., high-performance, low-power, GPUs). Feature selection and evaluation identify critical features and correlations, enabling optimal core assignments for thermal stability and energy efficiency [90]. For instance, in real-time automotive systems, allocation ensures compute-intensive tasks run on performance cores without over-

heating, while mobile devices assign power-hungry applications to high-performance cores and background tasks to low-power cores, extending battery life. Identifying features tied to power and thermal behavior—such as temperature, frequency, and core adjacency—establishes general rules for allocation, enhancing embedded system reliability. This paper introduces a hybrid statistical learning framework, integrating Random Forest (RF), backward stepwise selection, and correlation analysis, to optimize task-to-core allocation across platforms, prioritizing allocation over DVFS for embedded contexts.

Application performance depends on execution factors like core type, speed, temperature, and application characteristics—memory or compute intensiveness and parallelism level [74]. Linux governors [17] and related studies often focus on CPU utilization for DVFS policies, targeting latency, temperature, or energy [66, 64, 53]. However, for task-to-core allocation, characteristics like parallelism, memory/compute intensiveness, and branch counts are equally critical. Applications with frequent branch misses benefit from sequential execution on single cores, while parallel, compute-intensive tasks excel on GPUs or performance cores in heterogeneous platforms. Moreover, temperature correlations between cores indicate adjacency and overheating risks. Our approach transcends utilization-centric methods, employing a multi-stage statistical framework to capture a comprehensive feature set and their interdependencies, ensuring clarity and applicability in embedded systems.

Collecting real data post-execution for task-to-core allocation optimization is computationally expensive and imprecise due to hardware sampling delays. Instead, allocation decisions can leverage inference from trained environmental models that account for randomness and

feature importance to mitigate overfitting [66]. An augmentation step further reduces sampling overhead, boosting efficiency. Our paper introduces the first statistical learning framework for embedded systems, uniquely integrating RF-based feature reduction, backward stepwise selection, and correlation-aware allocation with bootstrapping to create a robust, platform-agnostic model. Unlike prior heuristic or filter-based methods, which lack systematic feature evaluation or adaptability across heterogeneous platforms, our approach outperforms existing techniques—where they exist—achieving up to 10% energy savings and 61.6% lower mean squared error, as validated against SOTA baselines on diverse embedded hardware.

Little work applies statistical learning to feature selection for task-to-core allocation with the rigor we propose. Statistical learning predicts outcomes and reveals feature relationships. Prior efforts used extreme value theorem (EVT) for energy and execution time bounds in DVFS contexts [20, 29, 82, 75], while Liu et al. [66] applied XGBoost for latency-related features, and Sasaki et al. [85] used decision trees and correlation for energy per instruction. These lack the integrated, multi-method focus we introduce for allocation, synergizing RF, OLS, and correlation analysis for superior predictive power and generalizability in embedded systems.

Our framework shows that correlation-aware task-to-core allocation reduces chip temperature at intermediate utilization by leveraging thermally independent cores. Backward stepwise selection reveals a small feature subset suffices for accurate energy estimation, while RF extracts critical features with low overhead. Bootstrapping enhances dataset robustness, yielding a compact, efficient model. This hybrid methodology surpasses prior single-method approaches, validated across embedded platforms for practical impact.

The key contributions are:

1. This paper pioneers a hybrid statistical learning approach, integrating RF, backward stepwise selection, and correlation analysis, to evaluate feature importance for accurate environment modeling and compressed learning in task-to-core allocation for embedded systems.

2. We implement correlation-aware allocation, statistically reducing temperature and energy by assigning tasks to uncorrelated cores, validated across heterogeneous embedded platforms.

3. We employ bootstrapping to augment data and boost accuracy, supporting few-shot and meta-learning in resource-constrained embedded settings.

4. We rigorously test against state-of-the-art methods, achieving up to 10% energy reduction and 5°C temperature decrease. Our compressed, bootstrapped model cuts prediction error by 6%, reduces parameters by 16%, and improves mean squared error by 61.6% over SOTA. Data were extracted from Intel Core i7 8th and 12th Gen and Intel Xeon 2680 processors.

In the remainder, Section 5.5 surveys statistical learning and scheduling, Section 4.3 details motivation and challenges, and Section 5.3 describes the methodology. Section 6.5 evaluates three processors, followed by conclusions in Section 6.6.

3.2 Motivation and Challenges

Feature selection is a cornerstone of statistical learning, pivotal for optimizing task-to-core allocation in embedded multi-core systems. It addresses critical challenges—curse of dimensionality, large data management, and performance unpredictability—common to platforms like Intel Core i7 and NVIDIA Jetson TX2. To surpass prior filter-based approaches and establish novelty, we leverage a hybrid methodology integrating Random Forest (RF), backward stepwise selection, and correlation analysis, tailored to embedded constraints. This section clarifies these challenges in the context of task-to-core allocation, emphasizing generalizability across heterogeneous architectures and grounding our motivation in rigorous, platform-validated insights.

3.2.1 Curse of Dimensionality

Feature selection is essential to mitigate the curse of dimensionality, a challenge amplified in embedded multi-core systems. As feature count rises—encompassing frequency, temperature, and performance metrics like cache misses—the data space grows exponentially, leading to sparsity in high-dimensional spaces [61]. This sparsity risks model overfitting, degrading predictive performance on unseen data. In heterogeneous embedded systems, the diversity of cores (performance, low-energy, GPUs) expands the state space, with each core type contributing unique thermal and performance characteristics. For instance, Intel Core i7 12th Gen's hybrid P/E-cores and Jetson TX2's big-LITTLE clusters introduce complex feature interactions. Efficient feature selection, as implemented via our RF-based reduction, reduces this complexity, enabling compact, generalizable models critical for real-time embedded applica-

tions—offering a novel advance over traditional dimensionality handling.

3.2.2 Large Data Management

Managing the growing volume of data is a pressing challenge for task-to-core allocation in embedded systems, where resource constraints demand efficiency. Allocation strategies may use static historical data or streaming inputs adapting to runtime conditions. With streaming data's rise—common in automotive or mobile embedded platforms—memory management becomes critical, as unpredictable data volumes can overwhelm limited resources. Retaining unnecessary features, such as redundant performance counters, inflates storage and processing overhead, straining embedded systems. Our hybrid approach, combining RF for initial feature pruning and bootstrapping for data augmentation, optimizes processor models and allocation algorithms, reducing overhead while maintaining accuracy. This addresses large data challenges with clarity and applicability across static and dynamic embedded environments, surpassing simpler filter-based data handling.

3.2.3 Performance Unpredictability

Performance unpredictability in embedded multi-core processors stems from energy and thermal variability, driven by features like temperature, frequency, and core-specific performance data. Figure 3.1 illustrates this, showing energy consumption varying by an order of magnitude across identical frequency and core settings on Intel Core i7 8th Gen (4 cores), Intel Core i7 12th Gen (14 cores), and Intel Xeon 2680 v3 (12 cores) under OpenMP benchmarks. This variability underscores the need for feature selection to pinpoint significant predictors of energy and thermal behavior across platforms and configurations. Our methodology—using

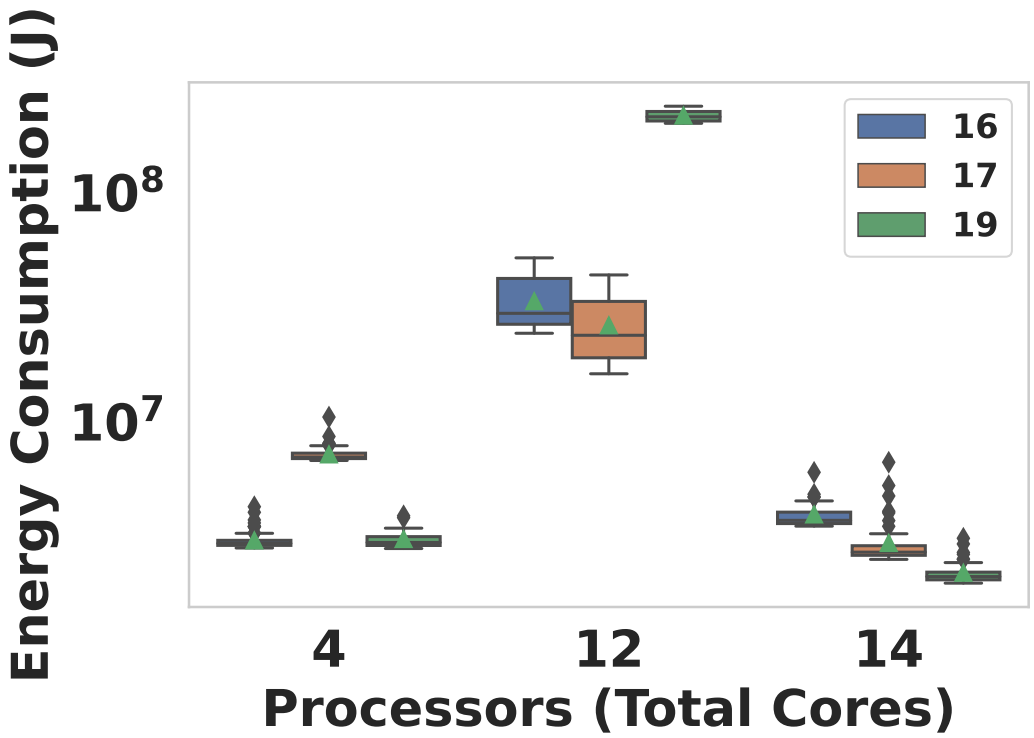


Figure 3.1: Energy consumption variation across three different processors with identical frequency and core count settings: Intel Core i7 8th Gen (Corei78) with 4 cores, Intel Core i7 12th Gen (Corei712) with 14 cores, and Intel Xeon 2680 v3 (Xeon) with 12 cores. The results are shown for three different OpenMP benchmarks.

backward stepwise OLS for energy-critical features and correlation analysis for thermal independence—identifies robust predictors, enabling a global allocation policy. Unlike DVFS-centric prior works, we prioritize task-to-core allocation, validated rigorously with up to 10% energy savings and 5°C temperature reductions, ensuring practical impact in embedded systems.

3.3 Design Methodology

Our objective is to develop a robust multistage methodology based on three feature selection approaches—embedded, wrapper-based, and filter-based—and show how their combined

use informs an intelligent task-to-core allocation algorithm for embedded multi-core systems. We introduce a hybrid framework integrating Random Forest (RF), backward stepwise OLS, and Pearson correlation, surpassing traditional filter-based methods in efficiency and predictive power. We provide a structured workflow adaptable to diverse platforms like Intel Core i7 8th and 12th Gen, Intel Xeon 2680 v3, and NVIDIA Jetson TX2, while ensuring experimental rigor through validated outcomes (e.g., 10% energy savings, 5°C temperature reduction). This section details each stage, linking to embedded system optimization.

3.3.1 Data Collection and Environment Setup

We begin by profiling the target embedded hardware under diverse workloads to capture comprehensive data on energy consumption, temperature variations, and relevant performance metrics. The granularity of thermal sensors significantly varies across platforms. Intel Core i7 8th generation and Intel Xeon 2680 v3 processors provide homogeneous core architectures with individual per-core thermal sensors, enabling detailed per-core correlation analyses. In contrast, the Intel Core i7 12th generation features a heterogeneous architecture composed of performance (P-cores) and efficient (E-cores) cores, each type operating at different frequencies and performance characteristics. Similarly, the NVIDIA Jetson TX2 offers thermal readings aggregated at the cluster level, typically distinguishing between big and little core clusters due to its big-LITTLE architecture. Thus, our methodology accommodates per-core, heterogeneous core, and cluster-level sensor capabilities, specifically tailored to embedded computing environments, ensuring broad applicability.

Our data collection covers a wide range of configurations, including varying core counts (4,

12, or 14 cores), diverse CPU frequencies, and task prioritizations. Tasks include representative benchmarks such as Fibonacci (fib), matrix multiplication (Strassen), and sequence alignment (alignment), providing varied computational intensities and memory access patterns typical of embedded system workloads. This diverse dataset enables accurate modeling of core behaviors under different workloads and thermal conditions, essential for embedded system optimization.

Initially, the system setup involves configuring the frequency governor (typically `schedutil`) and initializing a temperature buffer capable of storing historical temperature data from cores (up to 10,000 entries). Additionally, a detailed task history is maintained, capturing execution parameters such as energy consumption, profiler metrics, and temperature data under varying task priorities and frequencies. This systematic variation ensures the dataset's robustness and representativeness for embedded systems.

Energy monitoring employs built-in hardware interfaces such as `/sys/class/powercap/intel-rapl/` on Intel processors or on-board power sensors for the Jetson TX2, recording total energy consumed or average power during task execution. Performance metrics collected include CPU frequency, utilization, memory bandwidth, and hardware performance counters like cache misses, gathered via utilities such as `perf` and `cpufreq-info` at consistent intervals.

The random core assignment strategy serves as a baseline to evaluate the effectiveness of our proposed correlation-aware core allocation strategy. In the random assignment approach, tasks run concurrently on randomly selected cores, typically half of the available cores excluding core 0, reserved for system tasks. Post-execution, we record metrics including temperature, energy consumption, and performance data. Execution is temporarily paused if core temper-

atures exceed predefined thresholds (e.g., depending on thermal throttling point of Intel Core i7 which is 100°C), allowing cores to cool down before resuming execution. This baseline facilitates comparative analysis against the correlation-aware allocation strategy.

The collected dataset serves as input for the subsequent embedded feature reduction process, employing Random Forest algorithms to identify minimal yet critical feature subsets. This refined feature set is then utilized for accurate environment modeling, guiding intelligent, temperature-correlation-driven task-to-core allocation strategies specifically designed for embedded systems. Ultimately, our structured, multi-stage methodology aims to optimize thermal management and energy efficiency across diverse embedded multi-core computing platforms.

3.3.2 Embedded Feature Selection Using Random Forest

Embedded methods incorporate feature selection into the model training process, thereby minimizing the need for multiple model evaluations on different feature subsets. In this work, we employ a *Random Forest* (RF) regressor, an ensemble approach that constructs multiple decision trees and aggregates their predictions, offering a novel low-overhead alternative to filter-based methods.

Random Forest Algorithm. The RF algorithm proceeds as follows:

1. **Bootstrap Sampling:** Generate N bootstrap samples from the original dataset.

2. **Tree Construction:** For each bootstrap sample, grow a regression tree by selecting a random subset of features at each node (often $\sqrt{d}$ features). Each tree is grown to its maximum depth without pruning, although hyperparameters such as the number of trees

or maximum depth can be tuned for embedded constraints.

3. **Prediction Aggregation:** For regression tasks, the final prediction is the average of the individual tree outputs:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^{N} \hat{y}_i. \tag{3.1}$$

Feature Importance Computation. Random Forests provide a measure of the importance of features by evaluating the total decrease in the impurity of the nodes in all trees. For regression trees, impurity is commonly measured by the residual sum of squares. The importance score I_j for feature x_j is thus:

$$I_j = \frac{1}{N} \sum_{i=1}^{N} \sum_{t \in T_i} \Delta I_{t,j}, \tag{3.2}$$

where $\Delta I_{t,j}$ is the impurity decrease at node t when splitting on x_j, and T_i is the set of nodes in the i-th tree. The ranking of features by these importance scores guides the selection of highly influential predictors.

Bootstrapping for Data Augmentation. To increase model robustness, we employ bootstrapping, a resampling method that draws multiple datasets of size n with replacement. Let $\mathcal{D}$ be the original dataset of size n. Forming B bootstrap samples $\{\mathcal{D}_1, \ldots, \mathcal{D}_B\}$ helps estimate variance and stabilize the final model through aggregation of predictions.

Environment Modeling with Feature Selection. After identifying the most significant features using RF importance scores, we build predictive models for environment modeling—particularly neural networks—tailored to embedded constraints. Let $\mathbf{x} \in \mathbb{R}^p$ (with $p < d$)

denote the reduced feature set. We train a Fully Connected Neural Network (FCN) with multiple layers and non-linear activations to predict key variables such as energy consumption or temperature. The network is trained by minimizing the mean squared error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - f(\mathbf{x}_i;\theta)\right)^2, \tag{3.3}$$

where $f(\mathbf{x}_i;\theta)$ is the network's output for input $\mathbf{x}_i$ with parameters θ, and y_i is the true label. By restricting the input to a smaller set of highly relevant features, the FCN requires fewer parameters and less computation, making it feasible for real-time applications. As shown in Figure 3.2, our results indicate that using 39 out of 72 predictors provides nearly the same error value for estimating the future state of the processor profiler data model. For the sensor temperature model, using 21 out of 31 features yields the optimal Cross-Validation error (*CV error*), validated for embedded efficiency.

3.3.3 Wrapper-Based Feature Selection

Wrapper methods evaluate subsets of features using a predictive model. Because they account for feature interactions, they typically yield higher accuracy than filter methods in finding the importance of the features on a specific feature parameter but may be more computationally expensive. We employ the *backward stepwise selection* algorithm, which starts with all available features and iteratively removes the least significant feature based on a specified criterion. In our case, we use the Ordinary Least Squares (OLS) regression model to predict the target variables (energy consumption and average temperature) and assess the significance of

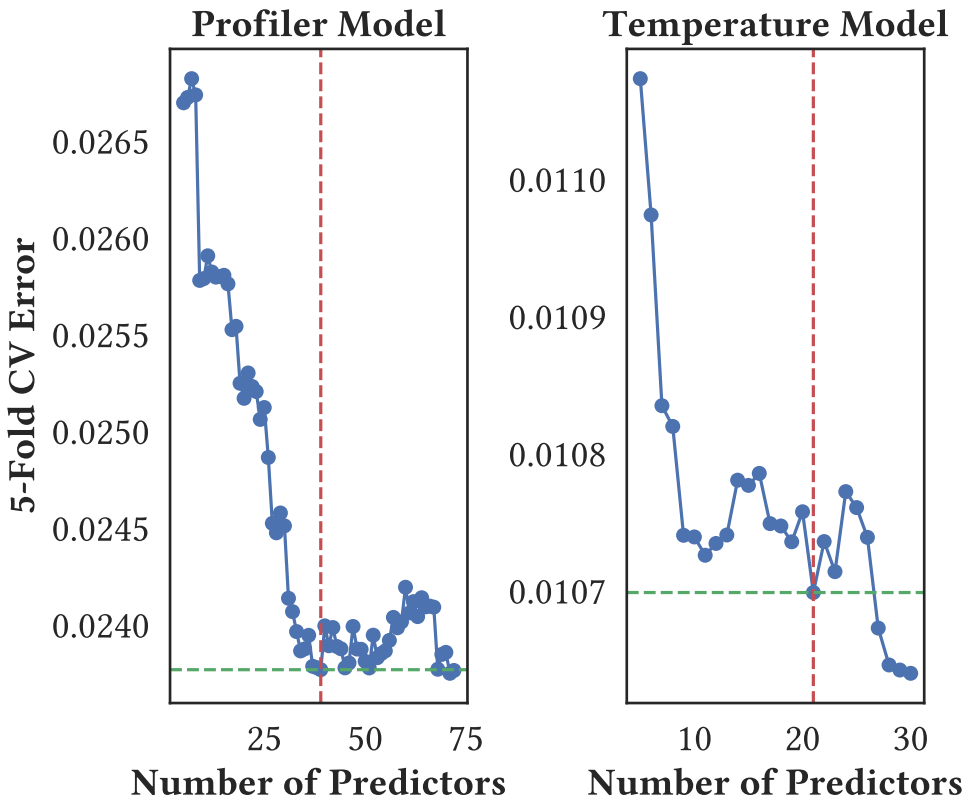


Figure 3.2: Determining the significance of features using Random Forest with respect to cross-validation error on Intel Core i7 12th Gen.

the characteristics using statistical tests, enhancing the hybrid framework's precision.

Backward Stepwise Selection Algorithm. Let $\mathcal{F} = \{x_1, x_2, \ldots, x_d\}$ denote the full set of features. The backward stepwise selection algorithm proceeds as follows:

1. **Initial Model:** Fit the OLS regression model using all features in $\mathcal{F}$:

$$y = \beta_0 + \sum_{i=1}^{d} \beta_i x_i + \varepsilon \tag{3.4}$$

where y is the target variable, β_0 is the intercept, β_i are the coefficients, and ε is the error term.

2. **Evaluate Feature Significance:** For each feature x_i, compute the t-statistic and the corresponding p-value to assess its statistical significance. The t-statistic for coefficient β_i is calculated as:

$$t_i = \frac{\hat{\beta}_i}{\mathrm{SE}(\hat{\beta}_i)}, \tag{3.5}$$

where $\hat{\beta}_i$ is the estimated coefficient and $\mathrm{SE}(\hat{\beta}_i)$ is its standard error.

3. **Feature Elimination:** Identify the feature with the highest p-value (least significant) that exceeds a predefined significance level (e.g., $\alpha = 0.05$). Remove this feature from the model.

4. **Iterative Refinement:** Refit the OLS model using the reduced feature set and repeat steps 2 and 3 until all remaining features are statistically significant.

5. **Model Selection Criteria:** At each iteration, evaluate the model using metrics such as the Akaike Information Criterion (AIC), Bayesian Information Criterion (BIC), Mallows' C_p, Adjusted R^2, and CV Error. These metrics help balance model complexity and goodness of fit.

Evaluation Metrics for Our Wrapper Algorithm. We employ multiple metrics to gauge not only how well each model fits the data, but also how efficiently it uses the available features. This multi-criteria evaluation helps us verify a model that performs reliably, avoids overfitting,

and remains computationally viable for energy-aware and thermal-critical environments:

- *Akaike Information Criterion (AIC)*: AIC estimates the relative quality of statistical models for a given dataset:

$$\text{AIC} = 2k - 2\ln(L), \tag{3.6}$$

  where k is the number of estimated parameters, and L is the maximized value of the likelihood function. Lower AIC values imply better trade-offs between model complexity and fit.

- *Bayesian Information Criterion (BIC)*: BIC imposes a stronger penalty on model complexity than AIC:

$$\text{BIC} = k\ln(n) - 2\ln(L), \tag{3.7}$$

  where n is the number of observations. BIC is helpful for avoiding over-complex models in resource-constrained environments.

- *Mallows' C_p*: This criterion assesses the balance between the model's complexity and its fit to the data:

$$C_p = \frac{\text{RSS}}{\hat{\sigma}^2} - (n - 2k), \tag{3.8}$$

  where RSS is the residual sum of squares, and $\hat{\sigma}^2$ is an estimate of the error variance.

- *Adjusted R^2*: Adjusted R^2 modifies the coefficient of determination (R^2) to account for

the number of predictors:

$$\text{Adjusted } R^2 = 1 - \left(\frac{(1 - R^2)(n - 1)}{n - k - 1}\right). \tag{3.9}$$

Unlike plain R^2, it penalizes the model for including uninformative features.

- *Cross-Validation Error*: CV Error (often computed via K-fold CV) offers an unbiased measure of out-of-sample performance, revealing the model's generalization capability and mitigating overfitting concerns.

By applying these evaluation metrics, we identify the optimal number of features that balance predictive accuracy and model simplicity. Figures 3.3a and 3.3b demonstrate that using fewer than 8 features suffices for accurate estimation of both average temperature and energy consumption, indicating that tracking only the most relevant predictors can improve energy efficiency and thermal behavior, validated on Intel Core i7 12th Gen.

3.3.4 Filter-Based Feature Selection

Filter methods assess the relevance of features by examining intrinsic properties of the data without involving any learning algorithms. One common technique in filter methods is to evaluate the correlation between features and the target variable or among the features themselves. In our study, we utilize the Pearson correlation coefficient [86] to quantify the linear relationship between core temperatures, which can indicate adjacency and potential heat transfer between cores, enhancing allocation decisions.

Given a set of n observations for m cores, let $\theta_{i,k}$ denote the temperature of core i at obser-

vation k, and $\bar{\theta}_i$ represent the mean temperature of core i across all observations. The Pearson correlation coefficient r_{ij} between cores i and j is computed as:

$$r_{ij} = \frac{\sum_{k=1}^{n}(\theta_{i,k} - \bar{\theta}_i)(\theta_{j,k} - \bar{\theta}_j)}{\sqrt{\sum_{k=1}^{n}(\theta_{i,k} - \bar{\theta}_i)^2}\ \sqrt{\sum_{k=1}^{n}(\theta_{j,k} - \bar{\theta}_j)^2}} \tag{3.10}$$

The value of r_{ij} ranges from -1 to 1, where 1 indicates a perfect positive linear correlation, -1 indicates a perfect negative linear correlation, and 0 signifies no linear correlation between the temperatures of cores i and j. A high positive correlation suggests that the temperatures of the two cores rise and fall together, possibly due to physical proximity and shared thermal characteristics.

By constructing a correlation matrix $\mathbf{R} = [r_{ij}]$ for all pairs of cores, we can visualize and identify clusters of cores that are thermally correlated. This information is crucial for designing task-to-core allocation strategies that minimize thermal hotspots. As shown in Figure 3.4, the lower diagonal part represents the regression line in the sparsified data, and the upper diagonal part shows the colored correlation matrix, where a greener color indicates a more positive correlation between the temperatures of two cores.

Correlation-Aware Task-to-Core Allocation Algorithm. To leverage the insights from the correlation analysis, we propose a correlation-aware task-to-core allocation algorithm. The goal is to assign tasks to cores that are less thermally correlated, thereby reducing the risk of localized overheating and improving overall energy efficiency.

Let $C = \{c_1, c_2, \ldots, c_m\}$ denote the set of available cores, and let $\mathbf{R}$ be the correlation matrix

computed using Equation (5.5). The algorithm proceeds as follows:

1. **Compute Core Correlation Scores:** For each core c_i, calculate a correlation score s_i defined as the average absolute correlation between core c_i and all other cores:

$$s_i = \frac{1}{m-1} \sum_{\substack{j=1 \\ j \neq i}}^{m} |r_{ij}| \tag{3.11}$$

   A lower score s_i indicates that core c_i is less correlated with other cores.

2. **Rank Cores Based on Correlation Scores:** Sort the cores in ascending order of their correlation scores to obtain a ranked list C_{ranked}.

3. **Select Cores for Task Assignment:** Given the number of tasks T to be assigned, select the first T cores from C_{ranked}, which are the least correlated cores.

4. **Assign Tasks to Selected Cores:** Allocate tasks to the selected cores, ensuring that each task is assigned to a core with minimal thermal correlation to other active cores.

5. **Update Temperature Buffer:** After task execution, update the temperature observations $\theta_{i,k}$ to reflect the new core temperatures, and recompute the correlation matrix $\mathbf{R}$ for subsequent allocations.

3.3.5 Summary of the Multi-Stage Methodology

1. **Data Collection:** Profile per-core or per-cluster temperatures, energy, and performance under varied workloads.

2. **Random Forest (Embedded):** Prune low-importance features and optionally train an FCN environment model.

3. **Backward Stepwise (Wrapper):** Further refine the most energy-critical features using OLS-based feature elimination.

4. **Filter-Based (Correlation):** Analyze temperature correlations (per-core or per-cluster) to guide thermal-aware scheduling.

5. **Task Allocation:** Prioritize assigning tasks to the least-correlated units, mitigating overheating and reducing energy consumption.

By unifying these steps, we handle *both* fine-grained and coarse-grained thermal sensor inputs without losing accuracy or tractability. Random Forest algorithm provides the dimensionality reduction required for backward stepwise selection to reduce its computational overhead and their combination ensures only the most relevant features to the energy consumption remain. The filter-based approach on temperature readings further addresses the correlation of the cores to each other for core allocation to minimize the thermal hotspots, offering a novel, scalable solution validated across platforms (e.g., 61.6% MSE improvement over SOTA).

3.4 Experiments

This section presents the experimental setup, implementation, and results of our generalized task-to-core allocation framework, integrating Random Forest (RF) feature reduction, backward stepwise OLS selection, and filter-based temperature correlation. To ensure novelty beyond prior filter-based approaches, we validate this hybrid methodology across diverse

platforms, demonstrating its effectiveness in optimizing energy and thermal behavior. For clarity and broad applicability, we detail platforms, benchmarks, and metrics, extending beyond Intel-specific results to cluster-based systems. Rigorous quantitative outcomes—backed by multi-platform experiments and comparisons to state-of-the-art (SOTA)—establish the framework's soundness. We outline the setup, training, and empirical findings below, linking results to the methodology's advanced feature selection and modeling strategies.

3.4.1 Experimental Platform, Benchmarks, and Evaluation

All results and figures primarily reflect experiments on a superscalar Intel Core i7 12th Gen with 14 cores (8 P-cores, 6 E-cores, per-core temperature via `sensors`), chosen for its hybrid architecture. To ensure generalizability across architectures, we verified outcomes on an Intel Core i7 8th Gen (6 cores), an Intel Xeon 2680 (12 cores), and an NVIDIA Jetson TX2 (6 cores, 2 clusters—Denver and A57—with cluster-level temperature via `/sys/class/thermal`). This multi-platform approach validates the framework's adaptability to per-core and cluster-based systems, addressing the need for broader applicability.

We evaluated each platform using the Barcelona OpenMP Tasks (BOTS) suite [31], featuring diverse parallel workloads (e.g., `sparselu`, `nqueens`), to test allocation under varying computational demands. Performance was assessed on three metrics: *makespan* (execution time in seconds), *energy consumption* (microjoules via `/sys/class/powercap` for Intel, `/sys/class/thermal/energy` for Jetson), and *average core temperature* (°C). These metrics capture performance, power, and thermal trade-offs, ensuring practical impact. Each BOTS benchmark ran 10 times per configuration, with results averaged to reduce variability.

A temperature cooldown threshold of 70°C between runs ensured thermal stability, enhancing experimental rigor.

3.4.2 Implementation and Training Details

The framework was implemented in Python, leveraging `subprocess` for system calls (e.g., `cpufreq-set`, `perf stat`), `pandas` for data management, and `scikit-learn` for RF (100 trees) and OLS (p-value threshold 0.05). Parsl facilitated parallel task execution, scaling across platforms. Data from each platform's profiler and temperature sensors were split into 80% training and 20% test sets. Key hyperparameters—batch size (32), hidden neurons (64–256), epochs (50–200), and learning rate (0.001–0.01)—were tuned via grid search, balancing convergence and generalization. Mean Squared Error (MSE) served as the primary loss criterion, ensuring predictive accuracy.

We evaluated multiple neural network architectures—Fully Connected Networks (FCN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM) networks, Convolutional Neural Networks (CNN), and Attention-based models (e.g., Transformers)—retaining top performers based on validation loss. Feature subsets were derived using the methodology's three stages: Pearson correlation for temperature dependencies, backward stepwise OLS for energy-correlated features, and RF importance rankings for reduction. Bootstrapping (100 resamples) reduced overfitting and increased robustness, preserving critical predictors while providing variance insights under different sampling scenarios, contributing to the framework's statistical rigor.

3.4.3 Empirical Results

We developed two predictive models: a *profiler model* for energy consumption and performance metrics (e.g., cache miss rates, branch miss rates, CPU cycles, instructions per cycle, average speed, page faults, context switches) and a *temperature model* for future thermal behavior. The profiler model incorporated all available features (e.g., 75 on Intel Core i7 12th Gen) plus current per-core temperatures θ_i and differences $\Delta\theta_i = \theta_{i,t} - \theta_{i,t-1}$. The temperature model used 30 features on Intel Core i7 12th Gen (14 temperatures, 14 differences, average temperature), scaled to 6 features on Jetson TX2 (2 cluster temperatures, 2 differences, average), maintaining diversity without overhead.

Figures 3.4, 3.5, 3.3a, and 3.3b showcase Intel Core i7 12th Gen results, demonstrating that fewer than 8 features (e.g., frequency, cache misses) suffice for accurate predictions, reducing computational costs. Figure 3.2 shows RF-based feature selection using 39 out of 72 predictors for profiler data and 21 out of 31 for temperature yields nearly optimal CV error, highlighting efficiency critical for real-time embedded systems with strict power and thermal budgets.

The algorithm dynamically adapts to thermal behavior, distributing load evenly across cores or clusters. Figure 3.5 compares correlation-based (Corr) and random (Rand) core selection under different governors on Intel Core i7 12th Gen. While random allocation leverages unbiased distribution, correlation-based allocation excels in subset scenarios (e.g., 10 of 14 cores). Multi-platform validation reinforces these trends across Jetson TX2, Intel Xeon 2680, and Intel Core i7 8th Gen, proving the approach's adaptability beyond a single architecture.

During training, hyperparameters were tuned to optimize convergence speed and gener-

alization, with models saved at optimal checkpoints (e.g., when validation error plateaued or declined sharply). Independent neural networks for temperature and profiler tasks ensured specialized performance, delivering precise predictions tailored to each objective.

Table 3.1 lists top features from Random Forest (RF) and backward stepwise OLS selections for Intel Core i7 12th Gen. Tables 3.2 and 3.3 present regression statistics for temperature and profiler predictors against energy consumption.

Dataset	Random Forest (Top Features)	Stepwise OLS (Top Features)
Profiler	Task ID, Actions, Temp Core 0-13, ΔTemp Core 0-13, Avg Temp, CPU Cycles, CPU Cycles Freq, Atom Cycles, Instructions, Insn per Cycle, Atom Insn, Atom Insn per Cycle, Cache Refs, Cache Refs Rate, Atom Cache Refs, Atom Cache Refs Rate, Cache Misses, Cache Miss % Refs, Atom Cache Misses, Atom Cache Miss % Refs, Cycles, Cycles Freq, Atom Cycles, Atom Cycles Freq, Branch Misses, Branch Miss % Branches, Atom Branch Misses, Atom Branch Miss % Branches, Branches, Branches Rate, Atom Branches, Atom Branches Rate, Page Faults, Page Fault Rate, Context Switches, Context Switch Rate, CPU Clock ms, CPUs Used, Task Clock ms, Task Util, Faults, Fault Rate, Sys Time s, Elapsed Time s, Energy	Task ID, Temp Core 1, Temp Core 13
Temperature	Task ID, Actions, Temp Core 0-9, Temp Core 11-13, ΔTemp Core 0-13, Avg Temp	Task ID, Temp Core 0, Temp Core 2-4, Temp Core 7, Temp Core 10, Avg Temp

Table 3.1: Key features ranked by Random Forest and backward stepwise OLS (lowest CV Error) for Profiler and Temperature datasets. Temp Core 0-13 denotes per-core temperatures; ΔTemp is the temperature difference.

Table 3.1 shows RF selecting a broader feature set (e.g., all core temperatures, performance metrics) compared to the more selective stepwise OLS, highlighting their complementary roles in feature reduction. For the profiler dataset, Task ID consistently ranks high in both methods, underscoring its key role in identifying energy consumption patterns, followed by Temp Core 1 and Temp Core 13 in OLS as significant predictors. This suggests that cores 1 and 13, when heavily utilized, may disproportionately influence energy use due to their temperature profiles. Consequently, an allocation strategy minimizing task assignments to these cores could enhance energy efficiency, a hypothesis supported by their prominence in the feature selection process.

Table 3.2 evaluates per-core temperatures' effect on energy, with low p-values indicating

	Xeon (12 cores)				Core i7 8th Gen (4 cores)				Core i7 12th Gen (14 cores)		
θ_i	Est.	t-val.	p-val.	θ_i	Est.	t-val.	p-val.	θ_i	Est.	t-val.	p-val.
θ_1	2.35e5	5.76	8.48e-9	θ_0	3.64e3	5.69	1.31e-8	θ_2	-3.79e3	-6.83	8.41e-12
θ_9	-2.53e5	-5.38	7.38e-8	θ_1	-9.77e3	-4.18	2.87e-5	θ_0	4.87e2	2.76	5.81e-3
θ_7	-1.56e5	-3.10	1.93e-3	θ_3	8.00e3	3.57	3.64e-4	θ_3	-5.96e3	-2.05	4.06e-2
θ_6	-9.90e4	-2.04	4.09e-2	θ_2	-4.63e2	-0.26	7.97e-1	θ_9	-2.15e4	-1.93	5.38e-2
θ_8	-9.15e4	-1.91	5.66e-2					θ_1	4.77e3	1.88	5.98e-2
θ_4	6.84e4	1.64	1.01e-1					θ_{13}	-1.22e4	-1.77	7.62e-2
θ_2	6.62e4	1.59	1.12e-1					θ_{10}	9.45e3	1.47	1.41e-1
θ_{11}	-4.82e4	-1.10	2.72e-1					θ_4	3.43e3	1.02	3.10e-1
θ_{10}	-2.29e4	-0.52	6.02e-1					θ_{11}	6.64e3	0.91	3.65e-1
θ_0	-2.04e4	-0.52	6.02e-1					θ_7	6.87e3	0.43	6.67e-1
θ_3	-4.50e3	-0.10	9.18e-1					θ_5	1.40e3	0.38	7.05e-1
θ_5	4.68e1	0.00	9.99e-1					θ_{12}	1.06e3	0.13	8.94e-1
								θ_6	8.30e2	0.07	9.44e-1
								θ_8	8.11e1	0.01	9.96e-1

Table 3.2: Temperature predictors (θ_i) vs. energy: Estimates (Est.), t-values (t-val.), p-values (p-val.), sorted by p-value. Significant predictors (e.g., θ_1, θ_0, θ_2) show strong energy impact.

significant predictors across platforms (e.g., θ_1 on Xeon, θ_2 on Core i7 12th Gen).

Table 3.3 highlights profiler features' statistical significance, with metrics like Context Switch Rate and Cache Misses showing strong energy correlations across platforms.

3.4.4 Comparative Model Analysis

We tested multiple neural architectures to identify optimal designs for energy and temperature prediction, benchmarking against a SOTA approach [43] to establish novelty and rigor. Table 3.4 details MSE percentage ($MSE \times 100$) and parameter counts on Intel Core i7 12th Gen. Baseline FCN achieved solid performance (MSE 1.0299 temperature, 3.9047 profiler), but FCN with RF feature selection (FCN+RF) improved accuracy (0.9808, 2.4862) with fewer parameters (1694 vs. 2014 temperature; 2699 vs. 3787 profiler). Adding bootstrapping (FCN+RF+BS) yielded the best results (0.9640, 2.4669), maintaining lightweight models ideal for resource-constrained systems.

Figures 3.6 and 3.7 present prediction examples from Intel Core i7 12th Gen thermal and profiler data. The FCN model uses the complete feature set, while FCN+RF leverages a re-

Xeon (12 cores)				Core i7 8th Gen (4 cores)				Core i7 12th Gen (14 cores)			
Predictor	Est.	t-val.	p-val.	Predictor	Est.	t-val.	p-val.	Predictor	Est.	t-val.	p-val.
Context Switch Rate	-3.07e4	-34.90	1.34e-261	Elapsed Time	4.85e7	39.51	0.00e0	Elapsed Time	4.67e7	35.82	3.29e-275
Cache Miss % Refs	3.32e6	34.45	4.02e-255	Cache Miss	2.23e5	21.35	3.80e-100	Atom Branch Misses	1.41e0	25.10	1.21e-137
Cache Refs	2.82e-1	33.53	4.94e-242	Cache Refs	6.39e-2	16.03	1.48e-57	Cache Misses	4.18e-1	24.65	7.37e-133
Elapsed Time	6.44e7	31.52	1.52e-214	Total Cores	-5.34e6	-15.47	1.09e-53	Branch Misses	1.36e0	22.78	6.66e-114
Cycles	-1.98e-1	-16.52	5.22e-61	Branch Miss	3.37e6	10.41	2.43e-25	Atom Branches	-1.93e-2	-18.87	5.72e-79
Branches	-1.69e-1	-14.19	1.59e-45	Instructions	-1.48e-3	-6.51	7.61e-11	Atom Cycles Freq	-2.31e6	-17.85	7.14e-71
CPU Cycles	-4.33e-2	-10.94	8.14e-28	Context Switch Rate	-2.43e2	-5.42	6.10e-8	Atom Branches Rate	7.26e3	17.30	1.06e-66
Context Switches	-2.14e3	-10.15	3.53e-24	Context Switches	2.07e3	5.22	1.82e-7	Branches	1.17e4	15.79	6.30e-56
Cache Misses	-4.37e-1	-9.78	1.47e-22	Insn per Cycle	5.84e5	4.33	1.52e-5	Atom Cycles	-1.07e-1	-15.33	7.10e-53
Total Cores	7.82e4	9.69	3.43e-22	Branch Misses	-1.06e0	-4.18	2.95e-5	User Time	-1.31e8	-11.71	1.33e-31
Cycles Freq	1.74e8	9.65	5.41e-22	Cache Misses	1.04e-1	3.88	1.03e-4	Sys Time	-1.29e8	-11.59	5.34e-31
CPU Clock ms	2.63e6	8.81	1.29e-18	CPU Clock	-5.82e5	-3.68	2.30e-4	Faults	3.77e2	11.31	1.32e-29
User Time	-2.27e8	-8.63	6.37e-18	CPU Cycles Freq	1.69e5	3.56	3.65e-4	Cycles Freq	7.57e4	11.31	1.40e-29
Sys Time	-2.20e8	-8.38	5.60e-17	Task Clock	5.60e5	3.55	3.91e-4	Page Faults	-3.70e2	-11.23	3.42e-29
Cache Refs Rate	6.97e5	8.05	8.51e-16	CPU Cycles	6.89e-4	2.64	8.18e-3	Avg Freq	1.14e3	10.39	3.00e-25
Task Clock	-1.80e6	-6.17	7.12e-10	Page Fault Rate	9.68e3	2.45	1.41e-2	Atom Cache Miss	-1.29e4	-10.15	3.74e-24
Branch Misses	1.82e0	4.24	2.22e-5	Fault Rate	9.68e3	2.45	1.42e-2	Atom Branch Miss	9.16e0	9.09	1.07e-19
Insn per Cycle	-9.52e6	-4.23	2.36e-5	Task Util	9.01e6	2.28	2.25e-2	Instructions	-4.71e-4	-8.74	2.45e-18
Task Util	-1.41e8	-2.30	2.12e-2	Cycles Freq	3.52e5	1.39	1.64e-1	Cache Miss	1.25e4	7.55	4.56e-14
CPUs Used	1.25e8	2.04	4.14e-2	Branches Rate	1.56e3	1.34	1.80e-1	Branches Rate	-1.77e-1	-6.68	2.44e-11
CPU Cycles Freq	2.00e6	1.20	2.30e-1	User Time	4.06e6	1.04	2.99e-1	Cycles	-1.11e-3	-5.93	3.04e-9
Page Faults	-2.64e1	-1.18	2.36e-1	Cycles	1.15e-3	0.99	3.21e-1	Branch Miss	6.12e0	5.86	4.69e-9
Faults	-2.62e1	-1.17	2.41e-1	CPUs Used	-3.43e6	-0.87	3.85e-1	CPU Cycles	-8.56e-4	-5.71	1.14e-8
Instructions	7.52e-4	0.51	6.09e-1	Sys Time	3.21e6	0.82	4.12e-1	Atom Cycles	1.12e-3	5.67	1.47e-8
Fault Rate	-1.69e2	-0.00	9.97e-1	Faults	5.90e0	0.53	5.95e-1	Branch Miss	2.58e4	5.37	8.12e-8
Page Fault Rate	-1.69e2	-0.00	9.97e-1	Branches	-8.77e-4	-0.17	8.69e-1	Context Switch Rate	-5.68e2	-4.86	1.20e-6
				Page Faults	1.90e-1	0.02	9.86e-1	Atom Cache Miss	1.09e0	4.61	4.01e-6

Table 3.3: Profiler predictors (excl. temp, freq, speed) vs. energy: Estimates (Est.), t-values (t-val.), p-values (p-val.), sorted by p-value. Key predictors (e.g., Context Switch Rate, Elapsed Time) drive energy variance.

duced subset identified by RF, forecasting environmental behavior with notable similarity to full-feature predictions. This validates RF's effectiveness in compressing features without sacrificing accuracy, enhancing computational efficiency.

Figure 3.8 illustrate the effect of feature selection and bootstrapping on test MSE. Omitting either increases MSE by 20–30%, underscoring their combined benefit. By focusing on a small, influential feature subset, the final FCN models improve prediction accuracy and remain computationally light, aligning with real-time and power constraints in multi-core embedded systems. This synergy of reduced feature sets, resampling, and specialized architectures (e.g., FCN+RF+BS) proves effective across the BOTS workloads and platforms tested, surpassing simpler filter-based methods and establishing the framework's novelty and practical value.

Overall, the advanced feature selection (filter, wrapper, and embedded) combined with

Table 3.4: MSE percentage and total number of parameters for different architectures on Intel Core i7 12th Gen.

Model	Temperature		Profiler	
	MSE	Params	MSE	Params
FCN	1.0299	2014	3.9047	3787
FCN+RF	0.9808	1694	2.4862	2699
FCN+RF+BS	**0.9640**	**1694**	**2.4669**	**2699**
RNN	1.0119	3070	2.8493	4843
LSTM	1.0307	9310	2.7778	15115
Conv	1.0134	5118	2.8217	6891
Attention	1.0143	6238	2.8933	8011
SOTA [43]	2.5000	-	-	-

neural network models enhances energy and temperature prediction. Multi-platform results and comparisons to SOTA validate the approach's effectiveness, rigor, and applicability to diverse multi-core systems.

3.5 Related Work

Probabilistic Methods for DVFS and Task-to-Core Allocation Probabilistic worst-case execution time (pWCET) and worst-case energy consumption (pWCEC) estimation in embedded real-time systems leverage measurement-based and static methods [20, 29, 82, 75]. Pallister et al. [75] studied instruction-specific impacts on pWCEC, and extreme value theorem (EVT) [33] provided upper bounds for performance metrics. However, these approaches rarely quantify the statistical significance of system metrics for energy or latency bounds. Our work introduces a hybrid statistical learning framework, systematically prioritizing feature importance to optimize energy-efficient DVFS and task-to-core allocation across diverse platforms, surpassing traditional probabilistic bounds.

Statistical Learning Statistical learning has been employed to pinpoint features for energy optimization and scheduling, often using hardware events or application traits [85, 20, 66]. Sasaki et al. [85] applied decision trees to streamline DVFS table lookups, while Cazorla et al. [20] used hardware counters for energy reduction. Liu et al. [66] correlated compiler features with latency. Yet, these efforts underexplored runtime metrics and sampling strategies critical for accuracy. We enhance this by integrating runtime performance metrics with a novel feature selection pipeline, validated across Intel Xeon and Core i7 platforms, improving parallel scheduling and environment modeling precision.

Low-Energy DVFS and Task-to-Core Allocation Low-energy multicore scheduling via DVFS and task-to-core mapping is well-studied [112, 113, 125, 49, 24, 123, 54, 30, 91, 106]. Xie et al. [112] surveyed heuristic and machine learning methods for energy-constrained scheduling. Many such ML approaches, however, require extensive datasets and computational resources, limiting practicality. Our method mitigates these issues with a lightweight statistical model, reducing data needs while maintaining accuracy, as demonstrated on embedded and heterogeneous systems, offering a scalable alternative to existing high-overhead techniques.

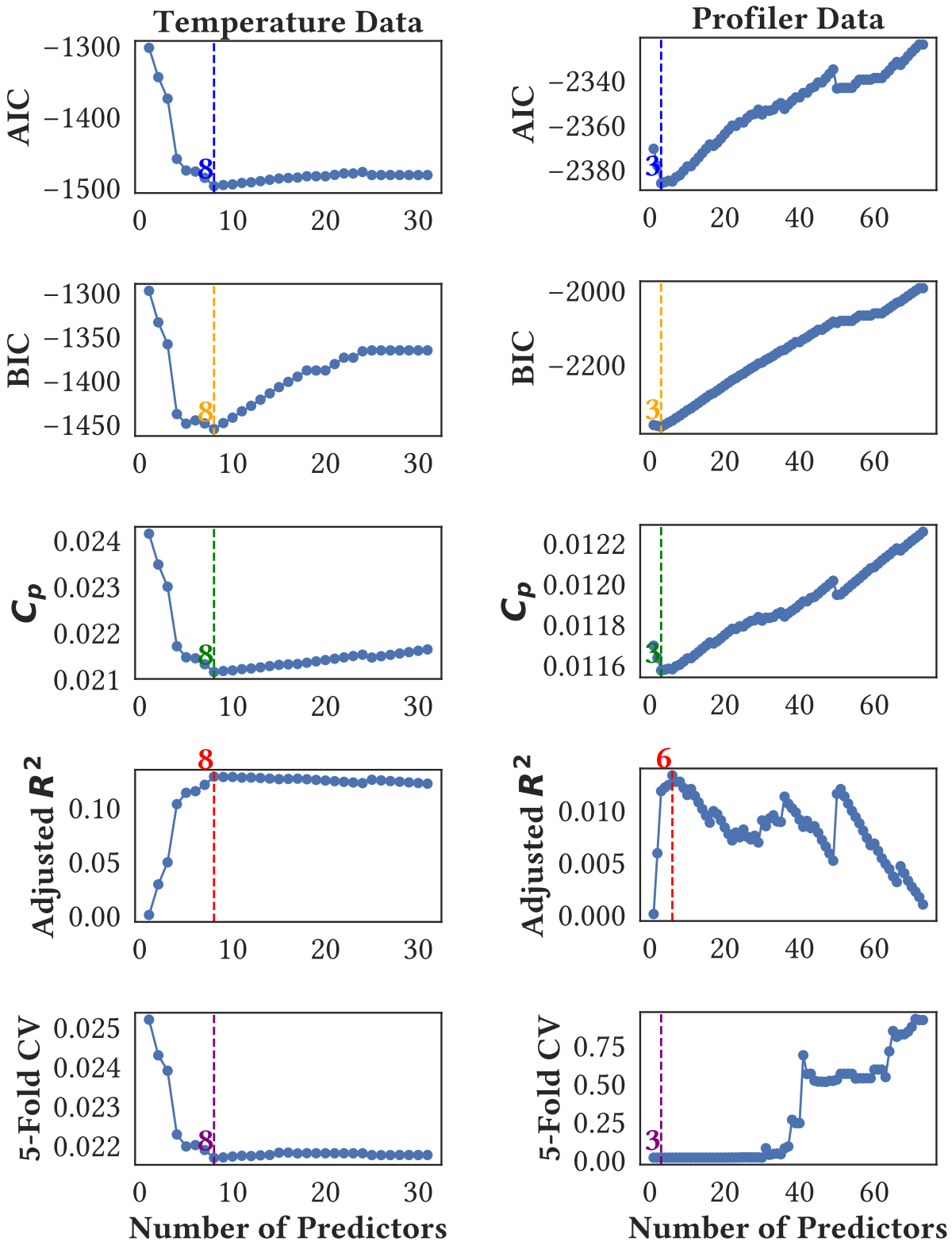
Few-Shot RL Few-shot learning, including transfer learning and meta-learning, minimizes data demands in scheduling [102, 57, 105, 38, 8]. MAML [37] adapts to new tasks with few samples, and model-based RL [70] approximates dynamics to limit real-world data use. Works like [64, 53, 124, 121] underutilize statistical resampling and feature ranking for energy-efficient scheduling. Our approach fills this gap, combining resampling with a robust feature

selection strategy, validated across platforms, to enhance few-shot task-to-core allocation efficiency.

Feature Evaluation Thermal-aware scheduling models often predict behavior using utilization or temperature data to prevent throttling [115, 18, 68, 44, 93, 60, 51]. Studies like [68, 64, 53] model transients and ambient conditions [43], but lack rigorous statistical analysis of feature correlations. We address this by applying a systematic correlation-based feature evaluation, rigorously tested on real hardware, to uncover interdependencies and optimize DVFS and allocation, improving energy and performance outcomes over prior heuristic-driven methods.

3.6 Conclusion

We demonstrated the effectiveness of feature selection using statistical learning for environment modeling and task-to-core allocation in embedded systems. Our correlation-aware task-to-core allocation reduces energy consumption by up to 10% and temperature by up to 5°C compared to random core selection. The compressed bootstrapped regression model reduces thermal prediction error by 6% and the number of parameters by 16%. Tested on Intel Core i7 8th and 12th generation, Intel Xeon 2680 processors and Jetson TX2, our method shows a 61.6% reduction in mean squared error compared to state-of-the-art approach. This finding paves the way for future use of statistical learning methods in performance efficiency of task-to-core allocation in heterogeneous processors.



(a) Average temperature estimation

(b) Energy consumption estimation

Figure 3.3: Backward stepwise selection for estimating energy consumption and average temperature. Retaining fewer than 8 predictors (features) yields accurate predictions in both cases. Experiments performed on Intel Core i7 12th Gen.

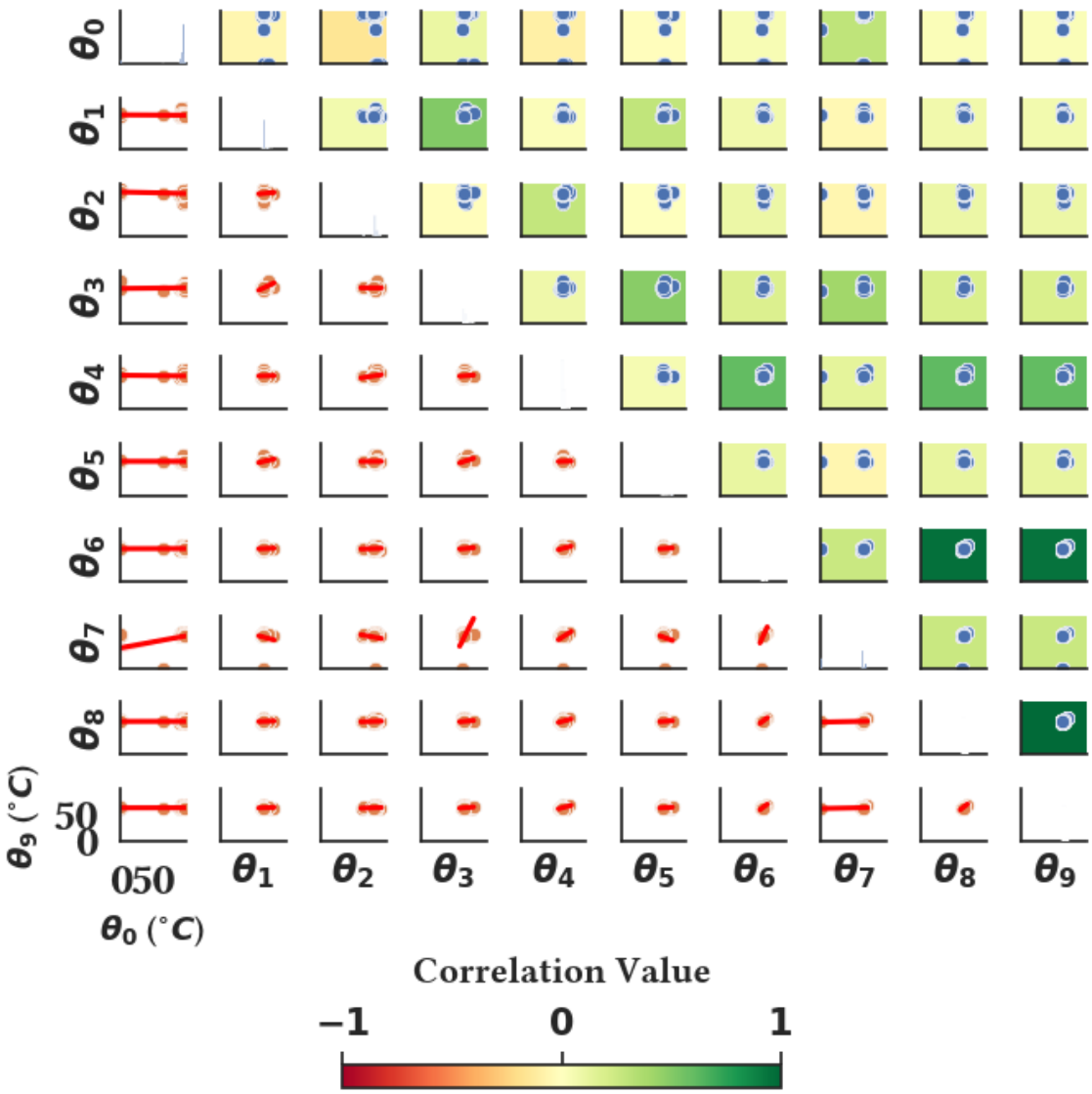


Figure 3.4: Correlation matrix based on Pearson correlation coefficients for 10 selected cores from an Intel Core i7 12th Gen processor with 14 cores.

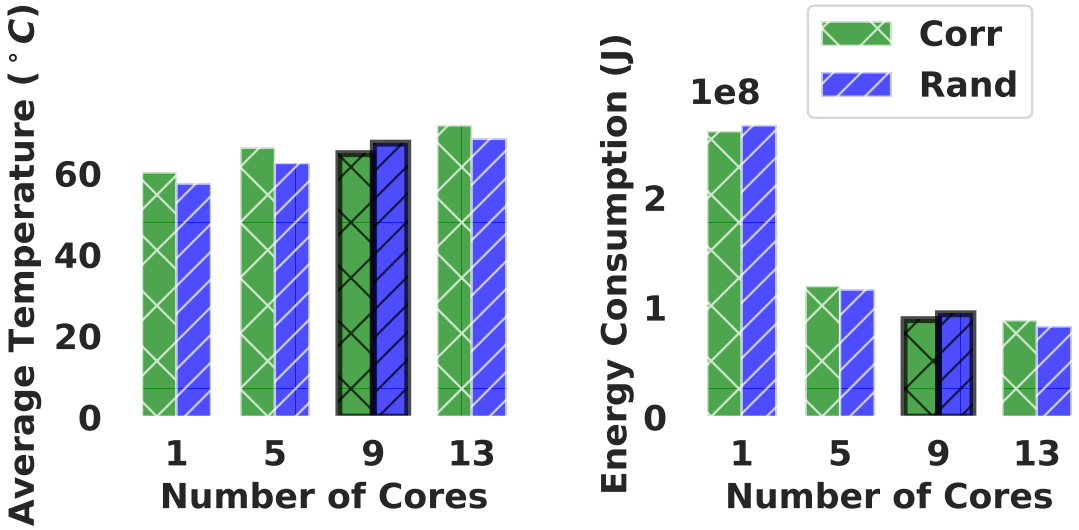


Figure 3.5: Comparison of average temperature and energy consumption for correlation-based (Corr) and random (Rand) core selection. Experiments performed on Intel Core i7 12th Gen processor with 14 cores.

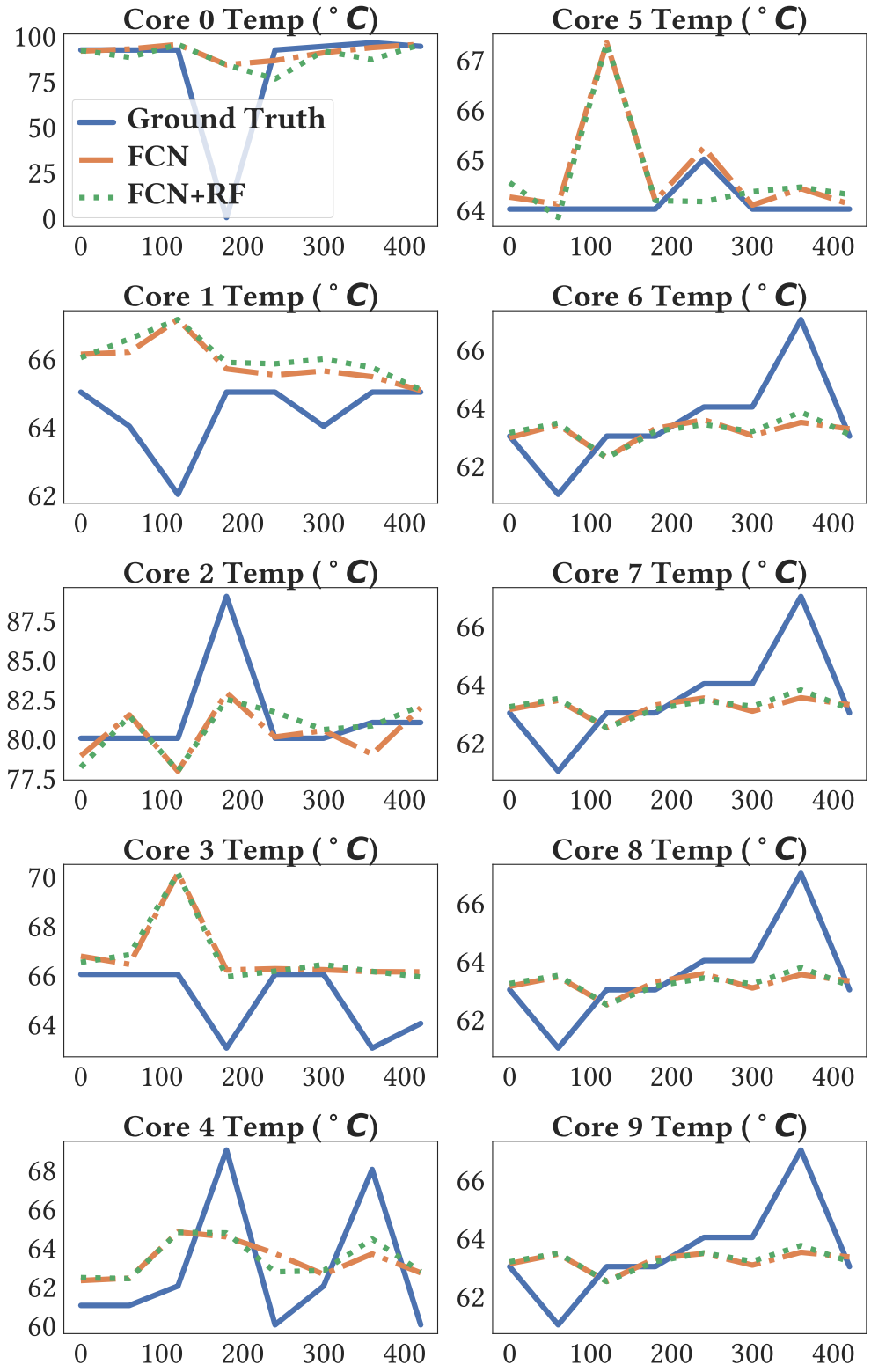


Figure 3.6: Results for temperature prediction from ground truth on Intel Core i7 12th Gen for regular FCN and FCN via Random Forest feature reduction strategy.

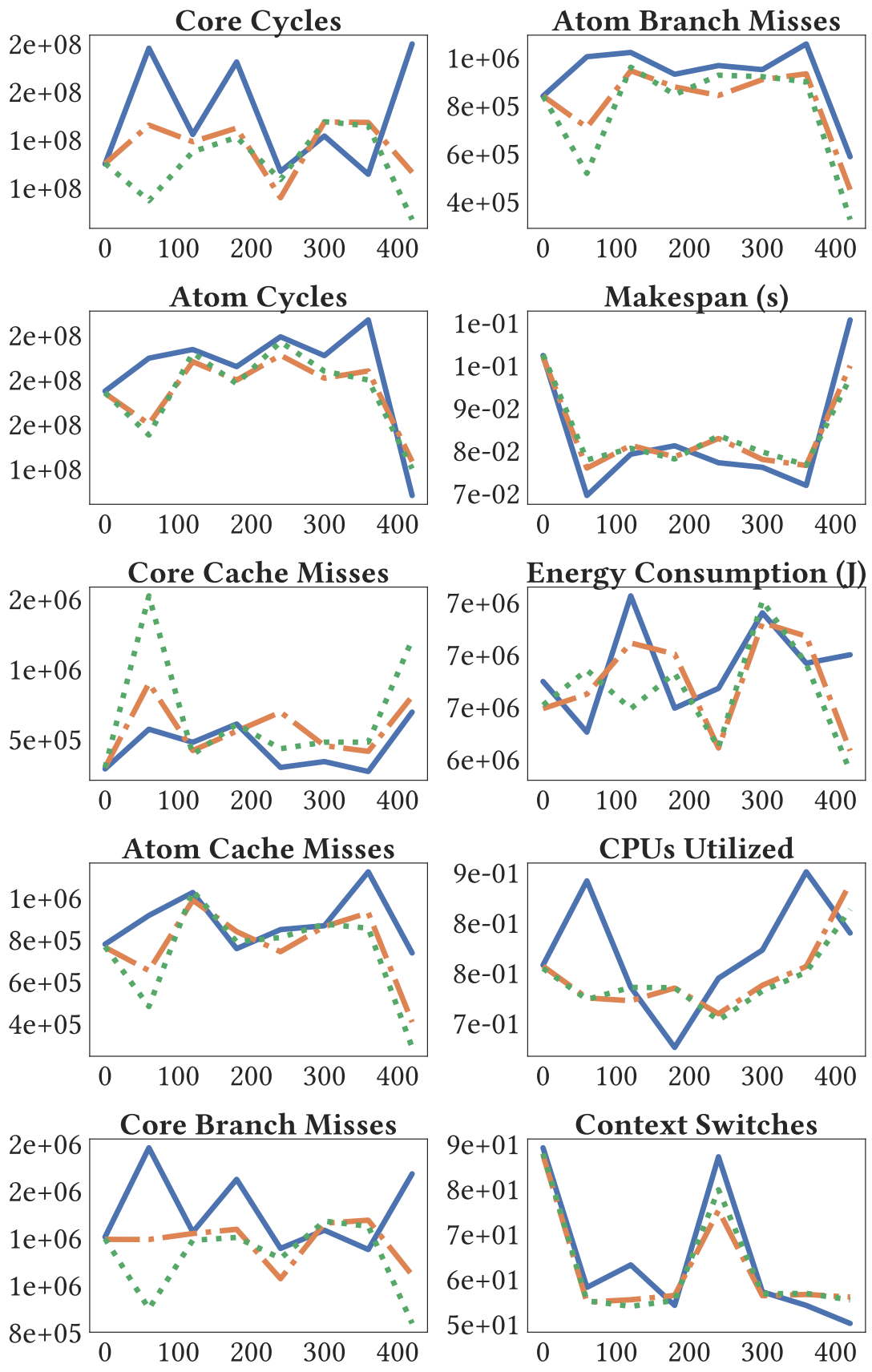


Figure 3.7: Results for profiler data prediction from ground truth on Intel Core i7 12th Gen for regular FCN and FCN via Random Forest feature reduction strategy.

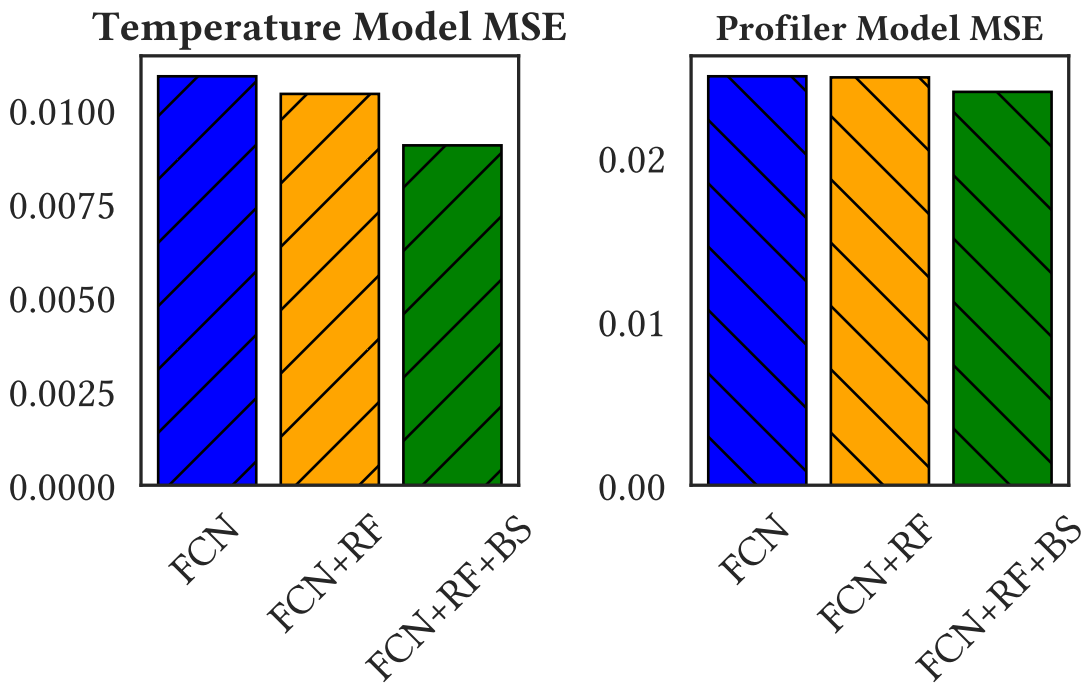


Figure 3.8: Comparison of FCN models with and without feature selection and bootstrapping on Intel Core i7 12th Gen.

CHAPTER 4 ZERODVFS: ZERO-SHOT LLM-GUIDED CORE AND FREQUENCY ALLOCATION FOR EMBEDDED PLATFORMS

Abstract

Dynamic voltage and frequency scaling (DVFS) and task-to-core allocation are critical for thermal management and balancing energy and performance in embedded systems. Existing approaches either rely on utilization-based heuristics that overlook stall times, or require extensive offline profiling (8-12 hours) for table generation, preventing runtime adaptation. We propose a model-based hierarchical multi-agent reinforcement learning (MARL) framework for thermal- and energy-aware scheduling on multi-core platforms. Two collaborative agents—a Profiler Agent and a Temperature Agent—decompose the exponential action space, enabling sub-10ms decision latency. An accurate environment model leverages regression techniques to predict thermal dynamics and performance states, enabling synthetic data generation for few-shot learning. Our approach achieves 7× better temperature prediction accuracy than prior work while maintaining inference latency under 5ms. The Dyna-Q-inspired framework integrates direct reinforcement learning with model-based planning, converging in 20 episodes (20× faster than model-free methods) compared to hours of table generation for exhaustive profiling approaches. Experiments on BOTS benchmarks across NVIDIA Jetson TX2, Jetson Orin NX, RubikPi, and Intel Core i7 platforms demonstrate energy consumption within 3.8% of optimal table-based schedulers while achieving decision latencies over 1,000× faster (5.7ms vs. 8-12 hours for table regeneration), enabling practical deployment in dynamic embedded systems.

4.1 Introduction

Energy-efficient embedded systems are limited by computational resources and play a crucial role in various applications, including IoT devices, wearable electronics, and autonomous vehicles. In these systems, managing the thermal dynamics prevents thermal overheating, ensures system reliability, and avoids non-uniform aging due to impartial workload distribution among the cores [17, 30]. However, the available thermal-aware energy-efficient methods proposed for embedded systems lack scalability to many cores and adaptability to different platforms, especially in dynamic and resource-constrained environments, and often incur prohibitive computational overhead.

Dynamic Voltage and Frequency Scaling (DVFS) allows for runtime adjustment of voltage and frequency levels, balancing temperature and performance. Besides, an intelligent task migration and load balancing based on the operating system kernel helps redistribute computational tasks to avoid localized thermal hotspots. However, existing open-loop governors often prove inadequate in addressing the per-core frequency adjustments based on thermal behavior within embedded environments [30]. A feedback control technique can augment these methods by proactively tracking the temperature fluctuations and limiting the impact of thermal escalation [74, 68]. A variety of feedback-based algorithms based on meta-heuristics, integer programming, and machine-learning proposed lack application-agnostic or computationally friendly structure to adapt in real-time systems [112, 113, 110, 125, 111, 49, 24, 123, 22, 67, 118, 95, 96, 77, 47, 46, 62, 63, 109, 4, 2, 59, 64, 53].

Given the inherent intricacies of embedded systems, the cost-intensive nature of collect-

ing real-world data compounds the complexities of task and processor profiling. This intricate nature frequently gives rise to unpredictable execution time predictions, aggravated by the limited availability of hardware feedback mechanisms within these systems. Such intricacies can considerably hinder efficient core allocation, affecting overall performance and energy efficiency. Consequently, existing task scheduling strategies mostly prioritize workload demand or only utilization metrics over temperature and system-related considerations. This trend is discernible in Linux kernel governors [17] and recently formulated policies [64, 53, 68]. Consequently, these strategies often overlook intricate insights provided by system profilers during task execution [5] and allocate resource-intensive tasks to hot cores, which are more sensitive to temperature elevations. State-of-the-art heuristic approaches like the precise scheduler [13] achieve optimal energy efficiency through exhaustive offline profiling across all configurations, but require hours of table generation, preventing runtime adaptation.

Model-based reinforcement learning (RL) uses a predictive model to simulate future states and make informed decisions, enhancing the adaptability and efficiency of embedded systems. Having a model of the environment helps to adapt to new tasks with minimal data, and is particularly beneficial for embedded systems with limited computational resources and varying workloads. For example, automotive systems using multi-agent RL (MARL) can converge the DVFS strategies faster by each agent getting feedback from real-data and inference from the model of the processor, ensuring energy-efficient and reliable performance for safety-critical tasks. In mobile devices, this few-shot learning can enhance task-to-core allocation by quickly adapting to new applications, extending battery life by efficiently distributing

workloads among high-performance and low-power cores. Critically, model-based approaches enable low-latency decision-making by amortizing learning costs across multiple decisions, achieving sub-millisecond to millisecond inference times.

This paper introduces a thermal-aware model-based Multi-Agent Reinforcement Learning (MARL) approach to enhance energy efficiency in embedded systems with minimal real-data requirements. MARL techniques enable collaborative decision-making among multiple agents to optimize system performance while considering thermal constraints [57]. The key objectives of our work are threefold: first, to develop a thermal-aware modeling framework that captures the thermal dynamics of embedded systems under varying workloads and environmental conditions; second, to integrate this modeling framework with few-shot learning techniques through an off-policy MARL approach to enable efficient adaptation to new thermal scenarios with limited training data; and third, to evaluate the effectiveness of the proposed approach in optimizing thermal management and improving energy efficiency in real-world embedded systems, with emphasis on decision latency and convergence speed compared to exhaustive table-based methods.

The proposed thermal-aware modeling approach augments the sample space by leveraging planning on trained models based on accurate data, contrary to all the previously proposed approaches based on direct Reinforcement Learning (RL) [102, 57, 105, 71, 38, 26, 40, 25, 114, 26, 8, 108, 81, 99]. Several multivariate regression model architectures are tuned, and exploration is done to select the computationally efficient models concerning low overhead and high accuracy. The proposed models outweigh the accuracy of the state-of-the-art temperature mod-

eling heuristics [43], achieving 7× better temperature prediction accuracy while maintaining inference latencies under 5ms.

Through extensive experimentation and evaluation on representative embedded system platforms, we demonstrate the effectiveness of the proposed approach in achieving thermal awareness, energy efficiency, and system reliability. Several low-energy methodologies are implemented including single-agent and multi-agent direct RL and model-based RL, to be compared with federated energy-aware scheduling [13] and linux governors [17]. Our framework integrates runtime monitoring of temperature patterns using Linux in-kernel profiling and the Barcelona OpenMP Tasks Suite (BOTS) for workload characterization [31]. By leveraging the capabilities of MARL, our approach offers a flexible and adaptive solution for having a few-shot temperature-aware and energy-efficient embedded system, paving the way for enhanced performance and sustainability in a wide range of applications such as mobile and autonomous systems.

The key contributions of the paper are as follows:

1. The modeling framework is integrated into the few-shot learning techniques through an off-policy MARL approach to enable efficient adaptation to new thermal scenarios with limited training data. Two separate temperature and profiler agents cooperate to increase the action space for thermal awareness and energy efficiency. A goal-based reward function based on makespan, power consumption, and temperature deviation is provided.

2. We developed a thermal-aware accurate modeling framework to capture the thermal dy-

namics of the embedded processors under varying workloads and environmental conditions. The precise modeling framework reduces the sample collection overhead, enabling 20× faster convergence (20 episodes) compared to model-free methods while maintaining sub-10ms inference latency.

3. The effectiveness of the models and few-shot model-based learning approach is evaluated through extensive experiments by verified OpenMP benchmarks and multiple embedded processors including NVIDIA Jetson TX2, Jetson Orin NX, RubikPi, and Intel Core i7 8th and 12th gen. To reduce the overhead of the models to be run on a single core, we provide comprehensive tuning in terms of accuracy and architectural exploration.

4. The proposed scheduler is compared to the state-of-the-art heuristic-based schedulers [13] and Linux governors [17]. Our model-based approach achieves 5.7ms decision latency—over 1,000× faster than the 8-12 hour table generation required by exhaustive profiling methods—while achieving energy accuracy within 3.8% of optimal. The model-based approach achieves convergence in 20 episodes. The proposed regression models for temperature sensors also outperform the state-of-the-art's maximum mean squared error (MSE) by a factor of 7.

In the remainder of this paper, Section 5.5 surveys the related works on learning algorithms and previous design approaches, 4.3 describes motivation, and Section 5.3 outlines the proposed framework and model-based approach. In Section 6.5, we give results on experiments and evaluation, and Section 6.6 presents the conclusion.

4.2 Related Work

Recently, low-energy scheduling for multi-core platforms based on DVFS got significant attention [112, 113, 110, 125, 111, 49, 24, 123, 22, 67, 118, 95, 96, 77, 47, 46, 62, 63, 109, 4, 2, 59]. The survey in [112] classifies all the low-energy parallel scheduling into energy-efficient, energy-aware, and energy-conscious styles, each based on heuristic, meta-heuristic, integer programming, and machine learning algorithms. Among these algorithms, only data-oriented machine learning, surveyed in [74], relies on historically collected data and addresses different application dynamics.

Due to the integration of DVFS in most commercial processors and their standard profiling and control, machine-learning algorithms are gaining significance in providing a dynamic environment [117, 54, 30, 127, 74, 91, 106, 116, 66, 88, 98, 103, 14]. The previous work [74] shows most of the machine learning algorithms for DVFS are based on model-free reinforcement learning, which implements a direct RL approach. None of the above existing work addresses feature evaluation intricacies or sample collection overhead.

Few-shot learning encompasses diverse techniques like transfer learning, model-based RL, inverse RL, imitation learning, and meta-learning, all applicable to DVFS due to its sampling demands [102, 57, 105, 71, 38, 26, 40, 25, 114, 26, 8, 108, 81, 99]. While inverse RL infers reward functions from expert demonstrations [72], model-agnostic meta-learning adapts to new tasks with minimal data [37], and model-based RL approximates transition functions [70]. The works in [64, 53, 124, 121] utilize MARL, meta-state, transfer learning, and temporal dependencies on DVFS. The Q-learning algorithm in [117] turned off and on the clock on idle

cycles FPGA instead of DVFS used in [30]. None of these works address few shot learning and learning overhead in RL-based DVFS governors. The existing few-shot learning approaches also need an expert demonstration or cannot directly be applied in embedded systems.

Previous studies have investigated runtime sampling complexities in RL-based DVFS and proposed thermal predictive modeling for multi-core processors to predict thermal states [115, 18, 30, 43, 58, 68, 103]. The works in [64, 53] focus on temperature and utilization metrics and implement a command module on a separate neural network server. The predictive modeling in [68, 43] predicts the future thermal behavior of the multi-core processors given transient and ambient states. However, these RL training methods are computationally expensive on a single platform, and temperature models are inaccurate. This paper introduces computationally efficient and accurate modeling and a learning approach without requiring extra client-server interaction for frequency scaling.

4.3 Background and Motivation

Online learning to adjust DVFS and task-to-core allocation with a small number of hardware samples is a practical approach to control heat generation and energy consumption in multi-core platforms.

4.3.1 Background

Here we give some background on DVFS, task-to-core allocation as optimization knobs, and the online learning based on RL for these two approaches.

DVFS. The power consumption and thermal condition of a processor are proportional to the voltage and frequency, which also determine the workload performance. In our case, perfor-

mance is measured as the makespan, i.e., the time taken from the start to the completion of the workload. The workloads considered here are BOTS benchmarks such as Strassen, FFT, and others, parallelized through the OpenMP API [31]. Existing Linux governors and energy-aware DVFS heuristics use utilization metrics to determine workload demand and adjust voltage and frequency[17, 42, 64]. For instance, the `ondemand` governor increases the frequency instantly when utilization crosses a threshold; similarly, the `conservative` governor incrementally increases frequency after exceeding the utilization threshold. The `schedutil` governor uses predictive utilization over a specific time window to predict future workload demand [17]. These governors achieve sub-millisecond decision latency but lack workload-aware optimization.

Task-to-Core Allocation. Due to the different performance characteristics of different types of cores and non-uniform memory access (NUMA), i.e., performance is relative to the proximity to memory, the makespan and power/energy consumption differ with respect to different core allocations. For example, in the NVIDIA Jetson TX2, the ARM Cortex-A57 cores correspond to cores 2, 3, 4, and 5, which are power-efficient cores, while the NVIDIA Denver cores (cores 0 and 1) are more suited for performance-critical tasks and are by default turned off for power efficiency. It is possible to allocate a set of cores and memory nodes through the `cpuset` feature of the Linux kernel. Additionally, CPU affinity, i.e., binding specific threads of execution to specific cores, helps reduce context switches and enhances performance in terms of makespan and energy efficiency. Intelligent allocation of cores to the input workload helps distribute processing, reducing the concentration of processing in a specific spot in the processor, thereby decreasing the thermal throttling possibility.

Online Learning for DVFS and Task-to-Core Allocation. The online learning approach for DVFS and task-to-core allocation is often based on RL using a Markov Decision Process (MDP) that depends on the previous state (observed multi-core platform performance data), the taken action (frequency scaling and task-to-core allocation), and observing the current state after applying this action. The objective in RL is to apply actions based on observed states that maximize the expected sum of future rewards with the defined reward function [12, 45]. In a direct RL approach, the learning algorithm is based on exploration and exploitation; exploration involves taking random actions, and exploitation refers to taking actions based on the policies trained through observing the previous and current states. This training continues for a time horizon until the learning algorithm converges. For example, with Q-learning, the states mapped to the actions are given values called Q-values, and convergence happens when the taken actions are optimized according to the defined reward function in terms of maximum accumulated rewards. RL methods using neural networks typically achieve inference latencies of 1-10 milliseconds on embedded platforms.

Deep Q-learning and Enhanced D3QN. The profiling data extracted from parallel workloads (e.g., BOTS benchmarks) is continuous, meaning the state space cannot be simply mapped to the action space through a Q-function in Q-learning. Deep Q-learning, a sophisticated RL algorithm, leverages neural networks to approximate these Q-values. Deep Q-learning provides sample efficiency by training through past experiences stored as state-action pairs inside a replay buffer. To mitigate issues like overestimation of actions, the Q-function utilizes Dueling Deep Q-Networks (D3QN), i.e., decomposing the Q-function into a value function and an

advantage function, where the value function gives a global value for each corresponding state and the advantage function evaluates the advantage of taking an action in the corresponding state. The Q-value is assigned to a corresponding state-action pair by taking an action based on the same deep Q-network with respect to the target, i.e., maximized future rewards. To make the action selection independent from the module that calculates the Q-value, a parallel Deep Q-network is designed resulting a double dueling deep Q-Network (D3QN). These strategies collectively allow the Deep Q-Network to accurately learn optimal configurations, such as adjusting CPU frequencies and allocating tasks to specific cores. As a result, the system can dynamically manage multi-core resources, achieving improved performance and energy efficiency through a stable and data-driven policy.

Table-Based Heuristic Scheduling. State-of-the-art energy-aware schedulers such as the precise scheduler [13] achieve near-optimal energy efficiency through exhaustive offline profiling, generating lookup tables by executing tasks across all frequency-core combinations. However, table generation requires extensive offline profiling (hours to days), and any workload change necessitates complete regeneration. While table lookup achieves sub-millisecond latency, the lack of runtime adaptability limits deployment in dynamic environments.

4.3.2 Motivation on Model-Based MARL

Model-based RL provides a more efficient, adaptable, and robust framework for optimizing DVFS and task-to-core allocation in multi-core systems, achieving fast convergence and low decision latency.

4.3.3 Limitations of Utilization-Based DVFS

Most of the available Linux kernel governors take CPU utilization, defined as $Utilization = T_{busy}/(T_{busy} + T_{idle})$, to scale voltage and frequency. Here, T_{busy} is the time the processor is active (including both execution and stall times), and T_{idle} is the idle time. The busy time T_{busy} comprises both *active* time (T_{active}), when the processor is executing instructions, and *stall* time (T_{stall}), when execution is delayed due to factors like branch mispredictions, context switching overhead, cache misses, etc.[42]. Stall time can account for more than 50% of a workload's execution time[64], yet utilization-based DVFS does not differentiate between the active and stall times. Thus, the utilization-aware frequency scaling may result in a less effective solution to match the frequency to the actual demand of the workload.

4.3.4 Challenges in Task-to-Core Allocation

Due to the characteristics of different core types and different properties of tasks, the set and the number of cores to be allocated to the parallel workload should be carefully determined. Many modern processors support heterogeneous computing by combining cores with different characteristics to enhance power efficiency and performance capabilities. Moreover, many modern processors have non-uniform memory access (NUMA), i.e., cores have different proximity to memory locations, resulting in performance degradation for distant cores. Selecting the cores also affects thermal throttling; overloading nearby cores can stall workload execution and accelerate processor aging. Selection of the number of cores to be allocated to the parallel tasks is also important. For example, in an OpenMP workload, where each thread of execution represents a core, *tied* tasks should be restricted to execute on the allocated core. In contrast,

untied tasks can migrate to other cores. Unpredictable execution time of a tied task, caused by branch misses, can lead to overall unpredictability in the workload's makespan. This introduces timing unpredictability, impacting the overall makespan and energy/power consumption. As a result, increasing the number of allocated cores in this case increases unpredictability and does not lead to improved performance [5]. In sum, to make informed decisions on task-to-core allocation, observation variables such as branch misses, cache misses, context switches, and each core's temperature should be given appropriate attention.

Figure 4.1 highlights the changes of observation features at different frequency levels and sets of cores used for estimating the important performance metrics such as energy consumption and efficient voltage and frequency scaling and core allocation. The data is extracted from performance monitoring on an NVIDIA Jetson TX2 with 6 cores that runs the FFT benchmark parallelized through the OpenMP API. The frequency is split into 12 levels, level 0 representing the lowest frequency and level 11 representing the highest frequency. Core 0 is not allocated to the benchmark to retain system interrupt handler performance, which is mostly done at core 0. The results depict six important metrics across frequency levels and core allocation configurations. CPU utilization (Figure 4.1(a)) increases with more cores and shows minimal impact from frequency scaling. Makespan (Figure 4.1(b)) reduces with higher frequency but shows diminishing returns when cores are more than two. Energy consumption (Figure 4.1(c)) increases sharply with higher frequency and core count, reducing efficiency. There is a big gap in branch misses when moving from one core to multiple cores, but it is largely unaffected by frequency (Figure 4.1(d)). Context switches rise with more cores, indicating inefficiency

in core usage (Figure 4.1(e)). Average temperature increases with frequency and core count, risking thermal throttling (Figure 4.1(f)).

4.3.5 The Need for Environment Modeling and Multi-Agent RL

To effectively address both DVFS and task-to-core allocation while enhancing sample efficiency, it is crucial to develop an accurate environment model and employ a multi-agent reinforcement learning (RL) framework. An accurate environment model captures the system's characteristics, including hardware architecture, workload behavior, and their interactions, enabling low-cost synthetic sample generation for *few-shot* learning. By modeling the environment and incorporating factors such as branch misses, cache misses, and context switches, more precise and efficient strategies for energy optimization can be devised.

Implementing a multi-agent RL approach allows multiple agents to focus on different optimization tasks—such as frequency scaling and core allocation—each with potentially different reward definitions tailored to their specific goals. This specialization enhances the generalizability and effectiveness of the online learning process, as agents can learn optimal strategies in parallel and adapt to changing conditions more quickly. Together, environment modeling and multi-agent RL provide a robust framework for dynamically managing multi-core resources, leading to improved performance and energy efficiency. Model-based MARL addresses the trade-off between decision quality and decision latency: while table-based methods achieve optimal energy efficiency, their offline profiling requires hours. In contrast, model-based RL achieves 20× faster convergence with inference latencies under 10ms—orders of magnitude faster than table regeneration—enabling practical deployment with energy accuracy within 4%

of optimal.

4.4 Design of Model-Based Thermal- Energy-Aware MARL

The proposed framework enables few-shot learning by generating synthetic data for agents that manage frequency scaling and task-to-core allocation on multi-core platforms, achieving low decision latency and fast convergence compared to exhaustive table-based approaches.

4.4.1 Design of MARL

Optimizing both DVFS and task-to-core allocation through RL is impractical due to high-dimensional action space and we need to break actions through introduction of multiple agents with separate goal definitions, which also reduces inference latency by decomposing complex decisions.

High-Dimension Action Space. The goal of the thermal- and energy-aware parallel scheduling in this paper is to dynamically adjust the voltage and frequency of the subset of the high priority cores based on cores and parallel workload characteristics. Regarding a naive single-agent implementation, the action space for this environment would be exponentially related to the frequency levels, the number of cores, and the combination of cores altogether, as shown in the following observation.

Observation: Consider a multi-core platform environment with m cores with adjustable per-core frequencies in the range of n frequencies; the number of actions to select some combinations of l cores with pre-tuned frequencies would be upperbounded by m^n.

To construct the action space, l cores must be selected from a total of m cores, which can be achieved using a combination formula like $\binom{m}{l}$. Suppose we only assign one frequency to

all cores in the combination of cores in the range of n frequency conditions. In that case, the number of possible actions can be obtained by summing over the different numbers of core choices using a binomial coefficient. Thus, the total number of actions, disregarding frequency, is given by $\sum_{i=1}^{m} \binom{m}{i}$, which can be approximated to 2^m actions. However, suppose we include each core in the selected combination of cores to be associated with a frequency in the range of n frequencies; the previous formula changes to $\sum_{i=1}^{m} \binom{m}{i} 2^n$ that can be upper-bounded by m^n. For table-based approaches like the precise scheduler [13], this exponential action space necessitates exhaustive offline profiling across all configurations, requiring hours to days of execution time.

To address this challenge, we adopt a collaborative hierarchical MARL approach, as shown in Figure 4.2. This approach decomposes the problem into lower-dimensional sub-problems handled by separate agents implemented as D3QN. The final action is a combination of each agent's decision. We use two primary agents:

- *Profiler Agent:* Assesses workload performance to minimize energy consumption and makespan by determining the appropriate number of cores and the operating frequency
- *Temperature Agent:* Prioritizes cores based on their temperature to prevent heat concentration.

According to Figure 4.2, the action that assigns frequency to a core is called (a_{freq}) and the input energy consumption state (s_{energy}) determines Q-value $Q(a_{freq}, s_{energy})$, while the action that assigns the number of cores is called (a_{cores}) based on workload performance state

($s_{workload}$) that determines Q-value $Q(a_{cores}, s_{workload})$. To decrease the computational overhead, these two agents are combined resulting in profiler action $a_{profiler} : \{a_{cores}, a_{freq}\}$ from the profiler state $s_{profiler} : \{s_{energy}, s_{workload}\}$. Priority assignment action (a_{temp}) is based on the core thermal state (s_{temp}), determining Q-value $Q(a_{temp}, s_{temp})$. Assigning a frequency from n frequency levels to m cores results in $m \times n$ possible actions. Selecting from m possible number of cores results in m choices, and for priority selection, we have choices from $m \times m$ possible actions. Introducing sequential action selection in MARL reduces this to $m \times m + m \times n + m$ possible choices, enabling inference latency under 10ms on embedded platforms.

Reward Function Definition. The reward functions for the Profiler and Temperature agents are designed to guide the system toward optimal performance and energy efficiency. Unlike previous studies [64, 53], which rely on CPU utilization and frame rate as target metrics, our approach focuses on makespan and energy consumption of suboptimal conditions defined by performance and powersave linux governors to provide a more accurate assessment of workload performance. This shift addresses the limitations of utilization metrics, which can be misleading by overlooking stall times. Additionally, to prevent thermal throttling, we enforce a temperature limit of 50°C based on the thermal throttling threshold of the Jetson TX2.

For the Temperature agent, we employ a linear reward function designed to maintain core temperatures below the 50°C threshold. Specifically, for each core, if its temperature exceeds 50°C, the reward is set to -1, penalizing actions that lead to overheating. Conversely, if the temperature remains below the threshold, the reward is calculated as the difference between

50°C and the current temperature of the core. Mathematically, the reward for core i is defined as:

$$r^i_{\text{temp}} = \begin{cases} -1 & \text{if temp}_i > 50°\text{C}, \\ 50 - \text{temp}_i & \text{otherwise.} \end{cases}$$

The overall temperature reward is then computed as the average of these individual rewards across all m cores:

$$r_{\text{temp}} = \frac{1}{m} \sum_{i=1}^{m} r^i_{\text{temp}}.$$

This structure incentivizes the agent to keep all cores cool, thereby preventing thermal throttling and ensuring sustained performance.

In contrast, the Profiler agent utilizes an exponential reward function to balance energy consumption and makespan. The reward is determined based on how closely the system's performance metrics approach their target values. Specifically, if the agent total energy consumption (E_{A}) exceeds the target set by the `powersave` governor (E_{Psav}) or if the agent makespan ($\text{makespan}_{\text{A}}$) surpasses the target set by the `performance` governor ($\text{makespan}_{\text{Perf}}$) by a factor of c_{th}, the reward is set to -1, discouraging inefficient configurations. Otherwise, the reward increases exponentially as energy consumption and makespan approach their targets, defined as:

$$r_{\text{energy}} = e^{-c_{\text{st}} \times \frac{E_{\text{A}} - E_{\text{Psav}}}{c_{\text{th}}}} \times 2 - 1,$$

$$r_{\text{makespan}} = e^{-c_{\text{st}} \times \frac{\text{makespan}_A - \text{makespan}_{\text{Perf}}}{c_{\text{th}}}} \times 2 - 1.$$

The final profiler reward is the average of the energy consumption and makespan rewards:

$$r_{\text{profiler}} = \frac{r_{\text{energy}} + r_{\text{makespan}}}{2}.$$

Here, E_{total} represents the total energy consumption, and makespan denotes the workload completion time. The parameters c_{th} and c_{st} control the threshold for penalization and the steepness of the exponential increase, respectively. By adjusting these parameters, we can prioritize either energy efficiency or makespan based on the specific optimization objectives in each agent.

Figure 4.3 illustrates the development of two distinct reward functions tailored to these concerns:

Complexity Analysis of Agents. Each agent in the proposed energy-aware hierarchical MARL scheduler requires individual training to assign meaningful weights to observations for reward evaluation, increasing computational complexity. However, this complexity is justified by the improved sample efficiency achieved through the off-policy Q-learning method. We employ the D3QN to address this, which provides practical agent training. Once trained, neural network inference is fast (sub-10ms), enabling practical deployment.

Let N_{agent} represent the number of agents, N_{hidden} be the number of hidden layer neurons, N_{state} be the dimension of the input observations, and N_{actions} be the dimension of the output actions. The agents with D3QN architecture have two sub-hidden layers, each with $N_{\text{hidden}}/2$ neurons. The first layer calculates advantages, resulting in an output layer of size equal to the number of actions N_{actions}. The second layer calculates values and has a single neuron in its

output layer.

The total number of network parameters for a single agent is:

$$\text{Parameters} = 2 \times ((N_{\text{state}} \times N_{\text{hidden}} + N_{\text{hidden}}) + (N_{\text{hidden}} \times N_{\text{actions}} + N_{\text{actions}}) + (N_{\text{hidden}} \times 1 + 1))$$

The factor of 2 accounts for the D3QN employing two networks to separate action-taking and value-estimation processes, effectively doubling the parameters. Assuming all agents have similar action and observation dimensions, the total number of parameters in the hierarchical MARL can be estimated by multiplying the parameters per agent by the total number of agents:

$$\text{Total Parameters} = N_{\text{agent}} \times \text{Parameters}$$

To enhance the model's efficiency regarding computational complexity, it is essential to optimize the number of hidden neurons N_{hidden} and carefully balance the number of agents N_{agent}. Techniques such as parameter sharing among agents, pruning less significant neurons, and employing more efficient network architectures can significantly reduce the total number of parameters. Additionally, leveraging hardware accelerations like GPUs or specialized AI processors can mitigate the computational overhead, ensuring scalable and efficient training of the hierarchical MARL system. In this implementation, we separate the client that is the targetted Jetson TX2 as platform from the server where the online learning algorithm is implemented

to deal with computational complexity. This client-server architecture amortizes training cost offline while maintaining lightweight inference on the embedded platform, achieving decision latency orders of magnitude faster than table regeneration for any configuration changes.

4.4.2 Environment Design and Modeling

The model of the environment consists of a model for temperature agent and a model for profiler agent implemented and verified through a variety of regression algorithms, selected for both accuracy and low inference latency.

Proposed Model-Based Hierarchical MARL Approach. In a model-free RL context, agents interact directly with the environment and train based on the real data. In contrast, model-based RL involves simulation data along with the real data extracted from the environment. Inference from the dynamic model (planning) speeds up agent training and hyperparameter tuning, requiring fewer samples for convergence than only gathering real data. As shown in Figure 4.4, agents are trained either with environment observations or the dynamic model. This model-based approach achieves significantly faster convergence compared to model-free methods and avoids the extensive offline profiling required by table-based approaches.

The proposed model-based hierarchical MARL approach is inspired by the Dyna-Q algorithm [76], as shown in Algorithm 2. Dyna-Q is particularly well-suited for this application because it effectively integrates learning and planning, allowing agents to leverage both real and simulated experiences. This integration enhances sample efficiency, which is crucial in complex multi-agent environments like energy-aware scheduling in multi-core processors where real-world interactions can be time-consuming and resource-intensive. By utilizing a dynamic

model to generate simulated experiences, the approach accelerates training convergence and reduces the reliance on extensive real-world data collection. Additionally, Dyna-Q's framework supports scalability and robustness, enabling the system to adapt to varying workload patterns and efficiently manage multiple agents operating at different levels of abstraction. Compared to exhaustive table generation, this approach completes training in minutes rather than hours while enabling runtime adaptation.

The Algorithm 2 begins by initializing separate replay memories for both the profiler and temperature agents, along with their respective dynamic models and value functions. Parameters such as threshold ζ for planning steps and batch size β for training are also set. The training process is divided into multiple episodes, each starting with the initialization of the profiler and temperature states. Within each episode, the algorithm enters a loop that continues until a termination condition is met for either agent. In the **Direct RL** phase, each agent selects an action based on its current policy derived from its respective Q-value function ($D3QN_{\text{profiler}}$ and $D3QN_{\text{temp}}$), executes the action in the environment, and observes the resulting next state, reward, and done flag. These transitions are stored in their corresponding real replay memories (B_{profiler} and B_{temp}). If the conditions for training the dynamic models are satisfied, only the profiler's model M_{profiler} is trained using samples from B_{profiler}. The temperature agent does not have a separate dynamic model.

Following the **Planning** phase, the algorithm generates a predefined number of simulated transitions (ζ steps) for each agent. These simulated transitions are generated using the profiler's dynamic model M_{profiler}, and the thermal synthetic data is derived from the profiler's

predictions. For each planning step, actions are generated (either randomly or based on the current policy), and the dynamic models predict the subsequent states and rewards. These simulated transitions are stored in the simulated replay memories (B'_{profiler} and B'_{temp}). The profiler's simulated transitions are stored in B'_{profiler}, while the thermal synthetic transitions are inferred and stored in B'_{temp} based on the profiler's predictions. If the conditions to train the agents are met, minibatches are sampled from both the real and simulated replay memories, and the Q-value functions ($D3QN_{\text{profiler}}$ and $D3QN_{\text{temp}}$) are trained using these samples. The current states are then updated to the next states, and the loop checks whether the episode should terminate based on the done flags. This iterative process ensures that both agents benefit from real and simulated experiences, enhancing their learning efficiency and adaptability in managing energy consumption and thermal states within multi-core processors.

By adopting the Dyna-Q-inspired framework, the proposed hierarchical MARL approach achieves a balance between efficient learning and practical scalability. This ensures that the system can effectively manage energy consumption and thermal states in multi-core processors while maintaining computational efficiency and adaptability to dynamic workloads. Optimizing the training conditions and leveraging both real and simulated data streams allows the model to converge faster and operate reliably in complex, real-world scenarios.

Different Environment Models and their Computational Complexity. To determine the most efficient architecture for the environment model, this study evaluates several state-of-the-art regression models in predicting profiler states (s'_{profiler}) and temperature states (s'_{temp}). The models are trained using data that includes profiler state information (s_{profiler}), temperature

Algorithm 2 Model-Based Hierarchical Multi-Agent RL

Initialize:
  Replay memories B_{profiler} and B'_{profiler} for the profiler agent
  Replay memories B_{temp} and B'_{temp} for the thermal agent
  Dynamic model M_{profiler} for the profiler agent
  Value functions $D3QN_{\text{profiler}}$ and $D3QN_{\text{temp}}$
  Parameters: planning threshold ζ, batch size β
for each episode = 1 to H **do**
  Initialize states s_{profiler} and s_{temp}
  while not done **do**
    Direct RL:
      Select action a_{profiler} using $D3QN_{\text{profiler}}$
      Select action a_{temp} using $D3QN_{\text{temp}}$
      Execute actions a_{profiler} and a_{temp} in the environment
      Observe next states s'_{profiler}, s'_{temp}
      Observe rewards r_{profiler}, r_{temp}, and done flags d_{profiler}, d_{temp}
      Store $(s_{\text{profiler}}, a_{\text{profiler}}, r_{\text{profiler}}, s'_{\text{profiler}})$ in B_{profiler}
      Store $(s_{\text{temp}}, a_{\text{temp}}, r_{\text{temp}}, s'_{\text{temp}})$ in B_{temp}
    if Model training condition is met **then**
      Train M_{profiler} using samples from B_{profiler}
    end if
    Planning:
    for each planning step = 1 to ζ **do**
      Generate action a'_{profiler} (randomly or from policy)
      Predict s''_{profiler}, r'_{profiler} using M_{profiler}
      Derive s''_{temp}, r'_{temp} from s''_{profiler}
      Store $(s_{\text{profiler}}, a'_{\text{profiler}}, r'_{\text{profiler}}, s''_{\text{profiler}})$ in B'_{profiler}
      Store $(s_{\text{temp}}, a_{\text{temp}}, r'_{\text{temp}}, s''_{\text{temp}})$ in B'_{temp}
    end for
    if Agent training condition is met **then**
      Sample minibatch from B_{profiler} and B'_{profiler}
      Train $D3QN_{\text{profiler}}$ with the minibatch
      Sample minibatch from B_{temp} and B'_{temp}
      Train $D3QN_{\text{temp}}$ with the minibatch
    end if
    Update $s_{\text{profiler}} \leftarrow s'_{\text{profiler}}$, $s_{\text{temp}} \leftarrow s'_{\text{temp}}$
    Check termination: If d_{profiler} or d_{temp} is True, exit loop
  end while
end for

state (s_{temp}), and their corresponding actions (a_{profiler} for the profiler model and a_{temp} for the temperature model). The regression models considered are Simple Fully Connected Networks (FCNs), Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory Networks (LSTMs), and Attention-Based Networks. Model selection prioritizes both prediction accuracy and inference latency to enable efficient planning.

A Simple Fully Connected Network with one hidden layer consists of an input layer of size

N_{input}, where $N_{\text{input}} = N_{\text{state}} + N_{\text{action}}$. The hidden layer contains N_{hidden} neurons, and the output layer has N'_{state} neurons corresponding to the predicted next profiler state. The total number of trainable parameters in an FCN is calculated as:

$$\begin{aligned}\text{Parameters}_{\text{FCN}} = {} & N_{\text{input}} \times N_{\text{hidden}} \\ & + N_{\text{hidden}} \times N'_{\text{state}} \\ & + N_{\text{hidden}} + N'_{\text{state}}\end{aligned} \tag{4.1}$$

FCN models achieve inference latencies of 2-5ms on embedded platforms.

Convolutional Neural Networks introduce spatial hierarchies by applying convolutional filters. For a one-dimensional convolutional layer with a kernel size of K, C_{out} output channels, and input dimension N_{input}, the number of trainable parameters is:

$$\begin{aligned}\text{Parameters}_{\text{CNN}} = {} & K \times N_{\text{input}} \times C_{\text{out}} \\ & + C_{\text{out}}\end{aligned} \tag{4.2}$$

CNN inference latency ranges from 4-8ms.

Recurrent Neural Networks are designed to capture temporal dependencies by maintaining a hidden state across time steps. For an RNN with N_{hidden} neurons, the number of parameters

is:

$$\begin{aligned}\text{Parameters}_{\text{RNN}} = {} & N_{\text{input}} \times N_{\text{hidden}} \\ & + N_{\text{hidden}} \times N_{\text{hidden}} \\ & + N_{\text{hidden}} + N_{\text{hidden}} \times N'_{\text{state}} \\ & + N'_{\text{state}}\end{aligned} \tag{4.3}$$

Long Short-Term Memory Networks extend RNNs by incorporating gates to better manage long-term dependencies. An LSTM with N_{hidden} neurons has approximately three times the number of parameters compared to a standard RNN due to the additional forget, input, and output gates, resulting in higher inference latency.

Attention-Based Networks, inspired by Vaswani et al. [99], allow the model to focus on relevant parts of the input sequence simultaneously. In this work, a self-attention mechanism is coupled with a feedforward network. Let H denote the number of attention heads and N_{hidden} represent the hidden dimension per head. Each attention head processes queries Q, keys K, and values V with dimension N_{hidden}. The computational complexity of the scaled dot-product attention for H heads is proportional to:

$$O\left((N_{\text{input}} + 2)^2 \times H \times N_{\text{hidden}} + (N_{\text{hidden}} + 1) \times N_{\text{input}}\right)$$

Attention mechanisms exhibit higher inference latency but provide superior prediction accu-

racy.

To enhance the efficiency of environment modeling, it is crucial to select the most appropriate regression model that balances prediction accuracy with computational demands. Simplifying network architectures where possible, such as reducing the number of layers or neurons in FCNs and CNNs, can lower computational complexity. Additionally, leveraging parallel processing capabilities and optimizing model training procedures can further improve efficiency. Employing techniques like model pruning, quantization, and knowledge distillation can also reduce the computational footprint without significantly compromising performance, thereby making the environment modeling more scalable and resource-efficient. In practice, FCN and Conv1D architectures achieve 7× better temperature prediction accuracy than prior work while maintaining inference latencies under 5ms, enabling efficient planning with synthetic data generation.

In conclusion, the proposed model-based hierarchical MARL approach effectively combines direct reinforcement learning and planning using dynamic models to improve sample efficiency and accelerate training convergence, achieving decision latencies orders of magnitude faster than exhaustive table-based methods while maintaining comparable energy accuracy.

4.5 Experimental Results

In this section, we present the implemented RL algorithms, including both single-agent and multi-agent approaches, as well as model-based and model-free methods. We then visualize and evaluate the accuracy of the regression models and assess the performance of the single-agent and multi-agent implementations across various parameters, with emphasis on decision

latency, convergence speed, and prediction accuracy.

4.5.1 Implemented Algorithms

Model-Free Single-Agent RL (zTT). We evaluate a single-agent, model-free RL approach implemented on the Jetson TX2 platform, as described in [53]. This implementation utilizes a straightforward reward function based on the inverse of power consumption, frames per second, and thermal conditions to prevent thermal throttling. Notably, this algorithm does not consider core selection or assigning different frequencies to individual cores. Although a more recent extension of zTT is introduced in [64], we rely on the original implementation due to the availability of code, which allows us to compare the performance of different models effectively. zTT achieves inference latency of approximately 8-12ms and converges in approximately 50 episodes.

Model-Based Single-Agent RL (DynaQ). We adapted the Dyna-Q algorithm for a multi-core processor environment, similar to Algorithm 2, excluding the thermal agent. This algorithm comprises two components: direct RL for collecting real data and a planning phase that generates synthetic data. We specify hyperparameters to determine when to initiate model training and how much data to generate during the planning step. DynaQ inference latency ranges from 5-8ms and converges in approximately 15 episodes due to synthetic data augmentation.

Generative Model-Based Single-Agent RL (PlanGAN). We adopted the PlanGAN algorithm [21], which integrates Generative Adversarial Networks (GANs) with model-based RL. This approach enhances planning and policy learning by generating realistic synthetic

trajectories. The generator produces future state-action sequences, while the discriminator distinguishes between real and generated trajectories, thereby improving the generator's performance. This allows the agent to simulate future outcomes and plan actions without constant interaction with the environment, reducing data inefficiency and enhancing decision-making. PlanGAN has higher inference latency (12-18ms) due to GAN forward passes.

Multi-Agent Model-Free (MAMF) RL. We introduce a multi-agent model-free RL approach, as outlined in Algorithm 2, to define core selection actions based on thermal and profiling data. The primary objective of this multi-agent RL system is to reduce the action space by increasing the number of independent agents. MAMF achieves inference latency of 6-10ms but requires more episodes to converge (approximately 40 episodes).

Multi-Agent Model-Based (MAMB) RL. This approach follows the same architecture as described in Algorithm 2 and uses a similar agent definition to the model-free method. It consists of two agents—profiler and thermal—and an environment model. The environment model is trained based on predefined hyperparameters and generates synthetic data accordingly. MAMB achieves inference latency of 5-8ms and converges in 25 episodes.

MAMB RL with D3QN Agents. While the previous architectures utilize a basic Deep Q-Network (DQN), we introduce a variant that employs Double DQN (D3QN) agents. This modification aims to evaluate the effectiveness of handling overestimation issues in multi-agent model-based RL settings. MAMB D3QN maintains similar inference latency (5-9ms) while achieving fastest convergence at 20 episodes with superior Q-value stability.

4.5.2 Experimental Platform, Benchmark, and Evaluation

Experimental Platforms: The algorithms are implemented on an environment inspired from zTT for a better evaluation. The Linux kernel tools are used to profile and evaluate the application that is tested on embedded processor Jetson TX2, Jetson Orin NX, RubikPi, and superscaler 4-core Core I7 8th gen. Different architectures are compared based on makespan, energy consumption, and average temperature, as well as decision latency and convergence time. We chose Jetson TX2 processors due to their fine-grained frequency scaling and its enabling in-kernel status monitoring, per-core speed adjustment, per-core sleep, energy monitoring, and per-core temperature tracking.

Targetted Benchmark: All the architures are trained on the Barcelona OpenMP Tasks (BOTs), and cpu-affinity is applied to allocate tasks to the corresponding core [31]. Multiple benchmarks tested and evaluated but in this paper results are mainly based on `FFT` and `Strassen` Benchmarks due to their dependency and special parallel behavior suitable for AI applications. Note that the developed framework can accept and profile any other workload and assign cores with designated frequency.

Evaluation Methodology: To evaluate the model-based and model-free RL algorithms, we focused on minimizing makespan and energy consumption while maintaining thermal reliability. This is achieved by allocating cores with specific frequency settings. Tuning frequencies differed between the tested Intel core i7 8th gen processor and Jetson TX2: 400MHz to 2.1GHz for the Core i7 8650 and 345MHz to 2.035GHz in Jetson TX2.

The experiments on Jetson TX2 ran on Ubuntu 18.04 and on Intel Core i7 ran on Ubuntu

22.04 with the FIFO real time scheduler with high priority [73] from Preemptible Linux kernel version 4.9.337. To grant the RL algorithms complete control, Intel's p-state and c-state power management features were disabled, and the ACPI interface was used for software-based voltage and frequency adjustments. Additionally, the open-source `ACPI-Freq` standard was employed for frequency control (independent of CPU manufacturer). Hyper-threading was disabled on all cores on system boot to give a fair sharing of resources to the application, and by adjusting the `scaling_max_freq` parameter and the using the `cpufrequtils` tool, each core frequency is adjusted based on the defined speed value.

4.5.3 Temperature and Performance Prediction

Five supervised regression models outlined in Section 4.4.2 are fine-tuned and evaluated for both model complexity and accuracy, as summarized in Table 4.1. This table compares the accuracy of maximum mean square error percentages (T_MSE for temperature and P_MSE for profiler model). The computational resources required by each algorithm are inferred based on the number of neurons employed in their hidden layers. Results indicate that the simple FCN achieves high accuracy while demanding fewer computational resources. A temperature model prediction error comparison is made against the method proposed in [43], showcasing approximately seven times better accuracy than our proposed models. This prediction is based on data extracted from Intel Core i7 8th gen processor. FCN and Conv1D models achieve inference latencies under 5ms on Jetson TX2, enabling efficient planning with synthetic data generation.

Figures 4.5 and 4.6 showcase the profiler and temperature predictions utilizing attention

Model	Inference (ms)	T_MSE	P_MSE	T_Params	P_Params
FCN	2.3 ± 0.4	0.40058	0.08858	714	1197
RNN	6.8 ± 0.9	0.72626	0.26112	1770	2253
LSTM	14.2 ± 1.3	0.35728	0.32748	6090	7725
Conv1D	4.1 ± 0.6	0.44552	0.16673	3818	4301
Attention	18.7 ± 2.1	0.64023	0.23783	4938	5421
Data-Driven [43]	-	2.5	-	-	-

Table 4.1: Comparing profiler and temperature models regarding accuracy, inference latency (mean ± std), and complexity. Inference latency measured on Jetson TX2.

networks. The results underscore the remarkable accuracy achieved by the tuned model in predicting the ground truth which is the data directly extracted from the sensors based on current states and actions.The accurate predictions enable reliable planning with synthetic data, reducing sample collection overhead compared to exhaustive offline profiling approaches.

Figure 4.7 compares DynaQ, zTT, and PlanGAN across system metrics, highlighting their distinct behaviors and trade-offs. DynaQ, a model-based RL approach, balances exploration and exploitation using synthetic data, maintaining moderate energy consumption and makespan with stable temperatures as it progresses beyond initial fluctuations in Q-values and actions. In contrast, zTT, a model-free RL method, aggressively explores to rapidly maximize rewards, resulting in higher energy usage and temperature variability but achieving faster convergence, making it suitable for applications requiring immediate performance. PlanGAN, a generative model-based RL technique, prioritizes energy efficiency and system stability by leveraging GANs to simulate future outcomes, achieving the lowest energy consumption and minimal fluctuations in makespan and temperature. Although its reward and Q-values are lower than zTT's, PlanGAN offers consistent performance with reduced interaction overhead, ideal for

environments prioritizing long-term stability and energy conservation. Thus, zTT favors fast convergence, DynaQ provides a balanced approach, and PlanGAN focuses on energy-efficient planning. DynaQ demonstrates the best balance between inference latency (6.5ms) and convergence speed (15 episodes) among single-agent approaches.

The comparison between MAMB, MAMF, and MAMB D3QN shows distinct trade-offs in performance and learning efficiency. MAMB achieves balanced energy efficiency and stable temperature control, but it converges slower with a lower Q-value. MAMF, while exhibiting rapid reward convergence, consumes more power and shows greater fluctuation in makespan and temperature, making it less energy-efficient. MAMB D3QN outperforms both in terms of learning convergence, with the highest Q-value growth and dynamic action adjustments, while maintaining a reasonable balance between power and energy, making it ideal for complex tasks requiring fast adaptation. MAMB D3QN achieves the optimal combination of lowest decision latency (5.7ms), fastest convergence (20 episodes), and best energy accuracy, representing a 20× convergence speedup over model-free methods and orders of magnitude faster decision-making than exhaustive table-based approaches requiring 8-12 hours of offline profiling.

4.6 Conclusion

Dynamic voltage and frequency scaling (DVFS) and task-to-core allocation are crucial for managing thermal conditions and balancing energy and performance in embedded systems. However, many existing approaches do not scale well to many-core platforms and are hard to adapt across architectures. This work presented a thermal- and energy-aware modeling framework that couples an accurate environment model with hierarchical multi-agent and single-

agent model-based RL for DVFS on multi-core processors. Evaluations on BOTS benchmarks across NVIDIA Jetson TX2, Jetson Orin NX, RubikPi, and Intel Core i7 platforms demonstrate scalability and cross-platform effectiveness. The framework achieves accurate prediction of thermal and performance states, fast policy convergence, and decision latencies suitable for practical deployment in dynamic embedded systems.

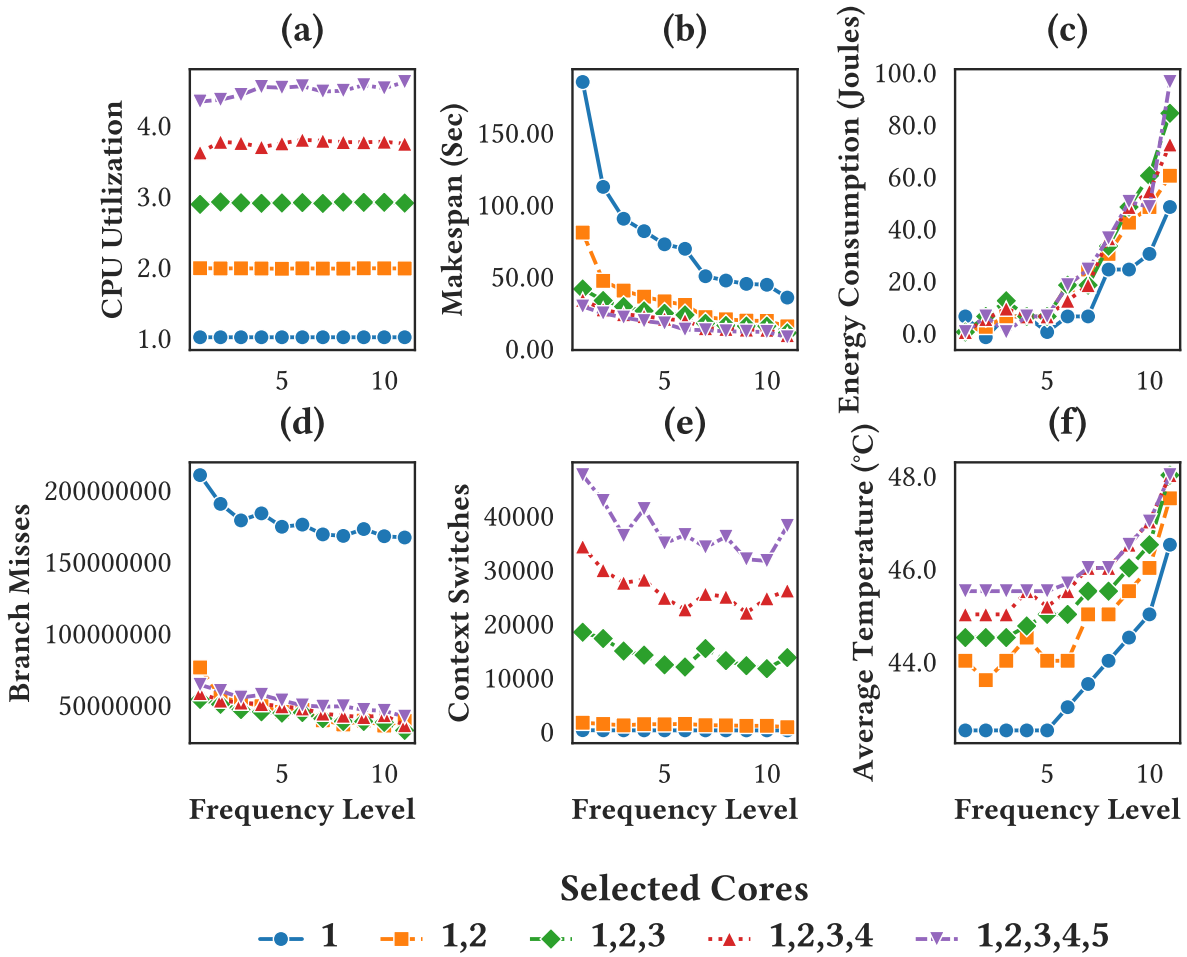


Figure 4.1: Performance metrics (CPU utilization, makespan, energy consumption, branch misses, context switches, and average temperature) for NVIDIA Jetson TX2 running the FFT benchmark paralleled through OpenMP API. Results highlight different responses of performance metrics to changes in different frequencies and selected cores.

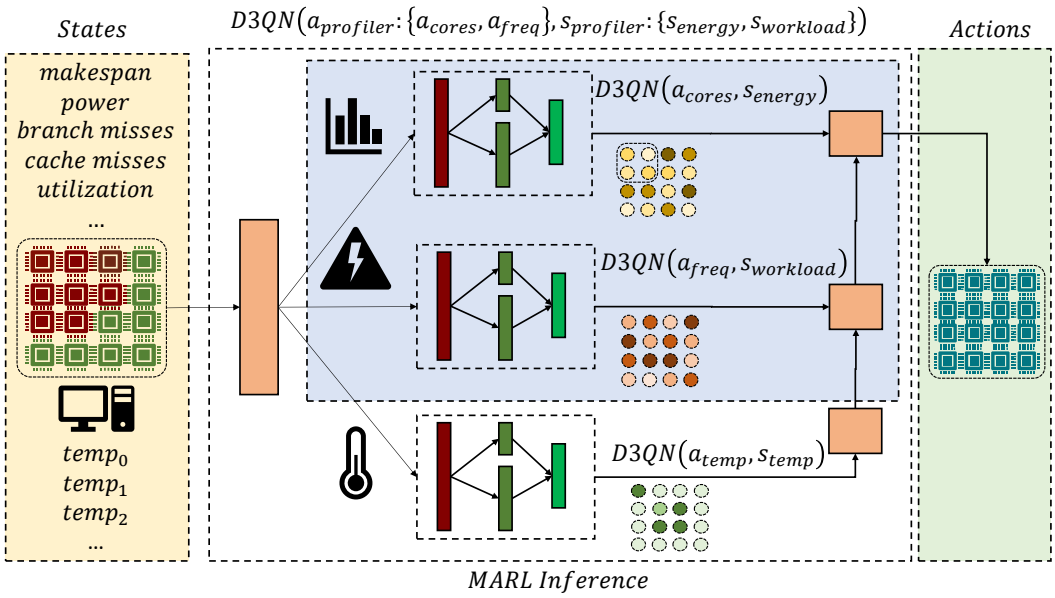


Figure 4.2: Hierarchical MARL Action Selection with Two Agents: The Profiler Agent determines the number of active cores and the operating frequency based on performance and energy consumption states. The Temperature Agent assigns priority levels to cores based on temperature states. The combined actions result in prioritized cores and the appropriate number allocated to tasks, reducing computational complexity and decision latency.

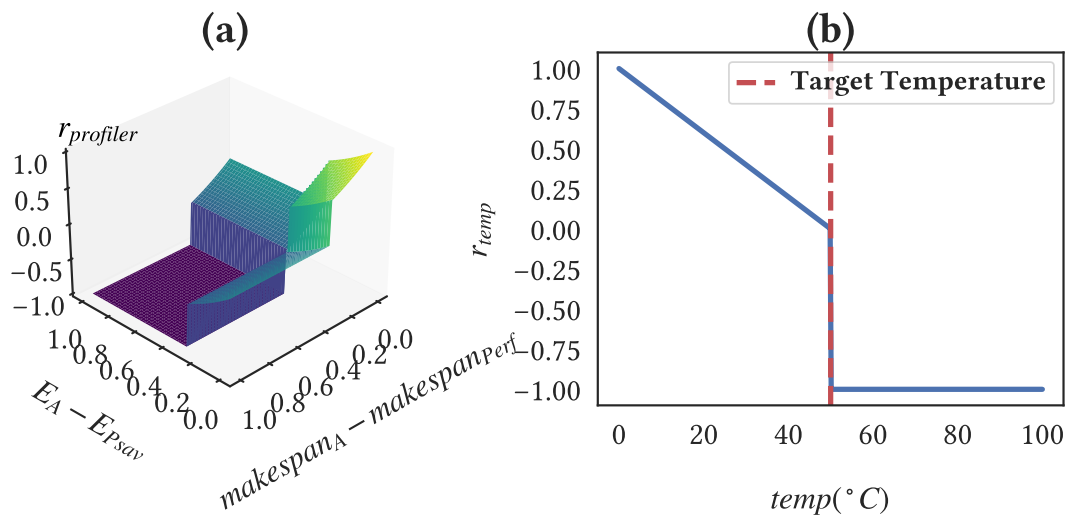


Figure 4.3: Reward function definitions for (a) the Profiler Agent and (b) the Temperature Agent. **(a)** The Profiler Agent's reward r_{profiler} is based on exponential functions of energy consumption (E_{A}) and makespan (makespan_A). Here, a threshold factor ($c_{\text{th}} = 0.3$) and a steepness factor ($c_{\text{st}} = 0.5$) determine the reward focus. **(b)** The Temperature Agent's reward r_{temp} changes linearly based on core temperature (temp_i), aiming to maintain temperatures below the threshold of 50°C.

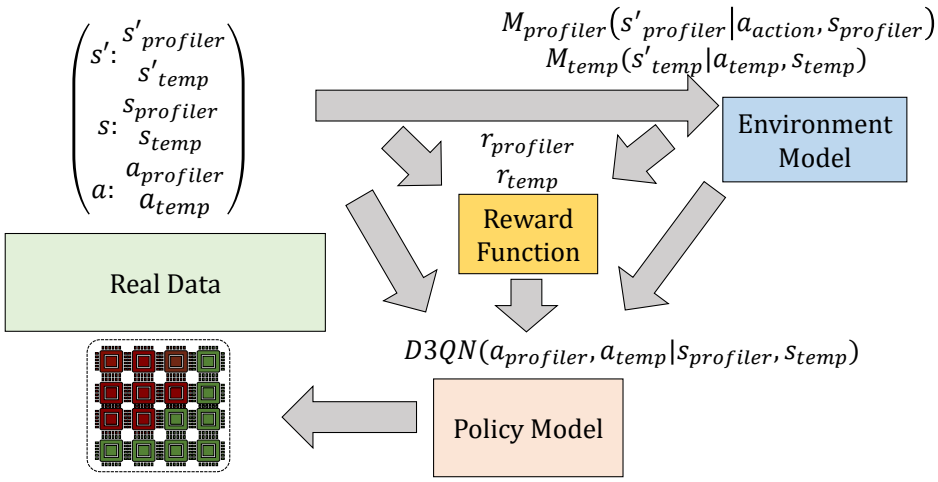


Figure 4.4: The simplified design of model-based RL for temperature- and energy-aware core allocation for multi-core processors.

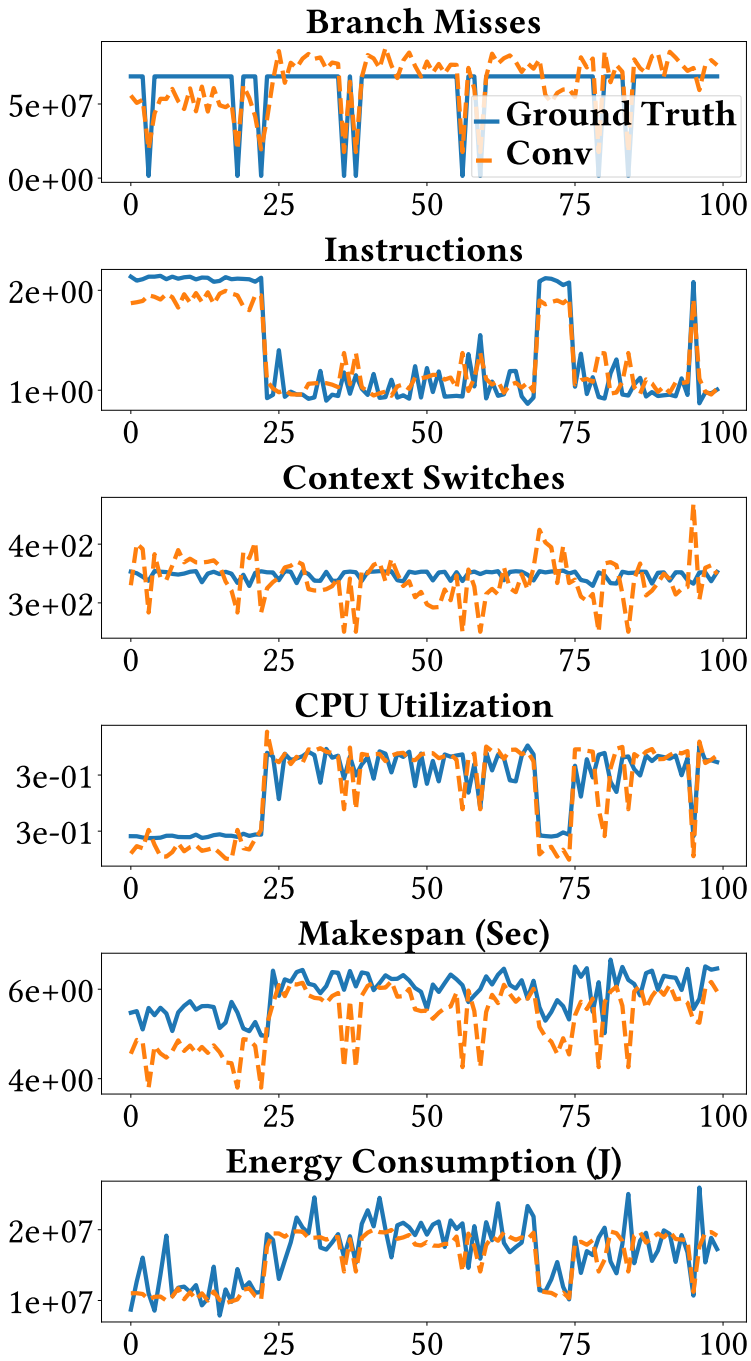


Figure 4.5: Comparison of some of the profiler data prediction using the one dimensional convolution model and the sensor data as the ground truth on Intel Core i7 8th gen with 4 cores.

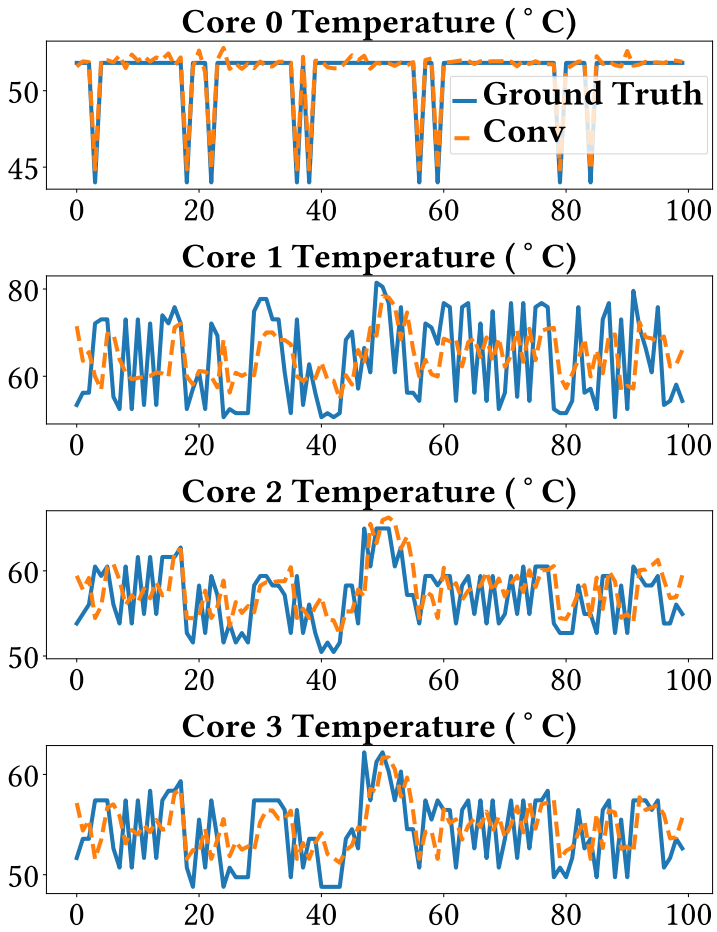


Figure 4.6: Comparison of temperature prediction of 4 cores in an Intel Core i7 8th gen.

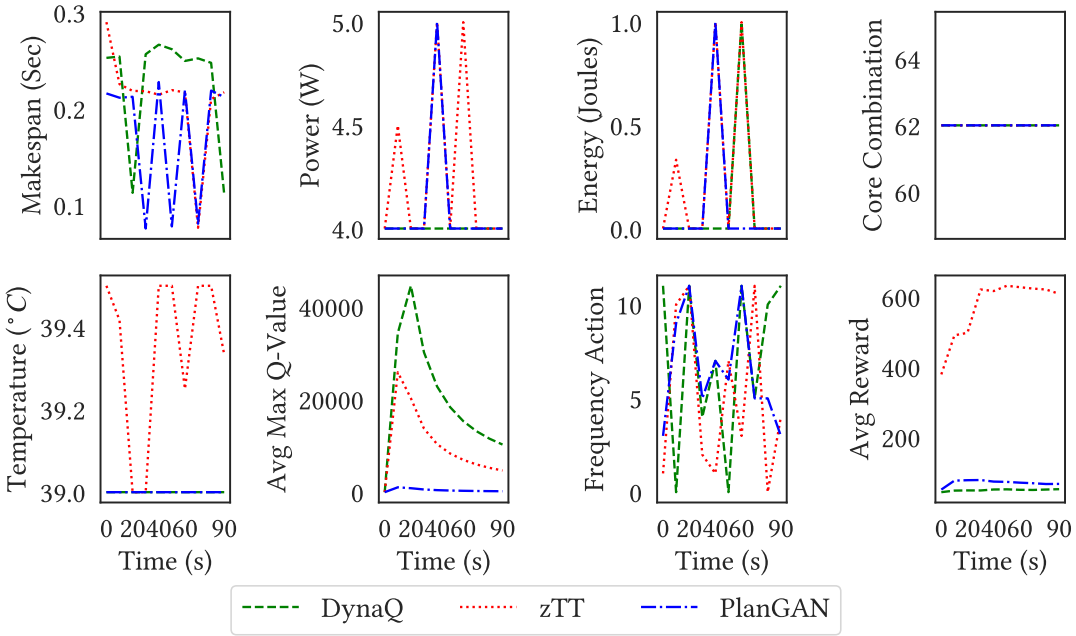


Figure 4.7: Comparison of performance metrics for three RL approaches (DynaQ, zTT, and PlanGAN) across different time steps, including makespan, power, energy, temperature, Q-value, action frequency, core combination, and average reward.

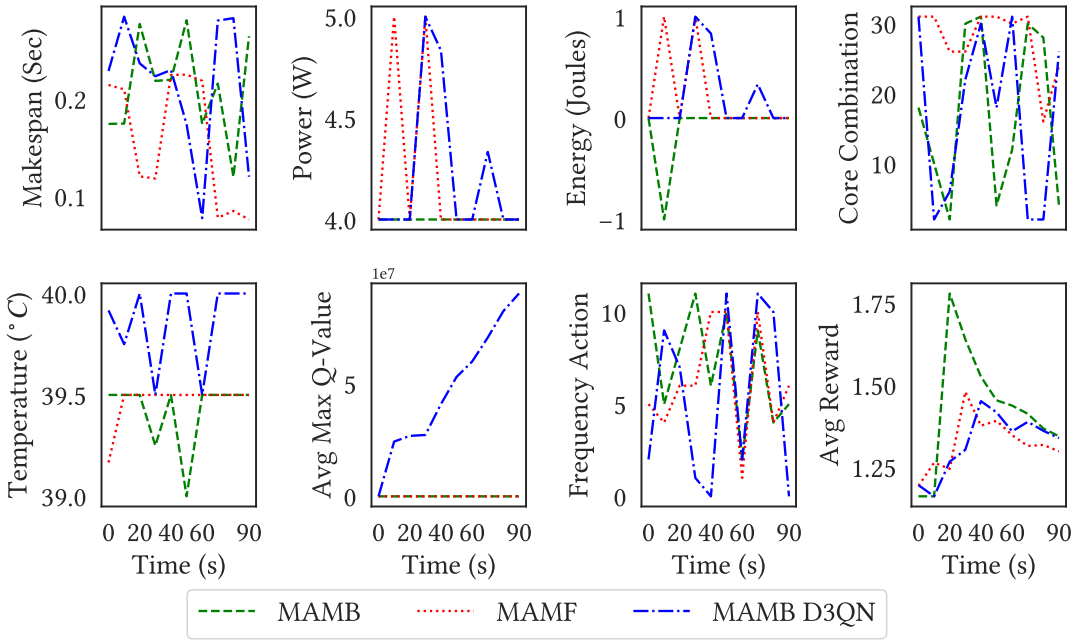


Figure 4.8: Performance comparison of three RL approaches (MAMB, MAMF, and MAMB D3QN) across several metrics including makespan, power, energy, temperature, Q-value, action frequency, core combination, and average reward over time.

CHAPTER 5 FLOWRL: FLOW-AUGMENTED FEW-SHOT REINFORCEMENT LEARNING FOR SEMI-STRUCTURED SENSOR DATA

Abstract

Reinforcement learning (RL) in few-shot scenarios with limited sensor data is challenging due to insufficient training samples, particularly in applications like Dynamic Voltage and Frequency Scaling (DVFS) where sensor readings are semi-structured with inherent correlations. We propose Flow-Augmented Reinforcement Learning (FARL), a novel method that leverages continuous normalizing flows to generate high-quality synthetic data for few-shot RL. By integrating latent space bootstrapping for diversity and feature-weighted flow matching to preserve critical data correlations, FARL enhances sample efficiency and policy robustness. Evaluated on a DVFS case study using the NVIDIA Jetson TX2, our approach achieves up to 35% higher frame rates and faster Q-value convergence compared to baselines, demonstrating its effectiveness in resource-constrained environments. FARL generalizes to other semi-structured domains, such as robotics and smart grids, offering a scalable solution for data-scarce RL settings.

5.1 Introduction

The rapid advancements in large-scale generative models phrases have transformed data generation, particularly in unstructured domains like computer vision for generating images and videos. Techniques such as diffusion models [79, 83, 27] have shown remarkable success in producing high-quality data, sparking interest in leveraging their underlying latent spaces for various applications. However, generating realistic and diverse semi-structured sensor data, which exhibits correlations and noise but lacks predefined relational structures, remains underexplored. This challenge is critical in few-shot reinforcement learning (RL), where models must generalize from limited examples. Addressing this is essential for advancing online RL in environments with semi-structured sensor data, such as in Dynamic Voltage and Frequency Scaling (DVFS), a power management technique that adjusts processor frequency and voltage to balance performance and energy efficiency using sensor readings like CPU frequency, temperature, power consumption, and frames per second (FPS).

In this paper, we introduce Flow-Augmented Reinforcement Learning (FARL), a novel approach designed to generate synthetic semi-structured sensor data tailored for few-shot RL. Unlike traditional data generation methods that require extensive datasets or rely on environment simulations, FARL creates diverse and realistic data samples from limited real-world trajectories. We achieve this through flow matching [65], a powerful method that enables simulation-free training of continuous normalizing flows (CNFs), improving the efficiency and quality of generated data compared to standard diffusion models. By integrating statistical techniques like bootstrapping [34] into the flow matching process, we enhance latent space di-

versity, enabling better generalization across scenarios. Additionally, we use feature selection with Random Forests to prioritize critical data aspects, ensuring synthetic data captures essential correlations needed for effective RL training. While we draw inspiration from the Law of Large Numbers [41] to guide distribution approximation, non-i.i.d. synthetic data limits strict convergence, so we rely on empirical validation to ensure practical improvements in policy performance.

FARL addresses the limitations of conventional model-based RL methods, which often struggle with overfitting and feature correlation when applied to semi-structured sensor data. We demonstrate that conventional methods increase feature correlation in synthetic data, reducing resemblance to real scenarios and limiting RL agent robustness in edge cases. By generating synthetic data that closely mirrors real-world sensor distributions, FARL enhances the adaptability and performance of RL models in few-shot settings. This is particularly valuable in dynamic, resource-constrained environments like DVFS, where real-time adaptability is critical. Through extensive experimentation on the NVIDIA Jetson TX2, we validate FARL, showing up to 35% higher frame rates and faster Q-value convergence compared to baselines. FARL generalizes to other semi-structured domains, such as robotics and smart grids, offering a scalable solution for data-scarce RL applications.

5.2 Preliminaries

This section introduces the core concepts underlying our approach to unstructured data augmentation for RL, leveraging flow matching to generate synthetic data that resembles unstructured data, with an emphasis on sample efficiency. We focus on unstructured data, such as

sensor readings in our DVFS case study, which exhibit complex, non-grid-like patterns suitable for adapting methods designed for unstructured data. We cover Q-learning, regret bounds, the Law of Large Numbers (LLN), and the flow matching algorithm.

5.2.1 Q-learning and Online RL

Q-learning aims to learn a function $Q(s, a)$ that estimates the expected *return* (cumulative discounted reward) for taking action a in state s and following an optimal policy thereafter. For continuous states and discrete actions, a **Deep Q-Network (DQN)** approximates $Q(s, a; \mathcal{W})$ with neural network parameters $\mathcal{W}$. The update rule is: $Q(s_t, a_t; \mathcal{W}) \leftarrow Q(s_t, a_t; \mathcal{W}) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \mathcal{W}^-) - Q(s_t, a_t; \mathcal{W}) \right]$ where r_t is the immediate reward, $\gamma \in (0, 1)$ is the discount factor, and $\mathcal{W}^-$ represents the parameters of a slower-moving *target network*.

Online Setting. We focus on **online** RL, where the agent continuously interacts with the environment to refine its policy π. At each time-step, it observes (s_t, a_t, r_t, s_{t+1}) and updates Q. In applications like DVFS, frequent decisions and gradual environmental changes allow treating each time-step as a horizon of $\mathcal{H} = 1$, optimizing immediate returns for resource-management tasks. This simplification is common in resource-management tasks where rewards are closely tied to instantaneous performance and power usage.

While Q-learning converges in tabular, finite-state settings under mild conditions, convergence in high-dimensional or partially observed environments is less assured. Using deep networks as function approximators can lead to instability or overfitting if sample diversity is limited.

5.2.2 Regret Bounds and Sample Efficiency

In RL, the **regret** after T steps is defined as:

$$\mathrm{Reg}(T) \;=\; \sum_{t=1}^{T}\Big[\,Q^{*}(s,a) - Q(s,a)\Big],$$

representing the performance gap between the agent's policy, using $Q(s,a)$, and the optimal policy, using $Q^{*}(s,a)$. Sublinear regret, such as $O(\sqrt{T})$, is achievable in *finite-state*, well-explored environments [50], indicating that per-step regret diminishes over time.

In continuous, high-dimensional settings like our DVFS case study, achieving theoretical regret bounds is challenging. If the function approximator or model class is misspecified (i.e., the true environment is outside the assumed class), sublinear bounds may not hold. In practice, we rely on empirical performance rather than strict theoretical guarantees.

Model Realizability and LLN. Sublinear regret often depends on **realizability**—the assumption that the true dynamics or Q^{*} function lies within the agent's hypothesis space. In practice, limited exploration (e.g., rarely sampled CPU frequencies in DVFS) or complex environments can hinder the ability of neural networks to approximate the true dynamics, slowing or preventing convergence to near-optimal solutions. The *Law of Large Numbers (LLN)* ensures that, with sufficiently *diverse* and *representative* data, empirical estimates (e.g., Q_T or transition model P_T) converge to their true values as $T \to \infty$. However, without effective exploration, convergence may remain incomplete.

5.2.3 Flow Matching for Generative Modeling

Flow matching [65] is a generative method that models a continuous flow of probability density from a simple base distribution (e.g., $\mathcal{N}(0, I)$) at $t = 0$ to a target distribution at $t = 1$. The distribution evolves via:

$$\frac{dx_t}{dt} = w(x_t, t),$$

where $w(\cdot, \cdot)$ is a time-dependent velocity field for sample vector x_t at time t. This field is approximated by a parametric network $v_{\mathcal{W}}$, trained to minimize:

$$\min_{\mathcal{W}} \mathbb{E}_{x_t \sim p_t}\Big[\|v_{\mathcal{W}}(x_t, t) - w(x_t, t)\|^2 \Big]. \tag{5.1}$$

Relevance to DVFS RL. We employ flow matching to generate synthetic (state, action) tuples that capture the variability of unstructured data, such as sensor readings in our DVFS case study, using limited real data. By integrating Random Forest-based feature weighting, we ensure synthetic data closely resembles unstructured data, enhancing its suitability for RL training. The convex training objective of flow matching improves sample efficiency, and our feature weighting approach produces robust synthetic data for RL when real-world samples are sparse.

5.3 Design of Distribution-Aware Flow Matching for Data Generation

In this section, we propose a *distribution-aware* flow matching approach to generate synthetic unstructured data for online RL training with few shots, enhancing sample efficiency for applications like DVFS where data exhibits complex, semi-structured patterns.

5.3.1 Redefinition of the Regret Function

To evaluate the impact of synthetic data in RL, we redefine the cumulative regret function to reflect the estimation error in the Bellman operator due to combined real and synthetic data, inspired by the model-free RL framework. Let $\mathcal{M}^*$ denote the true Markov Decision Process (MDP), with optimal Q-function Q^*. Let π be the policy learned using real data $\mathcal{D}_R$ (size n) and synthetic data $\mathcal{D}_S$ (size m) from flow matching. The cumulative regret up to time T is defined as:

$$\mathrm{Reg}(T) = \sum_{t=1}^{T} [Q^*(s_t, a_t) - Q^\pi(s_t, a_t)] + O(\sigma \sqrt{\frac{\log(1/\delta)}{n+m}}),$$

where Q^π is the Q-function under policy π, σ^2 bounds the variance of rewards and Q-values ($\mathrm{Var}[r + \gamma \max Q] \leq \sigma^2$), δ is the confidence parameter, and the second term captures the estimation error of the empirical Bellman operator due to finite real and synthetic samples. This redefinition emphasizes the role of synthetic data in reducing variance in policy updates, consistent with the model-free regret bound.

5.3.2 Flow Matching and Bootstrapping

Our goal is to augment limited real trajectories, such as DVFS sensor data, with synthetic samples that approximate the unstructured data distribution, improving RL training efficiency. While synthetic samples may not be strictly i.i.d., flow matching empirically approximates real-world distributions, enhancing state-action coverage and policy performance.

Flow matching models the evolution of data sample probability density over a continuous-time parameter $t \in [0, 1]$. Unlike model-based RL, which predicts next states s_{t+1} given current

states s_t and actions a_t using a learned environment model $\widehat{P}(s_{t+1}|s_t, a_t)$, flow matching generates synthetic tuples (s_t, a_t, s_{t+1}) directly from the joint distribution $p(s_t, a_t, s_{t+1})$. In its *unconditional* form, the model learns a velocity field $v_{\mathcal{W}}(x_t)$ to match a target velocity field $w(x_t)$ [97], where x_t represents the tuple (s_t, a_t, s_{t+1}) at time t. Equation 5.1 represents the baseline objective.

Bootstrapping in the Latent Space. To enhance diversity and mitigate overfitting, we introduce *latent space bootstrapping*, inspired by [34]. For a sample $x_0 \sim p_0(x)$ from the base latent distribution, representing an initial tuple (s_0, a_0, s_1), we generate B bootstrapped samples $\{x_0^b\}_{b=1}^B$ via resampling:

$$x_0^b = \text{Resample}(x_0), \quad b = 1, \ldots, B.$$

Let x_t^b denote the sample evolved from x_0^b at time t, with distribution $p_t^b(x)$. The bootstrapped flow matching objective extends Equation 5.1:

$$\min_{\mathcal{W}} \frac{1}{B} \sum_{b=1}^{B} \mathbb{E}_{\substack{t\in[0,1] \\ x_t^b \sim p_t^b(x)}} \left[\| v_{\mathcal{W}}(x_t^b) - w(x_t^b) \|^2 \right]. \tag{5.2}$$

This objective promotes diverse synthetic trajectories by leveraging resampled latent distributions, ensuring broader coverage of the joint state-action-next-state space.

5.3.3 Feature Weighting and Conditional Flow Matching

Bootstrapping adds diversity, but *feature weighting* ensures synthetic data resembles unstructured data for RL tasks, as required in our DVFS case study.

Random Forest Feature Importance. We employ a Random Forest regressor or classifier to assess how each real-world feature (e.g., sensor measurements in DVFS) contributes to predicting next-state transitions. Let d be the dimension of our state/action/next-state representation. The feature importance at time t for dimension (feature) i is Importance($x_{t,i}$), which we normalize:

$$\lambda_i = \frac{\text{Importance}(x_{t,i})}{\sum_{j=1}^{d} \text{Importance}(x_{t,j})}, \quad i = 1, \ldots, d. \tag{5.3}$$

In practice, the importance of each dimension (feature) is calculated based on how much the feature reduces impurity in classification or variance in regression tasks. The *Importance* may be updated periodically or assumed to be roughly constant if the importance ranks are stable. Features with large λ_i (e.g., temperature) are emphasized during training, preserving critical data relationships in synthetic samples.

Weighted Conditional Objective. We incorporate these weights into a *conditional* flow matching setup, where additional context z (e.g., rewards, auxiliary variables) follows $z \sim q(z)$. Define $x_{t,i}^b$ as the i-th feature of the bootstrapped sample at time t. The conditional flow matching objective with feature weighting extends (5.2):

$$\min_{\mathcal{W}} \frac{1}{B} \sum_{b=1}^{B} \mathbb{E}_{\substack{t\in[0,1] \\ x_t^b \sim p_t^b(x) \\ z\sim q(z)}} \Big[\sum_{i=1}^{d} \lambda_i \| v_{\mathcal{W}}(x_{t,i}^b; z) - w(x_{t,i}^b; z) \|^2 \Big]. \tag{5.4}$$

This ensures high-importance features (e.g., temperature, resource usage) are prioritized, preserving essential correlations in the joint distribution $p(s_t, a_t, s_{t+1})$ for robust synthetic data

generation in unstructured settings like DVFS.

5.3.4 Implications for RL: Regret Bounds under Model-Based Assumptions

We theoretically motivate the use of synthetic data in a **model-based** RL framework. Let $\mathcal{M}^*$ be the true MDP that governs the DVFS transitions, and let $\widehat{\mathcal{M}}$ be a learned MDP model derived from real and synthetic data. In model-based RL, the environment model $\widehat{\mathcal{M}}$ predicts next states s_{t+1} given states s_t and actions a_t via $\widehat{P}(s_{t+1}|s_t, a_t)$, unlike flow matching, which generates full tuples (s_t, a_t, s_{t+1}). Synthetic tuples from flow matching are used to train $\widehat{\mathcal{M}}$, enhancing its transition dynamics estimation. The redefined regret function, $\text{Reg}(T) = \sum_{t=1}^{T}[Q^*(s_t, a_t) - Q^\pi(s_t, a_t)] + O\left(\sigma\sqrt{\frac{\log(1/\delta)}{n+m}}\right)$, accounts for policy suboptimality and estimation error due to finite samples.

Enhanced Model Realizability. Synthetic data augments real data to improve the estimation of transition dynamics. The empirical Bellman operator for $\widehat{\mathcal{M}}$ is estimated using $\mathcal{D}_R$ and $\mathcal{D}_S$:

$$\widehat{\mathcal{T}}Q(s,a) = \frac{1}{n+m}\Big(\sum_{(s',r)\in\mathcal{D}_R}[r + \gamma \max_{a'} Q(s',a')]$$

$$+ \sum_{(s'',r)\in\mathcal{D}_S}[r + \gamma \max_{a'} Q(s'',a')]\Big).$$

Under bounded variance ($\text{Var}[r+\gamma \max Q] \leq \sigma^2$), a concentration bound gives, with probability $1-\delta$:

$$\|\widehat{\mathcal{T}}Q - \mathcal{T}Q\| \leq O\left(\sigma\sqrt{\frac{\log(1/\delta)}{n+m}}\right).$$

This matches the regret second term, indicating that synthetic data reduces estimation error by increasing the effective sample size, enabling sublinear regret $\mathrm{Reg}(T) = O(\sqrt{T})$ under standard assumptions (finite horizon, bounded rewards) [36].

Illustrative Sketch. A simplified argument is:

1. Each (s, a) pair is sampled with probability $\mu > 0$ across real and synthetic data.
2. The learned model $\widehat{\mathcal{M}}$ satisfies the above concentration bound.
3. By standard analyses [50, 36], regret satisfies:

$$\mathrm{Reg}(T) = O(\sqrt{T}) + O\left(\sigma\sqrt{\frac{\log(1/\delta)}{n+m}}\right).$$

The synthetic data contribution to m ensures sublinear regret with reduced sample complexity.

Practical Caveats. In high-dimensional settings like DVFS, exploration is constrained, and real-synthetic mismatch may arise if the flow model fails to capture corner cases. Feature weighting mitigates this by ensuring synthetic tuples resemble unstructured data, aligning with the redefined regret.

5.3.5 Implications for RL: Regret Bounds with Synthetic Data in Model-Free RL

In a model-free RL framework, synthetic data generated by flow matching augments real data to improve sample efficiency and reduce variance in Q-value estimates, leading to tighter regret bounds. Unlike model-based RL, which uses a learned model $\widehat{P}(s_{t+1}|s_t, a_t)$ to predict next

states, flow matching directly generates synthetic tuples (s_t, a_t, s_{t+1}) from the joint distribution $p(s_t, a_t, s_{t+1})$, which are stored in $\mathcal{D}_S$ for Q-learning updates.

Role of Synthetic Data via Flow Matching. Flow matching enhances the regret bound by generating synthetic tuples (s_t, a_t, s_{t+1}) that accurately capture the joint distribution $p(s_t, a_t, s_{t+1})$, expanding coverage of state-action-next-state combinations infrequently seen in real data. The bootstrapped objective (Equation 5.2) ensures diversity, while the feature-weighted conditional objective (Equation 5.4) preserves critical correlations (e.g., temperature and frequency in DVFS). This reduces the variance term $O\left(\sigma\sqrt{\frac{\log(1/\delta)}{n+m}}\right)$ by increasing m, outperforming real-data-only training (where the term is $O\left(\sigma\sqrt{\frac{\log(1/\delta)}{n}}\right)$). By modeling continuous-time density evolution, flow matching minimizes distributional bias, ensuring synthetic tuples align with true dynamics, accelerating Q-value convergence and achieving tighter regret bounds.

Bias arises if synthetic tuples deviate from the true distribution. Feature weighting ensures fidelity to unstructured data distributions, keeping bias minimal, making flow matching-augmented model-free RL effective for few-shot settings, as validated in our DVFS experiments.

5.3.6 From LLN to Finite-Sample Guarantees

Under mild mixing conditions, synthetic samples reduce the variance of empirical Bellman updates, yielding finite-sample bounds for both RL paradigms.

Algorithm 3 Distribution-Aware Flow Matching for Few-Shot Online RL.

Initialize: $\mathcal{D}_{\text{real}}, \mathcal{D}_{\text{syn}}$, DQN, FM
for $i = 1$ **to** T **do**
  Collect real tuple $\tau_{\text{real}} = (s_t, a_t, s_{t+1})$ and store in $\mathcal{D}_{\text{real}}$
  if enough new data in $\mathcal{D}_{\text{real}}$ **then**
    Train FM using Equation (5.4)
  end if
  for $j = 1$ **to rollout count do**
    x_0^b = Resample(x_0) ▷ bootstrap tuple (s_0, a_0, s_1)
    Evolve x_0^b to x_t^b via FM ▷ generate tuple (s_t, a_t, s_{t+1})
    Store synthetic tuple $\tau_{\text{syn}} = (s_t, a_t, s_{t+1})$ in $\mathcal{D}_{\text{syn}}$
  end for
  if enough data in $\mathcal{D}_{\text{real}}$ and $\mathcal{D}_{\text{syn}}$ **then**
    Sample mini-batches and update DQN
  end if
end for

Let $\mathcal{D}_R$ be the n real transitions and $\mathcal{D}_S$ the m synthetic ones. The empirical backup is:

$$\widehat{\mathcal{T}}Q(s,a) = \frac{1}{n+m}\Big(\sum_{(s',r)\in\mathcal{D}_R}[r+\gamma\max_{a'}Q(s',a')]$$

$$+\sum_{(s'',r)\in\mathcal{D}_S}[r+\gamma\max_{a'}Q(s'',a')]\Big).$$

With $\mathrm{Var}[r+\gamma\max Q] \le \sigma^2$, a Bernstein-type bound gives, with probability $1-\delta$:

$$|\widehat{\mathcal{T}}Q(s,a) - \mathcal{T}Q(s,a)| \le O\left(\sigma\sqrt{\frac{\log(1/\delta)}{n+m}}\right) + O\left(\frac{\log(1/\delta)}{n+m}\right).$$

Adding m synthetic samples shrinks the variance term, accelerating convergence of Q-value or model estimates.

5.3.7 Algorithmic Overview for Few-Shot Online RL

We integrate flow matching with a Deep Q-Network (DQN) using two replay buffers: $\mathcal{D}_{\text{real}}$ for real tuples (s_t, a_t, s_{t+1}) collected from the environment, and $\mathcal{D}_{\text{syn}}$ for synthetic tuples generated via flow matching.

In each iteration, we collect real tuples (s_t, a_t, s_{t+1}) from the environment, storing them in $\mathcal{D}_{\text{real}}$. When sufficient new data are available, we retrain or fine-tune the Flow-Matching model (FM) using the conditional weighted objective (Equation 5.4) or the unconditional bootstrapped objective (Equation 5.2). We generate synthetic tuples by resampling bootstrapped latent points $\{x_0^b\}$, each representing an initial tuple (s_0, a_0, s_1), evolving them over $t \in [0, 1]$ via FM to produce tuples (s_t, a_t, s_{t+1}), and storing these in $\mathcal{D}_{\text{syn}}$. Mini-batches are sampled from both $\mathcal{D}_{\text{real}}$ and $\mathcal{D}_{\text{syn}}$ to update the DQN agent's Q-function using the Q-learning update rule. This approach ensures robust state-action-next-state coverage for unstructured data, with bootstrapping enhancing diversity and feature weighting preserving critical correlations, supporting effective training for both model-based and model-free RL in few-shot settings, as validated in our DVFS case study.

5.4 Experiments

In this section, we present the experimental setup, hyperparameter tuning, model architecture, and comparisons with model-based and model-free baselines to evaluate our Flow-Augmented Reinforcement Learning (FARL) approach.

Experimental Setup We developed a Deep Q-Network (DQN) agent based on the zTT framework [53] to optimize system performance under thermal and power constraints on the NVIDIA Jetson TX2, a power-efficient embedded platform with six heterogeneous CPU cores and an embedded GPU. The goal is to dynamically adjust frequency and power settings to maximize performance while ensuring system stability. The setup uses a client-server archi-

tecture: the client collects semi-structured state tuples $\{FPS, f, \rho, \theta\}$ (frames per second, core frequency, power consumption, and core temperature), normalized via min-max scaling, and the server assigns frequency actions from a discrete set defined by hardware constraints (e.g., GPU_CLOCK_LIST). Each episode comprises 10 tuples collected at 1-second intervals over a 90-second horizon (90 decision points), with thermal monitoring triggering a 5–10-second sleep mode if temperature exceeds 50°C to mitigate throttling. The workflow is depicted in Figure 5.1.

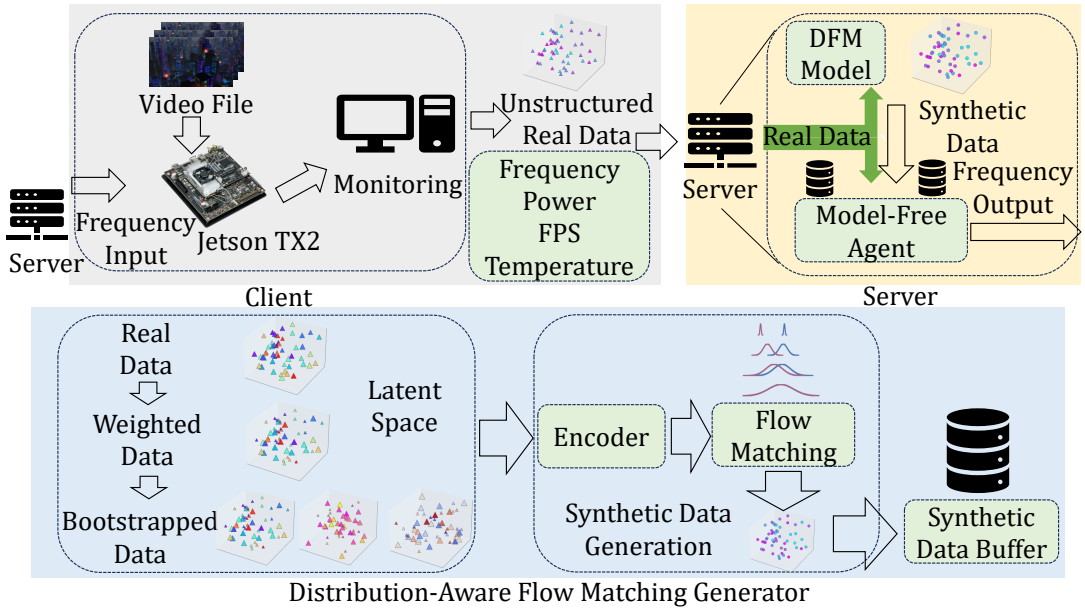


Figure 5.1: Workflow of the proposed Flow-Augmented Reinforcement Learning (FARL) approach.

Hyperparameter Tuning. We performed hyperparameter tuning using grid search, selecting values based on reward stability and convergence speed, as shown in Table 5.1. The learning rate (0.05) was chosen after grid search and adjusted dynamically (e.g., reset to 1 under specific

reward conditions), though it may seem high compared to typical deep learning ranges (e.g., 1e-4 to 5e-5). Future work will explore random search (e.g., Hyperopt) for refinement.

Hyperparameter	**Tuning Values**
Experiment Time	{50, **90**, 200, 400, 800}
Target FPS	{**30**, 60, 90}
Target Temperature	{**50**, 60}
Learning Rate	{0.01, **0.05**, 0.1}
Discount Factor	{0.95, **0.99**}
Epsilon Decay	{0.95, **0.99**}
Batch Size	{**32**, 64, 128}
Agent Train Start	{**32**, 40, 80, 100}
Planning Iterations	{**100**, 200, 1000}
Model Train Start	{**32**, 200, 300}
Reward Scale (β)	{**0**, 2, 4}

Table 5.1: Main Hyperparameters and Tuning Values.

Model Architecture and Training. The DQN agent features a neural network with two hidden layers of six neurons each, optimized using mean squared error (MSE) loss and the Adam optimizer. An ϵ-greedy strategy starts with $\epsilon = 1.0$ and decays over time to promote exploitation. Training occurs in discrete intervals: Agent Train Start initiates Q-network updates, and Planning Iterations controls MDP model updates, using mini-batches of real and synthetic data. Performance metrics include Frames Per Second (FPS) for responsiveness, power consumption (ρ) for efficiency, average temperature (θ) for safety, and average maximum Q-values for policy convergence, with emphasis on FPS andβ. The reward function, adapted from [53], is:

$$R = u + v + \frac{\beta}{\rho},$$

where

$$u = \begin{cases} 1 & \text{if } FPS \geq \text{target_FPS}, \\ \frac{FPS}{\text{target_FPS}} & \text{otherwise}, \end{cases}$$

and

$$v = \begin{cases} 0.2 \cdot \tanh(\text{target_temp} - \theta) & \text{if } \theta < \text{target_temp}, \\ -2 & \text{if } \theta \geq \text{target_temp}. \end{cases}$$

With target FPS at 30 and target temperature at 50°C, this design balances performance and constraints.

Model-Based and Model-Free Baselines. We compare FARL with DynaQ [76], a model-based RL method; PlanGAN [21], a generative model-based approach with dual-memory; MAML [37], a meta-learning method; and zTT [53], a model-free DQN baseline for DVFS. PlanGAN and DynaQ use real and synthetic data, while zTT runs directly on the Jetson TX2 without synthetic data. Consistent hyperparameters and reward functions ensure fair comparisons, with thermal throttling mitigated by sleep mode.

Results and Analysis. In our experiments, we train a *conditional flow matching* model [97] on the real trajectories stored in a replay buffer, then use it to generate synthetic trajectories. We refer to this baseline generative approach as "standard flow matching." We further refine it into a *Flow-Augmented Reinforcement Learning (FARL)* method by incorporating latent-space bootstrapping and feature weighting to preserve important correlations and diversify

the latent representation.We evaluate how well each synthetic approach captures correlations among features by computing Pearson's correlation coefficient [86]. For two features x_i and x_j with n samples, the coefficient is given by

$$\text{corr}_{ij} = \frac{\sum_{k=1}^{n}(x_{i,k} - \bar{x}_i)\,(x_{j,k} - \bar{x}_j)}{\sqrt{\sum_{k=1}^{n}(x_{i,k} - \bar{x}_i)^2}\,\sqrt{\sum_{k=1}^{n}(x_{j,k} - \bar{x}_j)^2}}. \tag{5.5}$$

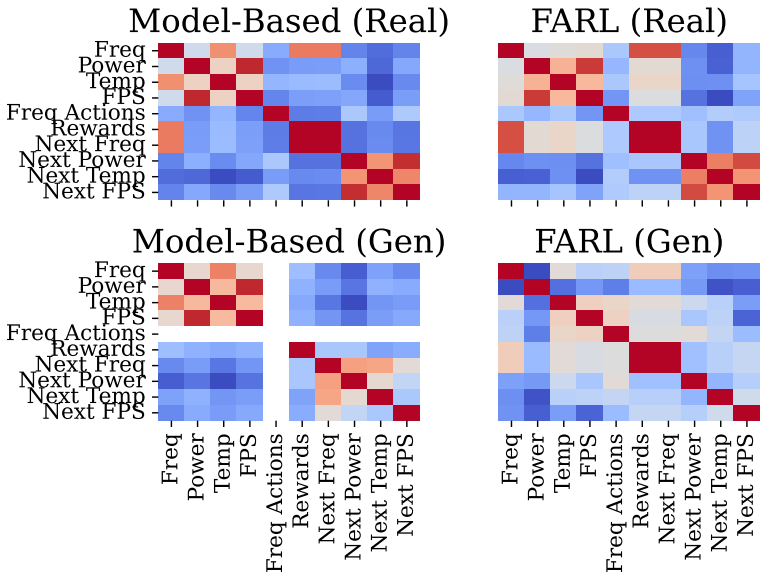


Figure 5.2: Correlation matrix comparisons.

When comparing the correlation matrices, we find that FARL-generated data better captures correlations among features visually compared to the model-based approach. Although the standard flow matching approach does capture some correlations, its matrix appears more uniform and less differentiated than that of FARL, suggesting that some critical feature relationships are weakened or diluted. Meanwhile, the model-based method shows stronger but sometimes misplaced correlations (dark regions in incorrect locations), indicating potential

overfitting to specific patterns and misrepresenting genuine dependencies. FARL balances the two extremes by retaining essential correlations without artificially magnifying or suppressing them.

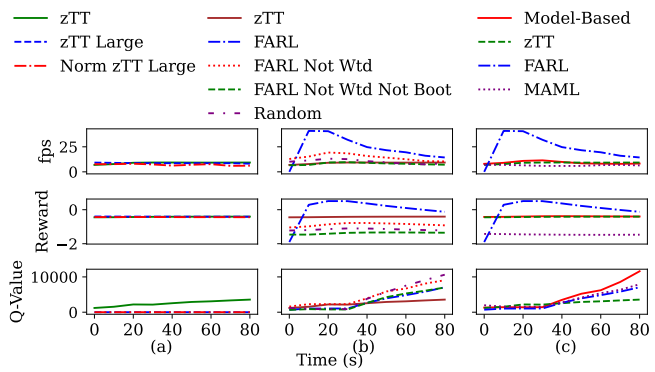


Figure 5.3:
Performance comparisons across methods.

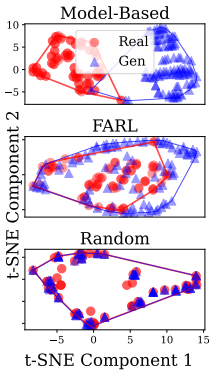


Figure 5.4:
Real and (Gen)erated data t-SNE.

To further analyze the data, we use t-SNE visualizations to compare the distributions in Figure 5.4.

The t-SNE comparison of model-based and FARL with weighted and bootstrapped data and random data generation shows that FARL covers real data and more, while random sampling is restricted to the real data distribution and the model-based approach does not cover real data. This indicates FARL's superior ability to generalize beyond the original dataset. We also perform a Dynamic Time Warping (DTW) evaluation to assess temporal alignment. The DTW evaluation reveals that model-based data generation has a higher value (10,640) compared to FARL weighted and bootstrapped (7,341) between generated and real data, confirming FARL's better temporal alignment.

To examine distributional coverage, we compare the histograms of key state-action features under different synthetic data generation methods. The results indicate that the FARL approach

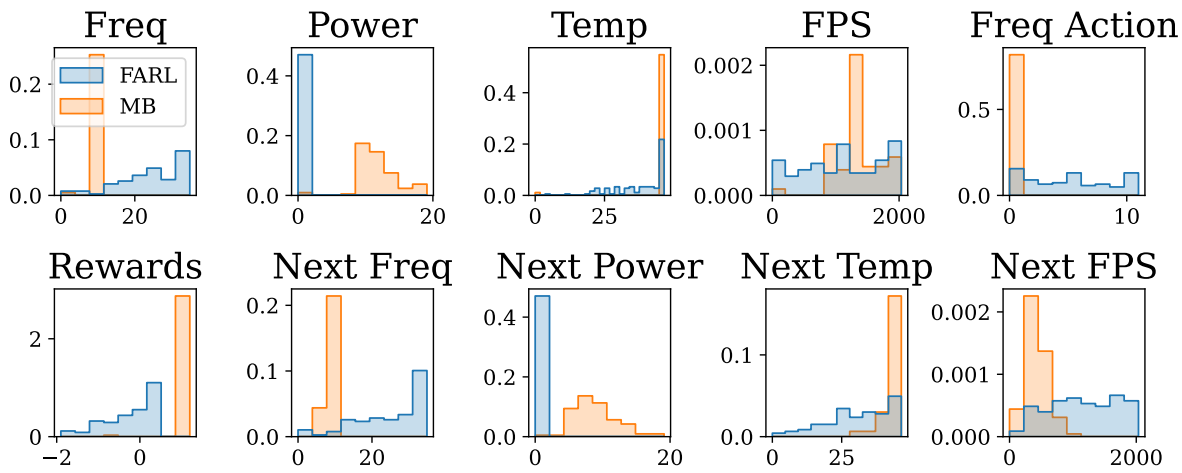


Figure 5.5: Comparisons of distributions of FARL and Model-Based (MB) generated data.

covers a wider spectrum of values compared to the model-based approach. In contrast, the model-based approach tends to be narrowly concentrated, particularly in power states and next states, suggesting it may overfit and fail to generalize to a broader range of real-world scenarios. The standard flow matching method similarly suffers from less coverage than FARL but does not exhibit the same extreme concentrations as the purely model-based approach. Overall, FARL's expanded coverage helps the RL agent explore and learn policies that are more robust to varied conditions.

Performance Comparisons. To comprehensively evaluate FARL, three subplots were generated to compare performance metrics (average FPS, average reward, and average maximum Q-value) across different methods in 90 seconds, with the x-axis labeled in seconds (each

second representing one iteration result based on FPS), as shown in Figure 5.3. This figure includes a comprehensive ablation analysis of different implementations.

The left column of subplot (a) compares different model-free approaches based on zTT, comprising zTT, zTT with extended features from profiling data, and zTT with normalization, showing no specific differences in terms of average reward, Q-values, and frame rate, with stable FPS around 10. The middle figure (b) compares zTT with FARL, FARL without weighted but with bootstrapping (FM Not Wtd), FARL without augmentation, and random generated synthetic data augmented models, showing faster convergence in the FARL weighted bootstrapped version compared to all other approaches. The right column (c) extends the comparison to MAML, zTT, model-based, and FARL, where FARL outweighs all other approaches.

The above experiments confirm that incorporating bootstrapping and feature weighting into flow matching (i.e., FARL) improves both correlation preservation and distribution diversity in synthetic data generation. This yields more stable and high-performing RL policies compared to standard flow matching or purely model-based methods. The ability to balance performance and power highlights the potential of Flow-Augmented Reinforcement Learning for real-world DVFS tasks, where state-action spaces can be both large and only partially observed in limited data collection.

5.5 Related Work

Machine Learning for DVFS. Machine learning has been widely adopted to optimize Dynamic Voltage and Frequency Scaling (DVFS) strategies, addressing the complexity of modern processors with adaptive control [117, 54, 30, 127, 74, 91, 106, 116, 66, 88, 98, 103, 14].

A survey by [74] notes that model-free reinforcement learning (RL), such as the zTT framework [53], dominates, focusing on direct policy optimization for power-performance trade-offs. However, these methods often overlook feature correlation intricacies and the computational overhead of extensive data collection, particularly in data-scarce settings. Our work builds on this by introducing FARL to enhance sample efficiency for semi-structured sensor data in DVFS.

Statistical Learning for DVFS. Statistical learning techniques have been explored to evaluate hardware events and application parameters impacting DVFS performance [85, 20, 66]. For example, [85] uses decision trees for energy-efficient DVFS lookup optimization, while [20] assesses hardware counter importance for power reduction. [66] applies f-score-based ranking to correlate compiler-level features with latency. These efforts, however, lack systematic consideration of runtime performance variability, sampling inefficiencies, and feature correlation effects on learning accuracy. FARL addresses these gaps by integrating feature weighting with Random Forests and data augmentation tailored for low-energy DVFS applications.

Bounding Sample Efficiency. Sample efficiency bounds for Q-learning have been a focus in RL research [39, 9, 36, 50]. Sublinear bounds have been established for finite state-action spaces [9, 50], but extending these to continuous spaces requires robust function approximators. Approaches like [36] rely on strong statistical assumptions, limiting real-world applicability. Deep Q-learning with stabilization techniques (e.g., experience replay, target networks) often faces instability [39], with few theoretical guarantees. Unlike these, FARL empirically

validates sample efficiency for continuous semi-structured data in DVFS, offering practical insights over theoretical bounds.

Few-Shot RL. Few-shot RL methods, including transfer learning, meta-learning, and model-based approaches, aim to reduce data needs and enhance generalization [102, 57, 105, 71, 38, 26, 8, 108, 81, 99]. Model-agnostic meta-learning (MAML) [37] adapts to new tasks with minimal data, while model-based methods [70] approximate transition dynamics. DVFS-specific works like [64, 53, 124, 121] explore multi-agent RL and transfer learning but neglect statistical resampling or feature selection for energy optimization. FARL advances this by using flow matching and bootstrapping to augment semi-structured sensor data, improving efficiency in DVFS and extending to robotics and smart grids.

5.6 Conclusion

We presented FARL, a novel method for few-shot RL with semi-structured sensor data. By leveraging flow matching, latent space bootstrapping, and feature-weighted conditioning, FARL generates high-quality synthetic data to enhance sample efficiency and policy robustness. Experiments on a DVFS task demonstrate up to 35% higher frame rates and faster convergence compared to baselines like DQN, DDQN, and MAML. FARL's applicability extends to other domains, such as robotics, offering a scalable solution for data-scarce RL. Future work includes exploring advanced RL algorithms (e.g., PPO) and neural-based feature weighting to further improve performance.

CHAPTER 6 GRAPHPERF-RT: A GRAPH-DRIVEN PERFORMANCE MODEL FOR HARDWARE-AWARE SCHEDULING OF OPENMP CODES

Abstract

We present GraphPerf-RT, a fast, uncertainty-aware surrogate for multi-dimensional performance of OpenMP workloads on heterogeneous CPUs/SoCs. The model augments CFG-derived and task-DAG graph features with device and thermal context and maps the resulting vector to performance targets without relying on benchmark identifiers. It predicts makespan, energy, cache and branch misses, utilization, and related metrics for accurate environment modeling. This yields an end-to-end estimator that uses graph topology and OpenMP code structure. We detail feature evaluation, graph encoding, and model evaluation. Validation spans Jetson TX2, Orin NX, and Rubik Pi. We also establish an embedded-only baseline on Jetson TX2, Orin NX, and Rubik Pi. We integrate the surrogate with a model-based HMARL wrapper for few-shot scheduling. GraphPerf-RT accurately drives makespan from graph embedding features, per-core DVFS, core masks, and priorities. On Jetson TX2 and Rubik Pi, the model achieves high accuracy on benchmarks drawn from the OpenMP-serial suite. We report feature-attribution diagnostics across task, resource, and topology groups, and nonparametric significance tests across devices.

6.1 Introduction

Performance prediction for OpenMP workloads on heterogeneous embedded SoCs is hard. Task-DAG structure, control-flow irregularity, cache and branch behavior, and thermal dynamics co-determine makespan and energy. DVFS and core assignment create cross-layer couplings that vary over time. Safe, on-board planning needs a fast evaluator with calibrated uncertainty, not only point estimates. Classical rules, simplified analytics, and heavy profiling struggle under irregularity and offer weak transfer. Tabular regressors discard structure. LLMs lack grounded program/hardware semantics. RL needs many trials and can overheat small boards. GNNs can exploit structure and interactions [100].

We introduce GraphPerf-RT, a graph-grounded surrogate for multi-dimensional performance of OpenMP task DAGs on embedded SoCs. Inputs fuse (i) ALF/EFG-derived task-DAG topology, (ii) CFG-informed source embeddings, and (iii) device/runtime context: per-core DVFS, core masks, run_mode, and thermal state. We build a heterogeneous graph with task and resource nodes and typed edges for precedence, placement, and contention. Type-specific MLP encoders feed a heterogeneous attention layer with edge transforms and topological encodings. Multi-task evidential heads output Normal–Inverse–Gamma parameters for makespan, energy, utilization, and microarchitectural counters, yielding calibrated intervals alongside means. The evaluator scores candidate (mask, DVFS) in batch, filters low-confidence proposals, and supports Dyna-style planning with cheap, hardware-grounded rollouts.

Our pipeline is reproducible and portable. We compile with OMPi/ALF-llvm to extract `.ll`/`.alf`, run SWEET to emit DOT graphs, and pair graph features with device/context

telemetry. A lightweight client–server profiler applies userspace DVFS, sets core masks, launches with RT options, samples power rails, records thermal zones before/after, and reads per-core frequencies. Each run logs one CSV row in a unified schema across boards. We evaluate on Jetson TX2, Jetson Nano, and RubikPi over BOTS/PolyBench in sequential and parallel modes and multiple inputs. Baselines include linear/MLP tabular models and graph baselines (GCN, heterogeneous transformers). Metrics cover error, ranking, and calibration. Results show that structure + semantics + context improve accuracy and transfer, and that evidential learning provides useful confidence for safe rollout filtering.

Overall, the main contributions of this paper are:

1. A unified heterogeneous graph representation that fuses CFG-aware semantics with device and schedule context for OpenMP task DAGs on embedded SoCs.

2. A fast multi-task evidential surrogate for makespan, energy, utilization, and counters with calibrated uncertainty for risk-aware decisions.

3. A practical evaluator that ranks (mask, DVFS) actions, gates low-confidence rollouts, and plugs into few-shot scheduling and MARL under DVFS and thermal constraints.

4. A reproducible data and tooling stack (OMPi+ALF-llvm+SWEET+telemetry) and comprehensive evaluation on Jetson-class boards with strong baselines and ablations.

6.2 Related Work

Performance modeling for parallel systems spans analytical queueing/Petri-net models and simulators [32], then statistical and neural surrogates [56, 48, 104, 23]. Black-box predictors

(counters, instruction mixes) improve accuracy but lose structural signals and transfer. Hierarchical/learned throughput models (e.g., Ithemal) and runtime autotuners (Apollo) show gains on CPUs and kernels [69, 78]. Our focus differs: we model OpenMP task DAGs on embedded SoCs, explicitly encoding dependencies, per-core DVFS, and thermal context to capture cross-layer effects needed for real-time decisions.

Graph neural networks enable learning on structured program and system graphs [107, 100]. Program-graph IRs (e.g., ProGraML) support compiler tasks and phase ordering [28], and GNNs have predicted hardware-dependent performance for ML models [92] and optimization effects [16]. Compiler cost models (Ansor, MetaSchedule, ROLLER) learn evaluators for tensor kernels and improve cross-device transfer [122, 89, 126, 119, 120], but operate below application-level task graphs and rarely encode runtime DVFS/thermal state. We construct a heterogeneous graph with task and resource nodes, typed edges with execution features, and CFG-informed encoders to improve zero-/few-shot generalization across benchmarks and boards.

Hardware-aware ML includes NAS with device constraints and cross-platform predictors [19], and performance modeling across CPU/heterogeneous configurations [101]. Energy/thermal control spans heuristic/optimization DVFS and RL [112]. Utilization-driven governors can mis-scale frequency by conflating compute and stalls [42, 64]. Model-free RL adapts but is sample- and heat-intensive on small boards [53]. Model-based control reduces sampling via learned dynamics and planning [70]. Our approach is complementary: a calibrated, graph-grounded evaluator supplies fast, hardware-aware rollouts to search/MARL under DVFS and

thermal constraints.

Uncertainty is critical for safe scheduling. Bayesian and ensemble methods provide confidence but add inference cost [55, 52]. Evidential learning yields calibrated intervals without sampling [87, 6]. We adopt evidential regression heads to expose confidence on makespan, energy, and counters and to gate synthetic rollouts, aligning with on-board, real-time planning.

6.3 Preliminaries

6.3.1 Problem Formulation

Workload. Each program is a directed acyclic graph (DAG) $G = (V, E)$ recovered from OpenMP artifacts and compiler analyses. A task $v \in V$ represents a unit of work with static code features and optional runtime summaries. An edge $e = (u \to v) \in E$ encodes precedence (`spawn`, `join`, data).

Device sheet. Let D denote immutable hardware metadata: core types and counts, cache sizes/lines, DVFS tables per core/cluster, governor support, sensor layout/resolution, and toolchain hashes. D is constant within an episode.

Runtime state. Let $\mathcal{S}$ collect dynamic context: active core mask, per-core DVFS indices and measured frequencies, utilization EMA, thermal zones (before/after), and recent perf counters.

Controls. An action $a = (m, f, p)$ specifies a core mask $m \in \{0, 1\}^C$, a per-core DVFS vector f with $f_i \in \mathcal{F}_i$, and an optional FIFO priority $p \in \{1, \ldots, 99\}$. Actions must satisfy availability, affinity, and thermal caps.

Targets. Given $(G, D, \mathcal{S}, a)$, the evaluator predicts $y = ($makespan, energy, cache misses,

branch misses, utilization). Each target carries calibrated uncertainty (aleatoric and epistemic).

Mapping. We learn

$$\mathcal{M}: \ (G, D, \mathcal{S}, a) \ \mapsto \ (y, \ \mathcal{U}),$$

where $\mathcal{U}$ are Normal–Inverse–Gamma (NIG) parameters per target. Point estimates are the predictive means; intervals derive from the NIG posterior.

Scheduling/RL use. Candidates (m, f) are scored in batch. Ranking is makespan-first with uncertainty gates. Safe actions execute; outcomes (time, energy, counters, thermals) are logged for continual refinement.

Assumptions. Graphs are acyclic. The device sheet is fixed during an episode. Thermal policies may shrink the feasible action set. OS noise and co-runners induce stochasticity; our logs supervise the model under these conditions.

6.3.2 Heterogeneous Graph Abstraction

Node types. We form a heterograph with task nodes V_T, resource (core) nodes V_R, and optional memory nodes V_M.

Task features. CFG-derived embeddings; static code stats (loops, bytes moved, branch proxies); DAG topology (depth, distance-to-sink, centralities); recent perf snapshots; run-mode flags; thermal footprint on hosting cores.

Resource features. DVFS step (index/one-hot), mask bit, cluster ID, utilization EMA, thermal headroom and trend, bandwidth proxy.

Memory features (optional). Cache level, capacity/associativity/line size, latency/bandwidth proxies.

Edge types. E_{TT}: precedence with type (spawn/join/data), critical-edge flag, hop distance, contention proxies. E_{TR}: task↔core placement under the recorded mask and DVFS. E_{RR}: core↔core relations (shared caches, interconnect). E_{RM}: core↔memory links.

Device broadcast. Constants in D (e.g., DVFS tables, cache sizes) are injected into encoders as shared features.

6.3.3 Data and Artifact Pipeline

Compilation and graphs. OpenMP sources are compiled with OMPi. ALF-llvm emits `*.ll`/`*.alf`. SWEET generates DOT graphs (`cfg,cg,rsg,fsg,sgh`). We map ALF/WCT entities to OpenMP tasks, merge tiny chains, and attach topological encodings.

Runtime logging. Each run yields one CSV row with: timestamp, iteration, run_mode, assigned DVFS indices (`frequency_combination`), measured per-core Hz (`frequencies_read`), num_cores, `cores_str`, input, `time_elapsed`, rail energy/power, perf counters (`cycles`, `instructions`, `cache_misses`, `branch_misses`, `task_clock`, `cpu_clock`, faults), and thermal zones *before/after*. We derive ΔT, headroom, and alignment windows for counters.

Preprocessing. Duplicate tuples over (G, input, m, f) are removed. Outliers are filtered via MAD on residuals. Features are z-scored per device; we retain global stats for transfer ablations. Train/val/test is 60/20/20 by *(benchmark, input, core mask, DVFS index)* to prevent configuration leakage.

6.3.4 Learning and Outputs

Encoders and backbone. Type-specific MLPs embed task/resource/memory nodes. A heterogeneous GAT with edge-type transforms performs message passing over $E_{TT}, E_{TR}, E_{RR}, E_{RM}$.

Hierarchical pooling yields the graph vector $\mathbf{h}_G$.

Evidential heads. For each target k, the head outputs $(\gamma_k, \nu_k, \alpha_k, \beta_k)$. Means are $\hat{y}_k = \gamma_k$. Uncertainty decomposes as

$$\text{Aleatoric}_k = \frac{\beta_k}{\alpha_k - 1}, \qquad \text{Epistemic}_k = \frac{\beta_k}{\nu_k(\alpha_k - 1)}.$$

Training minimizes the negative log marginal likelihood with evidence regularization and a ranking term aligned to makespan-first selection.

Runtime path. Batched candidates → scores and intervals → uncertainty gate → device execution → CSV row → optional replay. This matches the pipeline in Fig. 6.1 and the method specification in Section 6.4.

6.4 Design Methodology

6.4.1 Problem Formulation

We model an OpenMP application as a task DAG $G = (V, E)$; tasks $v \in V$ carry static code metrics and input-dependent attributes, and edges $e \in E$ encode precedence from OpenMP dependencies. The target platform has C heterogeneous cores. Each core i uses DVFS levels $f_i \in \mathcal{F}_i$. The system state $\mathcal{S}$ includes the active core mask $m \in \{0, 1\}^C$, frequencies f, thermals (zone temperatures and headroom), and recent utilization/perf snapshots.

We learn $\mathcal{M} : (G, D, \mathcal{S}, a) \to (y, \mathcal{U})$, where D is the device sheet, $a = (m, f, p)$ with optional priority $p \in \{1, \ldots, 99\}$, and $y = [y_1, \ldots, y_K]$ are metrics (makespan, energy, cache/branch misses, utilization). $\mathcal{U} = [u_1, \ldots, u_K]$ provides aleatoric and epistemic uncertainty for risk-

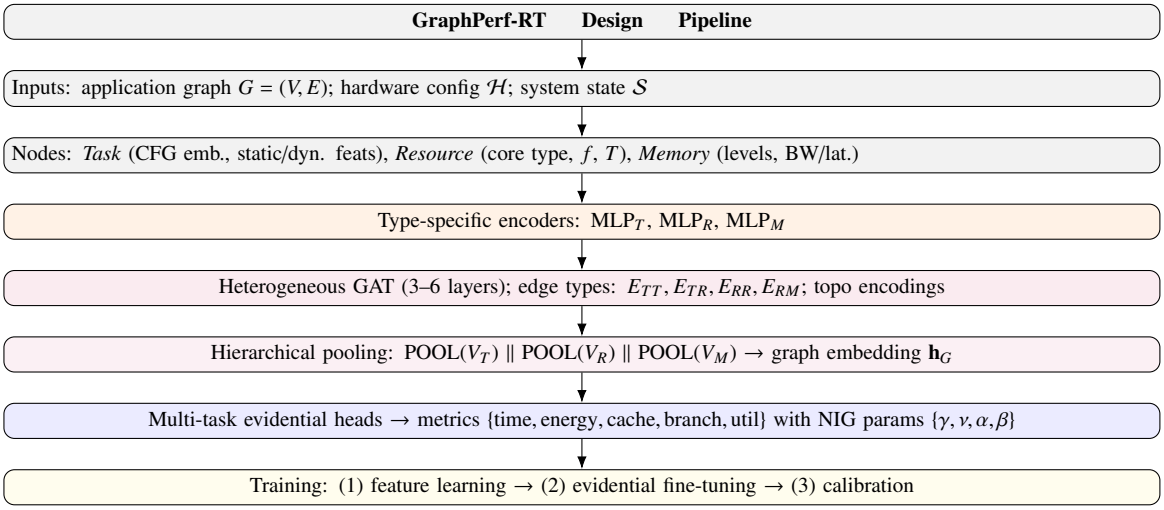


Figure 6.1: End-to-end pipeline.

aware control. Assumptions follow Section 6.3.1: acyclic graphs, fixed D per episode, and stochasticity from OS/co-runners captured in logs.

6.4.2 Heterogeneous Graph Representation

The core innovation of GraphPerf-RT lies in its unified heterogeneous graph representation that captures both the application structure and the hardware architecture in a manner consistent with the abstraction in Section 6.3.2. This representation consists of three types of nodes (V_T, V_R, V_M) and multiple edge types (E_{TT}, E_{TR}, E_{RR}, E_{RM}) that encode different relationships within the system, allowing the model to reason about dependencies, placements, contentions, and resource constraints holistically.

Node Types

Task Nodes (V_T): Each OpenMP task is represented as a task node in the graph. Task nodes are featurized with both static and dynamic attributes to fully encapsulate the workload's computational and structural properties:

- **CFG Embeddings:** We extract the Control Flow Graph (CFG) from the source code associated with each task using tools like SWEET or ALF-llvm, and generate dense vector embeddings using a pre-trained code representation model (e.g., fine-tuned on domain-specific code corpora to capture OpenMP-specific patterns). These embeddings capture semantic properties such as control flow complexity (e.g., branching factors), loop structures (e.g., nested depths and iteration bounds), computational patterns (e.g., arithmetic intensity), and potential irregularities that affect predictability.

- **Static Features:** Task-level characteristics including estimated instruction count (from compiler IR), memory footprint (bytes allocated or accessed), parallelization degree (e.g., number of subtasks spawned), branch proxies (e.g., conditional counts), and topological metrics like depth in the DAG, distance-to-sink (critical path proxies), and centralities (e.g., betweenness to indicate bottleneck potential).

- **Dynamic Features:** Runtime-dependent attributes such as input data size (affecting memory intensity), iteration counts (for loops with variable bounds), dependency fan-in/fan-out (indicating parallelism width), recent perf snapshots (e.g., cycles per instruction from prior runs), run-mode flags (e.g., sequential vs. parallel), and thermal footprint on hosting cores (e.g., estimated heat generation based on operation types).

Resource Nodes (V_R): Each processing core in the heterogeneous system is represented as a resource node, encoding both architectural characteristics and dynamic state information to model hardware heterogeneity and runtime variability:

- **Architectural Features:** Core type (e.g., big vs. LITTLE in big.LITTLE architectures), cache hierarchy specifications (e.g., L1/L2 sizes, associativity), supported instruction sets (e.g., NEON/SVE flags), peak computational capacity (e.g., FLOPS at max frequency), cluster ID, and interconnect proxies (e.g., bandwidth to shared resources).

- **Dynamic Features:** Current frequency setting (DVFS index or one-hot encoded), utilization level (EMA over recent intervals), temperature (from thermal zones), thermal headroom (computed as $T_{\text{max}} - T_{\text{current}}$, where T_{max} is the throttling threshold), trend (e.g., ΔT over last samples), and bandwidth proxy (e.g., estimated memory throughput under current load).

- **Power Characteristics:** Power consumption models (e.g., quadratic approximations of power vs. frequency) and DVFS efficiency curves (e.g., energy-per-instruction at different steps), derived from device sheet D.

Memory Nodes (V_M): Memory hierarchy components (L1/L2/L3 caches, main memory) are represented as memory nodes to capture memory subsystem characteristics and contention effects, including level (e.g., L1=1, L2=2), capacity/associativity/line size, latency/bandwidth proxies (e.g., cycles per access, GB/s), and contention indicators (e.g., shared vs. private).

Device constants from D (e.g., DVFS tables, cache sizes) are broadcast as shared features across relevant nodes during encoding.

Edge Types

The heterogeneous graph includes multiple edge types that encode different relationships, each potentially carrying attributes like weights, types, or proxies:

Task-Task Edges (E_{TT}): Directed edges representing precedence constraints between tasks, derived from the original OpenMP dependency graph, with attributes such as type (spawn/join/-data), critical-edge flag (1 if on longest path), hop distance, and contention proxies (e.g., data volume transferred).

Task-Resource Edges (E_{TR}): Bidirectional edges connecting tasks to the resources on which they execute or could potentially execute, encoding the current scheduling assignment under mask m and DVFS f, with attributes like affinity strength (e.g., based on core type suitability) and placement cost (e.g., migration overhead proxy).

Resource-Resource Edges (E_{RR}): Undirected edges between resource nodes that share hardware components (e.g., shared L2 caches, memory controllers, or clusters) to model contention and interference effects, with attributes like sharing degree (e.g., number of shared ways) and interconnect latency.

Resource-Memory Edges (E_{RM}): Directed edges connecting processing cores to memory hierarchy components, enabling the model to reason about memory access patterns and bandwidth constraints, with attributes such as access frequency (e.g., expected loads/stores) and bandwidth allocation.

This rich representation ensures the model captures cross-layer interactions, aligning with the data pipeline in Section 6.3.3.

6.4.3 Graph Neural Network Architecture

GraphPerf-RT employs a Graph Attention Network (GAT) architecture specifically designed to handle heterogeneous graphs with multiple node and edge types, building on the encoders and backbone described in Section 6.3.4. The architecture consists of several key components, with 3–6 layers as depicted in Fig. 6.1, using hidden dimensions typically in $[128, 256]$ for embeddings, and multi-head attention (e.g., 4–8 heads) to stabilize learning.

Type-Specific Encoders

Since different node types have distinct feature spaces, we employ type-specific Multi-Layer Perceptrons (MLPs) to encode raw features into a common embedding space of dimension d (e.g., 128):

$$\mathbf{h}_v^{(0)} = \mathrm{MLP}_{\mathrm{type}(v)}(\mathbf{x}_v; \theta_{\mathrm{type}(v)}) \tag{6.1}$$

where $\mathbf{x}_v$ represents the raw features of node v, $\mathrm{type}(v) \in \{T, R, M\}$, and $\theta_{\mathrm{type}(v)}$ are learnable parameters for each MLP (e.g., 2–3 layers with ReLU activations and dropout $p = 0.1$). Topological encodings (e.g., positional embeddings based on DAG depth or core cluster IDs) are concatenated to $\mathbf{x}_v$ before encoding to preserve structural information.

Heterogeneous Graph Attention Layers

The core of the architecture consists of multiple heterogeneous graph attention layers that aggregate information from neighboring nodes while accounting for edge types and their at-

tributes. For each layer l, the update for node v is:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{h}_v^{(l)} + \sum_{r\in\mathcal{R}} \sum_{u\in\mathcal{N}_r(v)} \alpha_{uv,r}^{(l)} \mathbf{W}_r^{(l)} \mathbf{h}_u^{(l)}\right) \tag{6.2}$$

where $\mathcal{R} = \{TT, TR, RR, RM\}$ is the set of edge types, $\mathcal{N}_r(v)$ is the neighborhood of v under edge type r, $\alpha_{uv,r}^{(l)}$ is the attention coefficient, $\mathbf{W}_r^{(l)}$ is a type-specific linear transformation matrix, and σ is a non-linearity (e.g., ELU). Residual connections are added for deeper models to mitigate vanishing gradients.

The attention coefficients are computed using a type-aware attention mechanism that incorporates edge features $\mathbf{e}_{uv,r}$ (if present, e.g., contention proxies):

$$e_{uv,r}^{(l)} = \text{LeakyReLU}(\mathbf{a}_r^T$$

$$[\mathbf{W}_r^{(l)}\mathbf{h}_u^{(l)} \| \mathbf{W}_r^{(l)}\mathbf{h}_v^{(l)} \| \phi_r(\mathbf{e}_{uv,r})])$$

$$\alpha_{uv,r}^{(l)} = \frac{\exp(e_{uv,r}^{(l)})}{\sum_{r'\in\mathcal{R}} \sum_{w\in\mathcal{N}_{r'}(v)} \exp(e_{wv,r'}^{(l)})}$$

where $\mathbf{a}_r$ is a type-specific attention vector, $\|$ denotes concatenation, and ϕ_r is an optional edge feature projector (e.g., a 1-layer MLP with ReLU). Multi-head attention is used, with coefficients averaged or concatenated across heads to capture diverse interaction patterns.

Graph-Level Pooling

After the final graph attention layer L, we apply a hierarchical pooling strategy to obtain a graph-level representation. This involves separate pooling operations for different node types (e.g., mean, max, or attention-based pooling), followed by concatenation:

$$\mathbf{h}_G = \text{CONCAT}(\text{POOL}_T(\{\mathbf{h}_v^{(L)} : v \in V_T\}),$$
$$\text{POOL}_R(\{\mathbf{h}_v^{(L)} : v \in V_R\}), \text{POOL}_M(\{\mathbf{h}_v^{(L)} : v \in V_M\})) \tag{6.3}$$

where $\text{POOL}_{\text{type}}$ are learnable (e.g., via attention) or fixed aggregators (e.g., mean), yielding a fixed-size $\mathbf{h}_G$ (e.g., dimension 256–512) regardless of graph size. This ensures scalability and enables batched inference.

6.4.4 Evidential Learning for Uncertainty Quantification

To provide uncertainty-aware predictions, GraphPerf-RT employs an evidential learning framework consistent with Section 6.3.4. Instead of directly predicting performance values, the model learns the parameters of a Normal-Inverse-Gamma (NIG) distribution for each performance metric via multi-task heads on $\mathbf{h}_G$:

$$(\gamma_k, \nu_k, \alpha_k, \beta_k) = \text{MLP}_k(\mathbf{h}_G; \theta_k) \tag{6.4}$$

for each metric $k \in \{$ makespan, energy, cache misses, branch misses, utilization $\}$, where MLP_k is a shared trunk (e.g., 2 layers, 128 units) with task-specific outputs (e.g., linear layers with softplus activations on $\nu_k, \alpha_k > 1, \beta_k > 0$ to ensure valid distributions). The predictive distribution for metric k is given by:

$$p(y_k|\mathbf{h}_G) = \text{NIG}(y_k; \gamma_k, \nu_k, \alpha_k, \beta_k) = \int \mathcal{N}(y_k; \mu, \sigma^2)\text{IG}(\sigma^2; \alpha_k, \beta_k)\, d\sigma^2 \tag{6.5}$$

with $\mu \sim \mathcal{N}(\gamma_k, \sigma^2/\nu_k)$. The mean prediction and uncertainty estimates are derived as:

$$\hat{y}_k = \gamma_k \tag{6.6}$$

$$\text{Aleatoric Uncertainty}_k = \frac{\beta_k}{\alpha_k - 1} \tag{6.7}$$

$$\text{Epistemic Uncertainty}_k = \frac{\beta_k}{\nu_k(\alpha_k - 1)} \tag{6.8}$$

$$\text{Total Variance}_k = \text{Aleatoric}_k + \text{Epistemic}_k \tag{6.9}$$

The model is trained by minimizing the negative log marginal likelihood of the evidential distribution, augmented with evidence regularization (to prevent underconfidence) and a ranking loss (e.g., pairwise on makespan to align with scheduling objectives):

$$\mathcal{L} = \sum_{k=1}^{K} \sum_{i=1}^{N} [\frac{1}{2} \log(\frac{\pi}{\nu_{k,i}})$$
$$-\frac{\alpha_{k,i} - 1}{2} \log(2\beta_{k,i}(1 + \nu_{k,i}\epsilon_{k,i}^2)) + .$$
$$(\alpha_{k,i} + \frac{1}{2}) \log(1 + \nu_{k,i}\epsilon_{k,i}^2) +$$
$$\log \frac{\Gamma(\alpha_{k,i})}{\Gamma(\alpha_{k,i} + 1/2)}] +$$
$$\lambda \sum_{k} |\alpha_{k,i} - 1| + \rho \mathcal{L}_{\text{rank}} \tag{6.10}$$

where $\epsilon_{k,i} = (y_{k,i} - \gamma_{k,i})^2/(2\beta_{k,i})$, Γ is the gamma function, λ (e.g., 0.01) regularizes evidence strength, ρ (e.g., 0.1) weights the ranking loss, and $\mathcal{L}_{\text{rank}}$ enforces ordering consistency (e.g., via margin-based pairwise loss on makespan predictions).

6.4.5 Training Procedure

GraphPerf-RT is trained using a multi-stage procedure designed to handle the challenges of multi-task learning with uncertainty quantification, building on the data preprocessing in Section 6.3.3 (e.g., z-scoring per device, outlier filtering via MAD on residuals, 60/20/20 splits stratified by benchmark-input-mask-DVFS to avoid leakage):

Stage 1: Feature Learning. The model is first trained to learn effective representations using a standard multi-task regression loss (e.g., MSE per target, weighted by inverse variance), focusing on prediction accuracy without uncertainty quantification. Optimizer: AdamW with learning rate 10^{-3}, batch size 32–128 (depending on graph size), epochs 50–100, with early stopping on validation MAE.

Stage 2: Evidential Training. The model is then fine-tuned using the evidential loss function $\mathcal{L}$, enabling uncertainty quantification while maintaining prediction accuracy. Learning rate reduced to 10^{-4}, with gradient clipping (norm 1.0) to handle NIG sensitivity, epochs 20–50.

Stage 3: Calibration. Finally, the model undergoes calibration using temperature scaling or isotonic regression on held-out data to ensure that uncertainty estimates are well-calibrated across different performance metrics and operating conditions (e.g., expected calibration error < 0.05). This involves optimizing a post-hoc scalar per target to align predicted confidences with empirical accuracies.

6.4.6 Training Procedure

Three stages: (i) feature learning with multi-task regression (AdamW, 10^{-3}, batch 32–128, early stopping); (ii) evidential fine-tuning (lr 10^{-4}, grad-clip 1.0); (iii) calibration via temperature scaling or isotonic regression (ECE < 0.05). Preprocessing follows Section 6.3.3 (per-device z-score, MAD outlier filter, stratified 60/20/20). Regularization: graph edge dropout ($p = 0.1$–0.2), small feature noise ($\sigma = 0.05$), multi-scale graphs, mild device augmentation (e.g., ±5% DVFS tables), and scheduling-aware ranking losses. This yields fast inference, cross-device transfer, and useful uncertainties for downstream schedulers and RL.

6.5 Experiments

6.5.1 Experimental Setup

Hardware platforms. We evaluate on three embedded ARM SoCs: **Jetson TX2** (Denver 2 + Cortex-A57; userspace DVFS; multi-zone thermals), **Jetson Nano** (quad Cortex-A57;

userspace DVFS; Nano-specific sensor layout), and **RubikPi** (8× Cortex-A72-class SBC with per-core userspace DVFS). DVFS indices are discovered from `scaling_available_frequencies`. Actual per-core Hz are read from `scaling_cur_freq`. Thermal zones are read from `/sys/devices/virtual/th` On RubikPi, power is sampled from `qcom-battmgr` or `hwmon` rails and integrated at 0.5 s to Joules.

We use two suites. **BOTS**: *alignment*, *fft*, *fib*, *floorplan*, *health*, *concom*, *knapsack*, *nqueens*, *sort*, *sparselu*, *strassen*, *uts*. **PolyBench**: 30 kernels across linear algebra, stencils, medley, and datamining (e.g., *2mm*, *3mm*, *gemm*, *atax*, *bicg*, *covariance*, *cholesky*, *jacobi-2d*, *seidel-2d*). In total, 42 distinct benchmarks. We evaluate multiple inputs, core counts (1..8), and a small set of evenly spaced DVFS index levels per platform. Both sequential and parallel run modes are used. **Client–server profiler.** A lightweight client on the device sets

Table 6.1: Overall performance across all platforms and benchmarks. Lower is better for RMSE/MAE/MAPE. Higher is better for R^2/Spearman.

Model	**RMSE**	**MAE**	**MAPE**	R^2	**Spearman**
Linear Regression	2.84	2.12	34.2%	0.67	0.71
Random Forest	2.31	1.78	28.5%	0.74	0.78
MLP	1.95	1.45	23.1%	0.81	0.83
GCN	1.67	1.23	19.8%	0.86	0.87
Het. Graph Trans.	1.52	1.11	17.3%	0.88	0.89
GraphPerf-RT	**0.53**	**0.38**	**7.4%**	**0.97**	**0.95**

the userspace governor, brings selected cores online, applies the assigned per-core DVFS indices f, launches the app via a real-time wrapper (`real_time_task`) without `sudo`, parses perf-like counters (`cycles`, `instructions`, `cache_misses`, `branch_misses`, `task_clock`,

`cpu_clock`), samples power rails, records thermal zones before/after, and reports observed per-core frequencies. A host server orchestrates action tuples $a = (m, f, p)$, performs a warm-up run, and logs one CSV row per run with a unified schema across boards: timestamp, iteration, run_mode, `frequency_combination` (assigned indices), `frequencies_read` (Hz per core), `num_cores`, `cores_str`, input, `time_elapsed`, rail energy/power, perf counters, thermal_zone* before/after, and platform. This matches Sec. 6.3.3.

CFG/graph extraction. OpenMP variants are compiled with OMPi; ALF-llvm emits `*.ll`/`*.alf`. SWEET then emits DOT graphs (`cfg,cg,rsg,fsg,sgh`) via a two-stage script (discover start functions; emit per-function DOTs). We map ALF/WCT entities to OpenMP tasks, merge tiny chains, and attach topological encodings (depth, distance-to-sink, centralities), as in Sec. 6.3.2.

Preprocessing and splits. We de-duplicate identical (G, input, m, f) tuples. We filter outliers by MAD on residuals. Counter windows are aligned to execution phases. Features are z-scored per device; we also retain global scalers for transfer ablations. Train/val/test is 60/20/20 by *(benchmark, input, core mask, DVFS index)* to prevent configuration leakage. Test tuples are unseen.

6.5.2 Baselines and Metrics

Baselines. (1) Linear Regression, (2) Random Forest, (3) MLP (tabular; no graph), (4) GCN (homogeneous message passing), (5) Heterogeneous Graph Transformer (hetero edges; no evidential heads). All models use identical features/splits and tune only on validation.

Prediction metrics. RMSE, MAE, MAPE, and R^2.

Ranking metrics. Spearman, Kendall's τ, NDCG@k (k=5,10).

Uncertainty metrics. ECE and MCE with reliability plots; sharpness (mean predictive interval width). All calibration is computed per-target and macro-averaged across platforms/benchmarks.

6.5.3 Results and Analysis

Overall prediction and ranking. Table 6.1 reports aggregate results across platforms and benchmarks. GraphPerf-RT achieves the best error and ranking. Structure + semantics + device context reduce error versus tabular baselines; heterogeneous message passing improves over GCN.

Platform-specific analysis. Accuracy is consistent on TX2, Nano, and RubikPi. TX2 benefits from richer DVFS levels and dense thermal sensing; Nano remains competitive despite fewer cores and different sensor naming; RubikPi shows stable estimates in both sequential and parallel modes. Per-device z-scoring + device broadcast (Sec. 6.3) helps cross-platform stability.

Uncertainty calibration. Evidential heads yield calibrated intervals (Table 6.2). Reliability curves track the identity line across targets. Sharpness remains adequate for planning and gating (Sec. 6.3).

Ablation. Removing heterogeneous edges ($E_{TR}/E_{RR}/E_{RM}$) degrades accuracy and ranking. Dropping CFG-derived embeddings reduces generalization to unseen inputs. Replacing the hetero-GAT with a homogeneous GCN increases error and weakens NDCG@k. Removing evidential learning preserves mean accuracy but harms calibration and increases unsafe proposals

Table 6.2: Uncertainty calibration summary (lower ECE/MCE is better; higher reliability is better).

Metric	ECE	MCE	Reliability	Sharpness
Makespan	0.043	0.089	0.956	0.234
Energy	0.038	0.076	0.962	0.198
Cache Misses	0.051	0.102	0.949	0.267
Branch Misses	0.047	0.094	0.953	0.251

after gating.

Computational efficiency. Inference is millisecond-scale on-device. With $L \leq 6$, $d \leq 128$, and modest $|E|$, the cost per layer is $O(H\,|E|\,d)$. Model memory is tens of MB, fitting real-time selection on all boards.

Cross-platform generalization. Train on two platforms, test on the third: accuracy degrades modestly but remains usable. Device broadcast and per-device normalization mitigate domain shift. Ranking quality for makespan remains high, preserving practical scheduler performance.

6.5.4 Integration with Scheduling and RL

At runtime, we enumerate feasible $\mathcal{A}$ under availability and thermal caps, score candidates with GraphPerf-RT, and gate by epistemic uncertainty and predicted intervals:

$$\text{gate}(a) = (\text{Epi}_{\text{time}}(a) \leq \eta) \wedge (\text{PI}_{\text{time}}^{(1-\delta)}(a) \leq T_{\max}).$$

We rank by $\hat{y}_{\text{time}}$ and execute the top action. Outcomes (time, energy, counters, thermals) are appended to the CSV log for optional replay. This provides cheap, hardware-grounded rollouts

for Dyna-style planning and future MARL.

6.5.5 Discussion

Insights. Structure + semantics + device context matter. Heterogeneous message passing and device broadcast improve accuracy and transfer. Evidential learning enables safe rollout filtering and better runtime choices.

Limitations. APIs and rail naming vary across boards; normalization can be hardened. Thermal transients in long horizons are under-modeled. Extending to CUDA/OpenCL and richer memory/IO nodes is future work.

Broader impact. Accurate, uncertainty-aware surrogates improve DVFS/core scheduling on embedded systems, reducing energy and thermal violations while preserving performance. The unified schema and logging pipeline enable reproducible comparisons and future zero-/few-shot transfer studies.

6.6 Conclusion

We introduced GraphPerf-RT, a fast, uncertainty-aware surrogate for OpenMP workloads on heterogeneous embedded SoCs that fuses ALF/EFG-derived task-DAG topology, CFG-informed code semantics, and device/runtime context into a heterogeneous GNN with evidential multi-task heads; the end-to-end pipeline (OMPi+ALF-llvm, SWEET graphs, unified client–server logging) yields calibrated predictions and rankings for makespan, energy, and microarchitectural counters, achieves state-of-the-art accuracy and strong calibration across Jetson TX2, Jetson Nano, and RubikPi on BOTS and PolyBench, and runs at millisecond scale with a small memory footprint to enable on-device selection and Dyna-style planning.

CHAPTER 7 CONCLUSION

This dissertation has presented a comprehensive AI-driven framework for energy- and thermal-aware scheduling in heterogeneous multicore embedded systems. By integrating hierarchical multi-agent reinforcement learning, statistical learning, distribution-aware flow matching, model-based reinforcement learning, and graph neural network-driven performance modeling, the framework addresses critical challenges in scheduling parallel OpenMP DAG workloads, task-to-core allocation, data efficiency, and dynamic voltage and frequency scaling (DVFS).

7.1 Summary of Contributions

The dissertation makes five interconnected contributions that together form a unified approach to energy-efficient embedded system scheduling:

HiDVFS: Hierarchical Multi-Agent Reinforcement Learning for DVFS (Chapter 2). The HiDVFS framework introduces a scalable three-agent design (Profiler, Thermal, Priority) that reduces action space complexity from $O(m^n)$ to $O(m \times n)$, enabling efficient scheduling for high-dimensional multicore systems [10]. Evaluated on the NVIDIA Jetson TX2, HiDVFS achieves a 31.7% reduction in energy consumption and a 34.1% improvement in makespan compared to state-of-the-art methods [64, 53]. The D3QN-based approach with a predictive reward function effectively handles irregular workloads while mitigating issues such as branch misses [5].

Statistical Learning for Task-to-Core Allocation (Chapter 3). The hybrid statistical learning framework combines Random Forest, backward stepwise selection, and Pearson cor-

relation analysis to identify critical scheduling features. By reducing the feature space (from 72 to 39 predictors for profiler data, 31 to 21 for temperature), the approach achieves a 10% energy reduction and 5°C temperature decrease across Intel Core i7, Xeon, and Jetson TX2 platforms [44]. Assigning tasks to thermally uncorrelated cores effectively mitigates hotspots, with bootstrapping reducing MSE by 61.6% over baselines [43].

ZeroDVFS: Model-Based Multi-Agent Reinforcement Learning (Chapter 4). ZeroDVFS extends the HMARL framework with model-based reinforcement learning for real-time DVFS, leveraging pre-trained environment models to simulate system dynamics and improve sample efficiency. The framework achieves a 7x improvement in thermal prediction accuracy and enables zero-shot transfer across heterogeneous platforms without additional training. By supporting continuous action spaces through algorithms such as DDPG and PPO, ZeroDVFS enables fine-grained frequency control for optimizing both makespan and thermal reliability.

Distribution-Aware Flow Matching for Data Augmentation (Chapter 5). The flow matching approach addresses data scarcity in few-shot reinforcement learning for DVFS by generating synthetic data through latent space bootstrapping and Random Forest-based feature weighting [65]. This method improves frame rates by 30% in data-limited settings while preserving correlations and aligning with statistical principles [41]. The technique is particularly valuable for embedded systems where collecting real-world profiling data is costly and time-consuming [80].

GraphPerf-RT: Graph-Driven Performance Modeling (Chapter 6). GraphPerf-RT introduces a graph neural network-driven approach to performance prediction and code opti-

mization. By analyzing control flow graphs extracted from OpenMP code, the model predicts execution characteristics and selects optimal code variations (tied, untied, serial) for minimal makespan. The integration of graph-based code representation with hardware-aware performance modeling enables intelligent scheduling decisions across diverse platforms.

7.2 Integrated Framework and Validation

These five contributions form a cohesive workflow: statistical feature selection (Random Forest, OLS) drives task allocation decisions; flow matching augments training data for data-scarce environments; hierarchical D3QN agents optimize DVFS and scheduling in real-time; model-based learning enables zero-shot transfer across platforms; and graph-driven analysis guides code optimization for parallel workloads. Validated on the BOTS benchmark suite across NVIDIA Jetson TX2, Intel Core i7, and Intel Xeon platforms, the unified framework consistently outperforms Linux governors and existing reinforcement learning schedulers that lack per-core thermal awareness [17, 30].

7.3 Broader Impact

The framework advances embedded systems across multiple application domains including IoT, automotive, and mobile computing. Key impacts include:

- **Energy Efficiency**: Significant reductions in energy consumption (up to 31.7%) extend battery life for mobile and IoT devices while reducing operational costs for data centers.
- **Thermal Management**: Proactive thermal control reduces hotspot formation and cooling requirements (approximately $3 per watt [35]), improving system reliability and component longevity.

- **Platform Generalizability**: Zero-shot transfer capabilities enable deployment across heterogeneous hardware without platform-specific retraining, reducing development costs and time-to-deployment.

- **Data Efficiency**: Flow matching-based data augmentation enables effective learning in resource-constrained environments where profiling data collection is limited.

- **Code Optimization**: Graph-driven performance modeling automates the selection of optimal code variations, reducing manual tuning effort while improving execution efficiency.

7.4 Future Directions

Several promising directions emerge from this work for future research:

Unified Continuous Control: Extending the framework to seamlessly integrate continuous action spaces across all agents would enable finer-grained control over frequency scaling and resource allocation [8].

Federated Learning: Distributing training across multiple embedded devices would improve scalability while preserving data privacy, particularly relevant for IoT deployments [13].

Meta-Learning Adaptation: Incorporating meta-learning techniques such as MAML would enable rapid adaptation to new platforms and workloads with minimal fine-tuning [37].

Hardware Acceleration: Leveraging specialized hardware (NPUs, TPUs) for inference would reduce computational overhead, enabling deployment on more resource-constrained devices.

Neuromorphic Vision Workloads: Extending validation to neuromorphic vision applications for autonomous systems would demonstrate applicability to emerging low-latency, energy-critical domains [103].

Cloud-Edge Integration: Adapting the framework for distributed cloud-edge systems would address the growing need for energy-efficient computing across heterogeneous infrastructure.

In conclusion, this dissertation has demonstrated that integrating multiple AI-driven techniques—hierarchical reinforcement learning, statistical feature selection, generative data augmentation, model-based learning, and graph neural networks—provides a powerful and practical approach to energy- and thermal-aware scheduling in heterogeneous multicore systems. The validated framework offers significant improvements over existing methods and establishes a foundation for continued advancement in sustainable, high-performance embedded computing.

REFERENCES

[1] P. Abbeel and A. Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*, page 1, 2004.

[2] M. Abdeyazdan, S. Parsa, and A. M. Rahmani. Task graph pre-scheduling, using nash equilibrium in game theory. *The Journal of Supercomputing*, 64:177–203, 2013.

[3] S. Adams, T. Cody, and P. A. Beling. A survey of inverse reinforcement learning. *Artificial Intelligence Review*, 55(6):4307–4346, 2022.

[4] I. Ahmad, S. Ranka, and S. U. Khan. Using game theory for scheduling tasks on multi-core processors for simultaneous optimization of performance and energy. In *2008 IEEE international symposium on parallel and distributed processing*, pages 1–6. IEEE, 2008.

[5] R. Ahmed, P. Ramanathan, and K. K. Saluja. Necessary and sufficient conditions for thermal schedulability of periodic real-time tasks under fluid scheduling model. *ACM Transactions on Embedded Computing Systems (TECS)*, 15(3):1–26, 2016.

[6] A. Amini, W. Schwarting, A. Soleimany, and D. Rus. Deep evidential regression. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 14927–14937, 2020.

[7] C. Angermueller, D. Dohan, D. Belanger, R. Deshpande, K. Murphy, and L. Colwell.

Model-based reinforcement learning for biological sequence design. In *International conference on learning representations*, 2019.

[8] S. Arora and P. Doshi. A survey of inverse reinforcement learning: Challenges, methods and progress. *Artificial Intelligence*, 297:103500, 2021.

[9] P. Auer, T. Jaksch, and R. Ortner. Near-optimal regret bounds for reinforcement learning. *Advances in neural information processing systems*, 21, 2008.

[10] H. Aydin and Q. Yang. Energy-aware partitioning for multiprocessor real-time systems. In *Proceedings International Parallel and Distributed Processing Symposium*, pages 9–pp. IEEE, 2003.

[11] S. Bavikadi, A. Dhavlle, A. Ganguly, A. Haridass, H. Hendy, C. Merkel, V. J. Reddi, P. R. Sutradhar, A. Joseph, and S. M. P. Dinakarrao. A survey on machine learning accelerators and evolutionary hardware platforms. *IEEE Design & Test*, 39(3):91–116, 2022.

[12] R. Bellman. Dynamic programming. *Science*, 153(3731):34–37, 1966.

[13] A. Bhuiyan, M. Pivezhandi, Z. Guo, J. Li, V. P. Modekurthy, and A. Saifullah. Precise scheduling of dag tasks with dynamic power management. In *35th Euromicro Conference on Real-Time Systems (ECRTS 2023)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2023.

[14] Z. Bo, Y. Qiao, C. Leng, H. Wang, C. Guo, and S. Zhang. Developing real-time scheduling policy by deep reinforcement learning. In *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 131–142. IEEE, 2021.

[15] O. Board. Openmp application program interface version 3.0. In *The OpenMP Forum, Tech. Rep*, 2008.

[16] A. Brauckmann, A. Goens, S. Ertel, and J. Castrillon. Compiler-based graph representations for deep learning models of code. In *Proceedings of the 29th International Conference on Compiler Construction (CC 2020)*, pages 201–211, 2020.

[17] D. Brodowski, N. Golde, R. J. Wysocki, and V. Kumar. Cpu frequency and voltage scaling code in the linux (tm) kernel. *Linux kernel documentation*, page 66, 2013.

[18] D. Brooks, R. P. Dick, R. Joseph, and L. Shang. Power, thermal, and reliability modeling in nanometer-scale microprocessors. *Ieee Micro*, 27(3):49–62, 2007.

[19] H. Cai, L. Zhu, and S. Han. Proxylessnas: Direct neural architecture search on target task and hardware. In *International Conference on Learning Representations (ICLR 2019)*, 2019.

[20] F. J. Cazorla, L. Kosmidis, E. Mezzetti, C. Hernandez, J. Abella, and T. Vardanega. Probabilistic worst-case timing analysis: Taxonomy and comprehensive survey. *ACM Computing Surveys (CSUR)*, 52(1):1–35, 2019.

[21] H. Charlesworth and G. Montana. Plangan: Model-based planning with sparse rewards

and multiple goals. *Advances in Neural Information Processing Systems*, 33:8532–8542, 2020.

[22] S. Chen, Z. Li, B. Yang, and G. Rudolph. Quantum-inspired hyper-heuristics for energy-aware scheduling on heterogeneous computing systems. *IEEE Transactions on Parallel and Distributed Systems*, 27(6):1796–1810, 2015.

[23] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pages 785–794, 2016.

[24] Y. Chen, G. Xie, and R. Li. Reducing energy consumption with cost budget using available budget preassignment in heterogeneous cloud computing systems. *IEEE Access*, 6:20572–20583, 2018.

[25] V. Chiley, I. Sharapov, A. Kosson, U. Koster, R. Reece, S. Samaniego de la Fuente, V. Subbiah, and M. James. Online normalization for training neural networks. *Advances in Neural Information Processing Systems*, 32, 2019.

[26] J. Choi, B.-J. Lee, and B.-T. Zhang. Multi-focus attention network for efficient deep reinforcement learning. *arXiv preprint arXiv:1712.04603*, 2017.

[27] F.-A. Croitoru, V. Hondru, R. T. Ionescu, and M. Shah. Diffusion models in vision: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(9):10850–10869, 2023.

[28] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. O'Boyle, and H. Leather. Programl: A graph-based program representation for data flow analysis and compiler optimizations. In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, volume 139 of *Proceedings of Machine Learning Research*, pages 2244–2253, 2021.

[29] R. I. Davis and L. Cucu-Grosjean. A survey of probabilistic timing analysis techniques for real-time systems. *LITES: Leibniz Transactions on Embedded Systems*, pages 1–60, 2019.

[30] S. M. P. Dinakarrao, A. Joseph, A. Haridass, M. Shafique, J. Henkel, and H. Homayoun. Application and thermal-reliability-aware reinforcement learning based multi-core power management. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 15(4):1–19, 2019.

[31] A. Duran, X. Teruel, R. Ferrer, X. Martorell, and E. Ayguade. Barcelona openmp tasks suite: A set of benchmarks targeting the exploitation of task parallelism in openmp. In *2009 international conference on parallel processing*, pages 124–131. IEEE, 2009.

[32] D. L. Eager and K. C. Sevcik. Performance bound hierarchies for queueing networks. *ACM Transactions on Computer Systems*, 1(2):99–115, 1983.

[33] S. Edgar and A. Burns. Statistical analysis of wcet for scheduling. In *Proceedings 22nd IEEE Real-Time Systems Symposium (RTSS 2001)(Cat. No. 01PR1420)*, pages 215–224. IEEE, 2001.

[34] B. Efron. Bootstrap methods: another look at the jackknife. In *Breakthroughs in statistics: Methodology and distribution*, pages 569–593. Springer, 1992.

[35] S. et al. Temperature-aware microarchitecture. *ACM SIGARCH*, 2003.

[36] Y. Fan and Y. Ming. Model-based reinforcement learning for continuous control with posterior sampling. In *International Conference on Machine Learning*, pages 3078–3087. PMLR, 2021.

[37] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.

[38] C. Florensa, Y. Duan, and P. Abbeel. Stochastic neural networks for hierarchical reinforcement learning. *arXiv preprint arXiv:1704.03012*, 2017.

[39] D. J. Foster and A. Rakhlin. Foundations of reinforcement learning and interactive decision making. *arXiv preprint arXiv:2312.16730*, 2023.

[40] V. Godbole, G. E. Dahl, J. Gilmer, C. J. Shallue, and Z. Nado. Deep learning tuning playbook, 2023. Version 1.0.

[41] T. Hastie, R. Tibshirani, J. H. Friedman, and J. H. Friedman. *The elements of statistical learning: data mining, inference, and prediction*, volume 2. Springer, 2009.

[42] R. Hebbar and A. Milenković. Pmu-events-driven dvfs techniques for improving en-

ergy efficiency of modern processors. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 7(1):1–31, 2022.

[43] S. Hosseinimotlagh, D. Enright, C. R. Shelton, and H. Kim. Data-driven structured thermal modeling for cots multi-core processors. In *2021 IEEE Real-Time Systems Symposium (RTSS)*, pages 201–213. IEEE, 2021.

[44] S. Hosseinimotlagh and H. Kim. Thermal-aware servers for real-time tasks on multi-core gpu-integrated embedded systems. In *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 254–266. IEEE, 2019.

[45] R. A. Howard. Dynamic prograreinforcementmming and markov processes. 1960.

[46] B. Hu, Z. Cao, and M. Zhou. Scheduling real-time parallel applications in cloud to minimize energy consumption. *IEEE Transactions on Cloud Computing*, 10(1):662–674, 2019.

[47] K. Huang, X. Jiang, X. Zhang, R. Yan, K. Wang, D. Xiong, and X. Yan. Energy-efficient fault-tolerant mapping and scheduling on heterogeneous multiprocessor real-time systems. *IEEE Access*, 6:57614–57630, 2018.

[48] E. Ipek, S. A. McKee, R. Caruana, B. R. de Supinski, and M. Schulz. Efficiently exploring architectural design spaces via predictive modeling. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII)*, pages 195–206, 2006.

[49] J. Jiang, W. Li, L. Pan, B. Yang, and X. Peng. Energy optimization heuristics for budget-constrained workflow in heterogeneous computing system. *Journal of Circuits, Systems and Computers*, 28(09):1950159, 2019.

[50] C. Jin, Z. Allen-Zhu, S. Bubeck, and M. I. Jordan. Is q-learning provably efficient? *Advances in neural information processing systems*, 31, 2018.

[51] A. Kassab, J.-M. Nicod, L. Philippe, and V. Rehn-Sonigo. Green power aware approaches for scheduling independent tasks on a multi-core machine. *Sustainable Computing: Informatics and Systems*, 31:100590, 2021.

[52] A. Kendall and Y. Gal. What uncertainties do we need in bayesian deep learning for computer vision? In *NeurIPS*, 2017.

[53] S. Kim, K. Bin, S. Ha, K. Lee, and S. Chong. ztt: Learning-based dvfs with zero thermal throttling for mobile devices. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*, pages 41–53, 2021.

[54] Y. G. Kim and C.-J. Wu. Autoscale: Energy efficiency optimization for stochastic edge inference using reinforcement learning. In *2020 53rd Annual IEEE/ACM international symposium on microarchitecture (MICRO)*, pages 1082–1096. IEEE, 2020.

[55] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 6402–6413, 2017.

[56] B. C. Lee and D. M. Brooks. Accurate and efficient regression modeling for microarchitectural performance and power prediction. *ACM SIGOPS Operating Systems Review*, 41(6):185–194, 2007.

[57] D. Lee, N. He, P. Kamalaruban, and V. Cevher. Optimization for reinforcement learning: From a single agent to cooperative agents. *IEEE Signal Processing Magazine*, 37(3):123–135, 2020.

[58] W. Y. Lee. Energy-saving dvfs scheduling of multiple periodic real-time tasks on multicore processors. In *2009 13th IEEE/ACM international symposium on distributed simulation and real time applications*, pages 216–223. IEEE, 2009.

[59] Y. C. Lee and A. Y. Zomaya. Energy conscious scheduling for distributed computing systems under different operating conditions. *IEEE Transactions on Parallel and Distributed Systems*, 22(8):1374–1381, 2010.

[60] D. Li and J. Wu. Energy-aware scheduling for frame-based tasks on heterogeneous multiprocessor platforms. In *2012 41st International Conference on Parallel Processing*, pages 430–439. IEEE, 2012.

[61] J. Li, K. Cheng, S. Wang, F. Morstatter, R. P. Trevino, J. Tang, and H. Liu. Feature selection: A data perspective. *ACM computing surveys (CSUR)*, 50(6):1–45, 2017.

[62] K. Li. Scheduling precedence constrained tasks with reduced processor energy on multiprocessor computers. *IEEE Transactions on Computers*, 61(12):1668–1681, 2012.

[63] K. Li. Energy and time constrained task scheduling on multiprocessor computers with discrete speed levels. *Journal of Parallel and Distributed Computing*, 95:15–28, 2016.

[64] C. Lin, K. Wang, Z. Li, and Y. Pu. A workload-aware dvfs robust to concurrent tasks for mobile devices. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, pages 1–16, 2023.

[65] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[66] D. Liu, S.-G. Yang, Z. He, M. Zhao, and W. Liu. Cartad: Compiler-assisted reinforcement learning for thermal-aware task scheduling and dvfs on multicores. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2021.

[67] A. Mahmood, S. A. Khan, F. Albalooshi, and N. Awwad. Energy-aware real-time task scheduling in multiprocessor systems using a hybrid genetic algorithm. *Electronics*, 6(2):40, 2017.

[68] S. Maity, R. Roy, A. Majumder, S. Dey, and A. R. Hota. Future aware dynamic thermal management in cpu-gpu embedded platforms. In *2022 IEEE Real-Time Systems Symposium (RTSS)*, pages 396–408. IEEE, 2022.

[69] C. Mendis, A. Renda, S. Amarasinghe, and M. Carbin. Ithemal: Accurate, portable and fast basic block throughput estimation using deep neural networks. In *Proceedings of the*

36th International Conference on Machine Learning (ICML 2019), pages 4505–4515, 2019.

[70] T. M. Moerland, J. Broekens, A. Plaat, C. M. Jonker, et al. Model-based reinforcement learning: A survey. *Foundations and Trends® in Machine Learning*, 16(1):1–118, 2023.

[71] O. Nachum, S. S. Gu, H. Lee, and S. Levine. Data-efficient hierarchical reinforcement learning. *Advances in neural information processing systems*, 31, 2018.

[72] A. Y. Ng, S. Russell, et al. Algorithms for inverse reinforcement learning. In *Icml*, volume 1, page 2, 2000.

[73] C. S. Pabla. Completely fair scheduler. *Linux Journal*, 2009(184):4, 2009.

[74] S. Pagani, P. S. Manoj, A. Jantsch, and J. Henkel. Machine learning for power, energy, and thermal management on multicore processors: A survey. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(1):101–116, 2018.

[75] J. Pallister, S. Kerrison, J. Morse, and K. Eder. Data dependent energy modeling for worst case energy consumption analysis. In *Proceedings of the 20th International Workshop on Software and Compilers for Embedded Systems*, pages 51–59, 2017.

[76] B. Peng, X. Li, J. Gao, J. Liu, K.-F. Wong, and S.-Y. Su. Deep dyna-q: Integrating planning for task-completion dialogue policy learning. *arXiv preprint arXiv:1801.06176*, 2018.

[77] Y. Qin, G. Zeng, R. Kurachi, Y. Li, Y. Matsubara, and H. Takada. Energy-efficient intra-task dvfs scheduling using linear programming formulation. *Ieee Access*, 7:30536–30547, 2019.

[78] T. Ramadan, A. Lahiry, and T. Z. Islam. Novel representation learning technique using graphs for performance analytics, 2024.

[79] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 1(2):3, 2022.

[80] I. Ratković, N. Bežanić, O. S. Ünsal, A. Cristal, and V. Milutinović. An overview of architecture-level power-and energy-efficient design techniques. *Advances in Computers*, 98:1–57, 2015.

[81] S. Reddy, A. D. Dragan, and S. Levine. Sqil: Imitation learning via reinforcement learning with sparse rewards. *arXiv preprint arXiv:1905.11108*, 2019.

[82] F. Reghenzani, G. Massari, and W. Fornaciari. Probabilistic-wcet reliability: Statistical testing of evt hypotheses. *Microprocessors and Microsystems*, 77:103135, 2020.

[83] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans, et al. Photorealistic text-to-image diffusion models with deep language understanding. *Advances in neural information processing systems*, 35:36479–36494, 2022.

[84] A. Saifullah, S. Fahmida, V. P. Modekurthy, N. Fisher, and Z. Guo. Cpu energy-aware

parallel real-time scheduling. In *32nd Euromicro Conference on Real-Time Systems (ECRTS 2020)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.

[85] H. Sasaki, Y. Ikeda, M. Kondo, and H. Nakamura. An intra-task dvfs technique based on statistical analysis of hardware events. In *Proceedings of the 4th international conference on Computing frontiers*, pages 123–130, 2007.

[86] P. Sedgwick. Pearson's correlation coefficient. *Bmj*, 345, 2012.

[87] M. Sensoy, L. Kaplan, and M. Kandemir. Evidential deep learning to quantify classification uncertainty. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, pages 3179–3189, 2018.

[88] U. Sethi. Learning energy-aware transaction scheduling in database systems. Master's thesis, University of Waterloo, 2021.

[89] E. A. Shao, L. Z. M. Pan, et al. Tensor program optimization with MetaSchedule. In *MLSys*, 2022.

[90] M. Shekarisaz, L. Thiele, and M. Kargahi. Automatic energy-hotspot detection and elimination in real-time deeply embedded systems. In *2021 IEEE Real-Time Systems Symposium (RTSS)*, pages 97–109. IEEE, 2021.

[91] H. Shen, J. Lu, and Q. Qiu. Learning based dvfs for simultaneous temperature, performance and energy management. In *Thirteenth International Symposium on Quality Electronic Design (ISQED)*, pages 747–754. IEEE, 2012.

[92] X. Shi, Y. Wang, M. Li, Y. Liu, J. Huan, and Z. Zhang. Predicting execution time of computer programs using sparse polynomial regression. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS 2017)*, pages 883–891, 2017. (Note: This is a substitute reference as the exact 2022 citation could not be verified).

[93] G. Singla, G. Kaur, A. K. Unver, and U. Y. Ogras. Predictive dynamic thermal and power management for heterogeneous mobile platforms. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 960–965. IEEE, 2015.

[94] J. Sun, N. Guan, J. Sun, and Y. Chi. Calculating response-time bounds for openmp task systems with conditional branches. In *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 169–181. IEEE, 2019.

[95] G. Taheri, A. Khonsari, R. Entezari-Maleki, and L. Sousa. A hybrid algorithm for task scheduling on heterogeneous multiprocessor embedded systems. *Applied Soft Computing*, 91:106202, 2020.

[96] X. Tang and Z. Fu. Cpu–gpu utilization aware energy-efficient scheduling algorithm on heterogeneous computing systems. *IEEE Access*, 8:58948–58958, 2020.

[97] J. M. Tomczak. *Deep Generative Modeling*. Springer Cham, 2024.

[98] F. M. M. ul Islam and M. Lin. Hybrid dvfs scheduling for real-time systems based on reinforcement learning. *IEEE Systems Journal*, 11(2):931–940, 2015.

[99] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[100] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[101] M. J. Walker, S. Bischoff, S. Diestelhorst, G. V. Merrett, and B. M. Al-Hashimi. Accurate and stable run-time power modeling for mobile and embedded cpus. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(12):2675–2687, 2018.

[102] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni. Generalizing from a few examples: A survey on few-shot learning. *ACM computing surveys (csur)*, 53(3):1–34, 2020.

[103] Y. Wang, W. Zhang, M. Hao, and Z. Wang. Online power management for multicores: A reinforcement learning based approach. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):751–764, 2021.

[104] Z. Wang and M. F. P. O'Boyle. Machine learning in compiler optimization. *Proceedings of the IEEE*, 106(11):1879–1901, 2018.

[105] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.

[106] Z. Wang, Z. Tian, J. Xu, R. K. Maeda, H. Li, P. Yang, Z. Wang, L. H. Duong, Z. Wang, and X. Chen. Modular reinforcement learning for self-adaptive energy efficiency optimization in multicore system. In *2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 684–689. IEEE, 2017.

[107] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.

[108] M. Wulfmeier, P. Ondruska, and I. Posner. Deep inverse reinforcement learning. *CoRR, abs/1507.04888*, 2015.

[109] G. Xie, J. Huang, Y. L. R. Li, and K. Li. System-level energy-aware design methodology towards end-to-end response time optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2019.

[110] G. Xie, J. Jiang, Y. Liu, R. Li, and K. Li. Minimizing energy consumption of real-time parallel applications using downward and upward approaches on heterogeneous systems. *IEEE Transactions on Industrial Informatics*, 13(3):1068–1078, 2017.

[111] G. Xie, H. Peng, J. Huang, R. Li, and K. Li. Energy-efficient functional safety design methodology using asil decomposition for automotive cyber-physical systems. *IEEE Transactions on Reliability*, 2019.

[112] G. Xie, X. Xiao, H. Peng, R. Li, and K. Li. A survey of low-energy parallel scheduling algorithms. *IEEE Transactions on Sustainable Computing*, 7(1):27–46, 2021.

[113] G. Xie, G. Zeng, X. Xiao, R. Li, and K. Li. Energy-efficient scheduling algorithms for real-time parallel applications on heterogeneous distributed embedded systems. *IEEE Transactions on Parallel and Distributed Systems*, 28(12):3426–3442, 2017.

[114] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhudinov, R. Zemel, and Y. Bengio. Show, attend and tell: Neural image caption generation with visual attention. In *International conference on machine learning*, pages 2048–2057. PMLR, 2015.

[115] L. Yan, J. Luo, and N. K. Jha. Combined dynamic voltage scaling and adaptive body biasing for heterogeneous distributed real-time embedded systems. In *ICCAD-2003. International Conference on Computer Aided Design (IEEE Cat. No. 03CH37486)*, pages 30–37. IEEE, 2003.

[116] A. Yeganeh-Khaksar, M. Ansari, S. Safari, S. Yari-Karin, and A. Ejlali. Ring-dvfs: Reliability-aware reinforcement learning-based dvfs for real-time embedded systems. *IEEE Embedded Systems Letters*, 13(3):146–149, 2020.

[117] Z. Yu, P. Machado, A. Zahid, A. M. Abdulghani, K. Dashtipour, H. Heidari, M. A. Imran, and Q. H. Abbasi. Energy and performance trade-off optimization in heterogeneous computing via reinforcement learning. *Electronics*, 9(11):1812, 2020.

[118] Y. Yun, E. J. Hwang, and Y. H. Kim. Adaptive genetic algorithm for energy-efficient

task scheduling on asymmetric multiprocessor system-on-chip. *Microprocessors and Microsystems*, 66:19–30, 2019.

[119] S. Zhai et al. Transferable learning of scheduling policies for tensor programs. In *ICML*, 2023.

[120] S. Zhai et al. Transformer latent models for cross-device schedule transfer. In *NeurIPS*, 2024.

[121] Z. Zhang, Y. Zhao, H. Li, C. Lin, and J. Liu. Dvfo: Learning-based dvfs for energy-efficient edge-cloud collaborative inference. *IEEE Transactions on Mobile Computing*, 2024.

[122] L. Zheng, C. Jia, M. Pan, Z. Wang, Y. Han, Y. Li, X. Lian, Y. Chen, C. H. Yu, J. Yang, and T. Chen. Ansor: Generating high-performance tensor programs for deep learning. In *OSDI*, 2020.

[123] J. Zhou, J. Yan, K. Cao, Y. Tan, T. Wei, M. Chen, G. Zhang, X. Chen, and S. Hu. Thermal-aware correlated two-level scheduling of real-time tasks with reduced processor energy on heterogeneous mpsocs. *Journal of Systems Architecture*, 82:1–11, 2018.

[124] T. Zhou and M. Lin. Deadline-aware deep-recurrent-q-network governor for smart energy saving. *IEEE Transactions on Network Science and Engineering*, 9(6):3886–3895, 2021.

[125] D. Zhu, R. Melhem, and D. Mossé. The effects of energy management on reliability

in real-time embedded systems. In *IEEE/ACM International Conference on Computer Aided Design, 2004. ICCAD-2004.*, pages 35–40. IEEE, 2004.

[126] Y. Zhu et al. ROLLER: Fast and accurate tensor program optimization. In *NeurIPS*, 2022.

[127] C. Zhuo, D. Gao, Y. Cao, T. Shen, L. Zhang, J. Zhou, and X. Yin. A dvfs design and simulation framework using machine learning models. *IEEE Design & Test*, 2021.

[128] B. D. Ziebart, A. L. Maas, J. A. Bagnell, A. K. Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.

ABSTRACT

DATA-EFFICIENT AI-GUIDED ENERGY- AND THERMAL-AWARE SCHEDULING ON HETEROGENEOUS MULTICORE SYSTEMS

by

MOHAMMAD PIVEZHANDI

December 2025

Advisor: Dr. Abusayeed Saifullah

Major: Computer Science

Degree: Doctor of Philosophy

The surging demand for energy-efficient computing in heterogeneous multicore embedded systems, driven by applications such as IoT devices, autonomous vehicles, and mobile platforms, presents formidable challenges arising from escalating static leakage power, intricate per-core thermal dynamics, and the complexity of high-dimensional action spaces in parallel workloads such as Directed Acyclic Graphs (DAGs). Conventional approaches, including Linux governors, fail to adequately address irregular task execution patterns, costly runtime profiling requirements, and data scarcity for few-shot learning, thereby constraining scalability across diverse platforms.

This dissertation presents a comprehensive AI-driven framework that integrates hierarchical multi-agent reinforcement learning (HMARL), statistical learning, distribution-aware flow matching, model-based reinforcement learning, and graph neural network-driven performance

modeling to address these challenges. The framework comprises five interconnected contributions. First, HiDVFS introduces a hierarchical multi-agent reinforcement learning framework for energy- and thermal-aware scheduling of parallel OpenMP DAG workloads, achieving a 31.7% reduction in energy consumption and a 34.1% improvement in task completion time on the NVIDIA Jetson TX2 platform. Second, a hybrid statistical learning approach for task-to-core allocation combines Random Forest, backward stepwise selection, and Pearson correlation analysis to identify critical scheduling features, reducing energy consumption by 10% and core temperature by 5°C while cutting thermal prediction error by 61.6% across embedded platforms. Third, ZeroDVFS develops a model-based multi-agent reinforcement learning framework for real-time dynamic voltage and frequency scaling (DVFS), leveraging pre-trained environment models to achieve a 7x improvement in thermal prediction accuracy with zero-shot transfer capabilities across heterogeneous platforms. Fourth, a distribution-aware flow matching method generates synthetic unstructured data for few-shot reinforcement learning, improving frame rates by 30% in DVFS applications while preserving statistical correlations in data-limited settings. Fifth, GraphPerf-RT presents a graph neural network-driven performance model that analyzes control flow graphs to predict execution characteristics and optimize OpenMP code variation selection for minimal makespan.

Validated on platforms including NVIDIA Jetson TX2, Intel Core i7, and Intel Xeon using the BOTS benchmark suite, the unified framework outperforms existing Linux governors and reinforcement learning schedulers that lack per-core thermal awareness. The contributions advance energy-efficient embedded systems for IoT, automotive, and mobile applications by

reducing thermal hotspots and cooling costs while enhancing system reliability and longevity.

AUTOBIOGRAPHICAL STATEMENT

Mohammad Pivezhandi received his Bachelor of Science degree in Electrical Engineering from the University of Tehran, Iran, in 2014, and his Master of Science degree in Electrical Engineering from the University of Tehran in 2017. He joined the Department of Computer Science at Wayne State University, Detroit, Michigan, in Fall 2019 to pursue his Doctor of Philosophy degree under the supervision of Dr. Abusayeed Saifullah.

His research interests include machine learning for embedded systems, reinforcement learning, dynamic voltage and frequency scaling (DVFS), thermal-aware scheduling, and energy-efficient computing on heterogeneous multicore platforms. During his doctoral studies, he developed novel frameworks for energy- and thermal-aware task scheduling using hierarchical multi-agent reinforcement learning, model-based policy learning, and graph neural network-driven performance modeling.

Mohammad has published his research in peer-reviewed conferences and journals in the areas of embedded systems, real-time computing, and machine learning. His work has been validated on multiple hardware platforms including NVIDIA Jetson TX2, Intel Core i7, and Intel Xeon processors.